

que indica cuántas veces ese registro es utilizado como fuente por las instrucciones que se están ejecutando. Si puede haber un máximo de, digamos, 15 instrucciones en ejecución en un momento dado, basta con un contador de 4 bits. Cuando se emite una instrucción, se incrementan las entradas del marcador correspondientes a sus registros operandos. Cuando se retira una instrucción, las entradas se decrementan.

El marcador también tiene un contador para cada registro que sigue la pista a los registros que se están usando como destinos. Puesto que sólo se permite una escritura a la vez, estos contadores pueden tener una capacidad de 1 bit. Las 16 columnas de la derecha de la figura 4-43 muestran el marcador.

En las máquinas reales, el marcador también sigue la pista al uso de las unidades funcionales, a fin de evitar emitir una instrucción para la cual no está disponible una unidad funcional. Para simplificar, supondremos que siempre está disponible una unidad funcional apropiada, por lo que no mostraremos las unidades funcionales en el marcador.

La primera línea de la figura 4-43 muestra I1 (instrucción 1) que multiplica R0 por R1 y coloca el resultado en R3. Puesto que ninguno de estos registros se está usando todavía, la instrucción se emite y el marcador se actualiza de modo que refleje que R0 y R1 se están leyendo y R3 se está escribiendo. Ninguna instrucción subsecuente podrá escribir en ninguno de éstos ni podrá leer R3 hasta que I1 se haya retirado. Puesto que esta instrucción es una multiplicación, terminará al final del ciclo 4. Los valores del marcador que se muestran en cada línea reflejan su estado después de emitirse la instrucción de esa línea. Las entradas en blanco son ceros.

Puesto que nuestro ejemplo es una máquina superescalar que puede emitir dos instrucciones por ciclo, se emite una segunda instrucción (I2) durante el ciclo 1. Ésta suma R0 y R2, y guarda el resultado en R4. Para ver si es posible emitir esta instrucción, se aplican las reglas siguientes:

1. Si se está escribiendo en cualquier operando, no emitir (dependencia RAW).
2. Si se está leyendo el registro del resultado, no emitir (dependencia WAR).
3. Si se está escribiendo en el registro de resultado, no emitir (dependencia WAW).

Ya vimos las dependencias RAW, que ocurren cuando una instrucción necesita usar como fuente un resultado que una instrucción anterior todavía no produce. Las otras dos dependencias son menos graves. Se trata básicamente de conflictos de recursos. En una **dependencia WAR** (escritura después de lectura, *Write After Read*), una instrucción está tratando de sobrescribir un registro que una instrucción anterior tal vez no haya terminado de leer todavía. Una **dependencia WAW** (escritura después de escritura, *Write After Write*) es similar. En muchos casos podemos evitar estas dependencias haciendo que la segunda instrucción coloque sus resultados en algún otro lugar (tal vez temporalmente). Si no existe ninguna de las tres dependencias anteriores, y la unidad funcional que se necesita está disponible, la instrucción se emite. En este caso, I2 usa un registro (R0) que está siendo leído por una instrucción pendiente, pero este traslape está permitido, por lo que I2 se emite. De forma similar, I3 se emite durante el ciclo 2.

Ahora llegamos a I4, que necesita usar R4. Por desgracia, vemos en la línea 3 que se está escribiendo en R4. Tenemos aquí una dependencia RAW, por lo que la unidad decodificadora

parará hasta que R4 esté disponible. Mientras está parada, la unidad deja de sacar instrucciones de la unidad de búsqueda. Cuando los buffers internos de la unidad de búsqueda se llenen, ésta dejará de prebuscar instrucciones.

Vale la pena señalar que la siguiente instrucción en el orden del programa, I5, no tiene conflictos con ninguna de las instrucciones pendientes; podría haberse decodificado y emitido si no fuera por el hecho de que este diseño requiere emitir las instrucciones en orden.

Veamos ahora qué sucede durante el ciclo 3. I2, al ser una suma (dos ciclos), termina al final del ciclo 3. Lamentablemente, no podemos retirarla (lo cual dejaría libre R4 para I4). ¿Por qué no? La razón es que este diseño también requiere retirar en orden. ¿Por qué? ¿Qué de malo podría tener realizar el almacenamiento en R4 ahora y marcar el registro como disponible?

La respuesta es sutil, pero importante. Supongamos que las instrucciones pudieran terminar en desorden. Si entonces ocurriera una interrupción, sería muy difícil guardar el estado de la máquina para poder restaurarlo después. En particular, no sería posible decir que todas las instrucciones hasta cierta dirección ya se ejecutaron y todas las instrucciones posteriores no. Esto se llama **interrupción precisa** y es una característica deseable de una CPU (Moudgill y Vassiliadis, 1996). El retiro en desorden hace que las interrupciones no sean precisas, y es por esto que algunas máquinas exigen que las instrucciones terminen en orden.

Retomando nuestro ejemplo, al final del ciclo 4 se pueden retirar las tres instrucciones pendientes, así que en el ciclo 5 ya es posible emitir por fin I4 junto con la recién decodificada I5. Cada vez que se retira una instrucción, la unidad decodificadora tiene que verificar si hay una instrucción en espera que ya sea posible emitir.

En el ciclo 6 I6 se para porque necesita escribir en R1 y R1 está ocupado. I6 por fin se inicia en el ciclo 9. Toda la secuencia de ocho instrucciones tarda 15 ciclos en completarse debido a muchas dependencias, aunque el hardware es capaz de emitir dos instrucciones en cada ciclo. Sin embargo, al examinar la columna *Emit.* de la figura 4-43 podemos ver que todas las instrucciones se emitieron en orden. De igual manera, la columna *Ret.* muestra que las instrucciones también se retiraron en orden.

Consideremos ahora un diseño alternativo: la ejecución en desorden. En este diseño, las instrucciones pueden emitirse en desorden y también retirarse en desorden. En la figura 4-44 se muestra la misma secuencia de ocho instrucciones, sólo que ahora permitiendo emisión y retiro en desorden.

La primera diferencia ocurre en el ciclo 3. Aunque I4 está parada, podemos decodificar y emitir I5 porque no está en conflicto con ninguna instrucción pendiente. Sin embargo, saltarse instrucciones causa un nuevo problema. Suponga que I5 hubiera utilizado un operando calculado por la instrucción que se pasó por alto, I4. Con el marcador actual, no nos habríamos dado cuenta de ello. Por consiguiente, tenemos que extender el marcador para seguir la pista a las escrituras realizadas por instrucciones pasadas por alto. Esto puede hacerse añadiendo un segundo mapa de bits, con un bit por registro, para contabilizar las escrituras efectuadas por instrucciones paradas. (Estos contadores no se muestran en la figura.) Ahora es necesario extender la regla para emitir instrucciones, a fin de prohibir la emisión de cualquier instrucción cuyo operando esté programado para escritura por una instrucción que era anterior pero se pasó por alto.

					Registros leídos								Registros escritos							
Cic.	#	Decodif.	Emit.	Ret.	0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7
1	1	R3=R0+R1	1		1	1										1				
	2	R4=R0+R2	2		2	1	1									1	1			
2	3	R5=R0+R1	3		3	2	1									1	1	1		
	4	R6=R1+R4	—		3	2	1									1	1	1		
3	5	R7=R1*R2	5		3	3	2									1	1	1		1
	6	S1=R0-R2	6	2	4	3	3									1	1	1		1
					3	3	2								1	1	1		1	
4	7	R3=R3-S1 S2=R4+R4	— 8		3	4	2		1					1		1	1	1		
					3	4	2		1					1		1	1	1		
					3	4	2		3					1		1	1	1		
					2	3	2		3						1	1	1	1		
5			6		2	1		3				1					1	1		
6			7			2	1	1	2				1		1	1		1	1	
7								1								1				
8								1								1				
9				7																

Figura 4-44. Funcionamiento de una CPU superescalar con emisión en desorden y terminación en desorden.

Examinemos otra vez I6, I7 e I8 en la figura 4-43. Vemos que I6 calcula un valor en R1 que es utilizado por I7. Sin embargo, también vemos que el valor nunca se vuelve a usar porque I8 sobrescribe R1. No existe una razón de peso para usar R1 como lugar donde retener el resultado de I6. Peor aún, la decisión de usar R1 como registro intermedio es pésima, aunque perfectamente razonable para un compilador o programador acostumbrado a la idea de ejecución secuencial sin traslape de instrucciones.

En la figura 4-44 introducimos una nueva técnica para resolver este problema: el **cambio de nombre de registros**. La unidad decodificadora inteligente cambia el uso de R1 en I6 (ciclo 3) e I7 (ciclo 4) a un registro secreto, S1, que el programador no ve. Ahora I6 puede emitirse junto con I5. Las CPU modernas suelen tener docenas de registros secretos que usan para cambiar el nombre de los registros. En muchos casos esta técnica puede eliminar las dependencias WAR y WAW.

En I8 podemos volver a usar el cambio de nombre de registros. Esta vez R1 cambia de nombre a S2 para que la suma pueda iniciarse antes de que R1 se desocupe, al final del ciclo 6. Si en este caso el resultado realmente tiene que estar en R1, siempre cabe la posibilidad de

copiar S2 otra vez ahí justo a tiempo. Mejor aún, todas las instrucciones futuras que lo necesiten pueden cambiar el nombre de su fuente al del registro donde en verdad está guardado. En todo caso, la suma de I8 logró efectuarse antes de este modo.

En muchas máquinas reales, el cambio de nombre está incorporado profundamente en la organización de los registros. Hay muchos registros secretos y una tabla que establece la correspondencia entre los registros que el programador ve y los registros secretos. Así, el verdadero registro que se está usando para, digamos, R0, se encuentra examinando la entrada 0 de esta tabla de correspondencia. Así, no existe un registro R0 real, sólo un vínculo entre el nombre R0 y uno de los registros secretos. Este vínculo cambia con frecuencia durante la ejecución para evitar dependencias.

Si examina la cuarta columna de la figura 4-44, verá que las instrucciones no se emitieron en orden. Tampoco se retiraron en orden. La conclusión de este ejemplo es sencilla: gracias a la ejecución en desorden y el cambio de nombre de registros pudimos reducir el tiempo de ejecución casi a la mitad.

4.5.4 Ejecución especulativa

En la sección anterior presentamos el concepto de reordenar las instrucciones para mejorar el desempeño. Aunque no lo mencionamos explícitamente, lo que buscábamos ahí era reordenar las instrucciones dentro de un solo bloque básico. Es momento de examinar este punto con más detenimiento.

Los programas de computadora se pueden dividir en **bloques básicos**, cada uno de los cuales consiste en una secuencia lineal de código con un punto de entrada arriba y uno de salida abajo. Un bloque básico no contiene estructuras de control (como enunciados if o enunciados while), así que su traducción a lenguaje de máquina no contiene ramificaciones. Los bloques básicos se conectan mediante enunciados de control.

Un programa que adopta esta forma se puede representar como grafo dirigido, como se ilustra en la figura 4-45. Ahí calculamos la suma de los cubos de los enteros pares e impares hasta cierto límite y los acumulamos en *sumapar* y *sumaimpar*, respectivamente. Dentro de cada bloque básico, las técnicas de reordenamiento de la sección anterior funcionan de maravilla.

El problema es que casi todos los bloques básicos son cortos y no contienen suficiente paralelismo como para aprovecharlos de forma eficaz. Por consiguiente, nuestro siguiente paso es permitir que el reordenamiento cruce las fronteras de los bloques básicos en un intento por llenar todas las ranuras de emisión. Las mayores ganancias se obtienen cuando una operación que podría ser lenta se puede adelantar en el grafo e iniciarse antes. Ésta podría ser una instrucción LOAD, una operación de punto flotante o incluso el inicio de una cadena de dependencias larga. La acción de adelantar código que inicia después de una ramificación se denomina **levantamiento de código**.

Imagine que en la figura 4-45 todas las variables se guardan en registros excepto *sumapar* y *sumaimpar* (por falta de registros). Podría ser razonable adelantar sus instrucciones LOAD al principio del ciclo, antes de calcular *k*, a fin de iniciarlas antes y que los valores cargados estén disponibles cuando se necesiten. Desde luego, sólo uno de ellos se necesitará en cada iteración, y el otro LOAD se desperdiciaría, pero si la caché y la memoria usan filas de

```

sumapar = 0;
sumaimpar = 0;
i = 0;
while (i < limit) {
    k = i * i * i;
    if ((i/2) * 2 == 0)
        sumapar = sumapar + k;
    else
        sumaimpar = sumaimpar + k;
    i = i + 1;
}

```

(a)

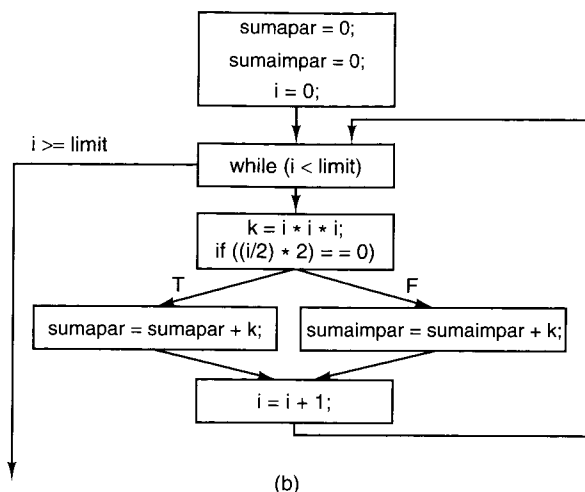


Figura 4-45. (a) Fragmento de programa. (b) El grafo de bloque básico correspondiente.

procesamiento y hay ranuras de emisión disponibles, podría valer la pena hacerlo. La ejecución de código antes de saber si va a necesitarse se llama **ejecución especulativa**. El uso de esta técnica requiere apoyo del compilador y del hardware, además de ciertas extensiones arquitectónicas. En casi todos los casos, reordenar las instrucciones más allá de las fronteras de un bloque básico rebasa la capacidad del hardware, por lo que el compilador debe cambiar de lugar las instrucciones explícitamente.

La ejecución especulativa introduce algunos problemas interesantes. Uno de ellos es que es indispensable que ninguna de las instrucciones especulativas tenga efectos irrevocables, porque después podría resultar que no debieron de haberse ejecutado. En la figura 4-45 no hay problema si buscamos *sumapar* y *sumaimpar*, y tampoco si efectuamos la suma tan pronto como tenemos *k* (aún antes del enunciado *if*), pero no sería correcto guardar los resultados en la memoria. En secuencias de código más complicadas, una forma común de evitar que el código especulativo sobrescriba registros antes de que se sepa si es lo que debe hacerse es cambiar el nombre de todos los registros de destino empleados por el código especulativo. De este modo, sólo se modificarán registros de borrador, y no habrá problema si después resulta que el código no se necesitaba. Si el código es necesario, los registros de borrador se copian en los verdaderos registros de destino. Como podrá imaginar, el manejo de contador para llevar la contabilidad de todo esto no es sencillo, pero si se cuenta con suficiente hardware, es factible.

Sin embargo, el código especulativo introduce otro problema que no puede resolverse cambiando el nombre de los registros. ¿Qué sucede si una instrucción ejecutada especulativamente causa una excepción? Un ejemplo doloroso, aunque no fatal, es una instrucción **LOAD** que causa un fallo de caché en una máquina con tamaño de línea de caché grande (digamos, 256 bytes) y una memoria mucho más lenta que la CPU y la caché. Si un **LOAD**

que realmente se necesita para en seco la máquina durante muchos ciclos mientras la línea de caché se está cargando, pues ni modo: así es la vida; la palabra era necesaria. Por otra parte, parar la máquina para traer una palabra que resulta innecesaria es contraproducente. Un número excesivo de tales “optimaciones” puede hacer que la CPU sea más lenta que si no las tuviera. (Si la máquina tiene memoria virtual, que veremos en el capítulo 6, un **LOAD** especulativo podría incluso causar un fallo de página, que requiere una operación de disco para traer la página requerida. Los fallos de página falsos pueden arruinar el desempeño, así que es importante evitarlos.)

Una solución que implementan varias máquinas modernas es tener una instrucción especial de carga especulativa que trata de traer la palabra de la caché, pero si no está ahí simplemente se da por vencida. Si el valor en realidad se necesita y está ahí, podrá usarse, pero si no está el hardware deberá ir por él en ese momento. Si resulta que el valor no era necesario, no se habrá pagado ningún precio por el fallo de caché.

El siguiente enunciado ilustra una situación mucho peor:

if ($x > 0$) $z = y/x$;

donde *x*, *y* y *z* son variables de punto flotante. Suponga que todas las variables se buscan y se colocan en registros por adelantado y que la división de punto flotante (lenta) se levanta de modo que se inicie antes de la prueba *if*. El problema es que *x* es 0 y el error de división entre cero hace que el programa termine. El resultado neto es que la especulación ha hecho que un programa correcto falle. Peor aún, el programador incluyó código explícito para evitar esta situación, y de todos modos ocurrió. No es probable que el programador esté muy contento.

Una posible solución es tener versiones especiales de las instrucciones que podrían causar excepciones. Además, se añade a cada registro un bit, llamado **bit venenoso**. Cuando una instrucción especulativa especial falla, en lugar de causar un error asigna 1 al bit venenoso del registro de resultado. Si posteriormente una instrucción normal toca ese registro, ocurrirá el error (como debería). En cambio, si el resultado nunca se usa, el bit venenoso tarde o temprano se pone en 0 y no habrá pasado nada.

4.6 EJEMPLOS DEL NIVEL DE MICROARQUITECTURA

En esta sección presentaremos breves ejemplos de tres procesadores de vanguardia, mostrando cómo usan los conceptos que exploramos en este capítulo. La exposición será breve por necesidad, pues las máquinas reales tienen una complejidad enorme y contienen millones de compuertas. Los ejemplos son los mismos que hemos estado usando hasta ahora: Pentium II, UltraSPARC II y picoJava II.

4.6.1 La microarquitectura de la CPU Pentium II

El Pentium II representa el extremo superior de la familia de CPU de Intel. Maneja operandos y operaciones aritméticas de 32 bits, así como operaciones de punto flotante de 64 bits. Además, maneja operandos y operaciones de 8 y 16 bits, lo cual es un legado de anteriores

procesadores de la familia. El Pentium II puede direccionar hasta 64 GB de memoria y lee palabras de la memoria en grupos de 64 bits. En la figura 3-50 se ilustra un sistema Pentium II representativo.

Como vimos antes, y como se ilustra en la figura 3-43, el paquete SEC del Pentium II consiste en dos circuitos integrados: la CPU (que incluye las cachés de nivel 1 divididos) y la caché de nivel 2 unificada. En la figura 4-46 se muestran los componentes primarios de la CPU: las unidades de Búsqueda/Decodificación, Despacho/Ejecución y Retiro, que juntas actúan como una fila de procesamiento de alto nivel. Estas tres unidades se comunican a través de una reserva de instrucciones, un lugar donde se mantiene la información relacionada con instrucciones cuya ejecución todavía no termina. La información de la reserva de instrucciones se guarda en una tabla llamada **buffer de reordenamiento (ROB, ReOrder Buffer)**. En pocas palabras, la unidad de Búsqueda/Decodificación busca instrucciones y las divide en microoperaciones que guarda en el ROB. La unidad de Despacho/Ejecución toma microoperaciones del ROB y las ejecuta. La unidad de Retiro finaliza la ejecución de cada microoperación y actualiza los registros. Las instrucciones ingresan en el ROB en orden, se pueden ejecutar en desorden, pero se retiran en orden.

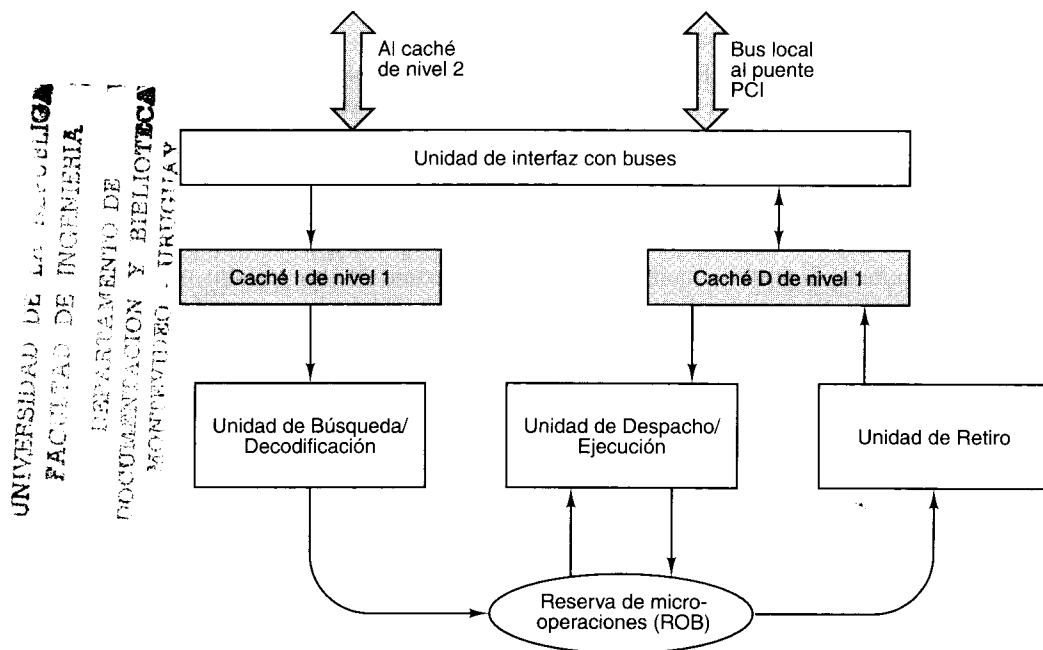


Figura 4-46. Microarquitectura del Pentium II.

La unidad de interfaz con buses se encarga de comunicarse con el sistema de memoria, tanto la caché de nivel 2 como la memoria principal. La caché de nivel 2 no está conectada al bus local, por lo que la unidad de interfaz con buses se encarga de buscar los datos de la

memoria principal a través del bus local y de cargar todas las cachés. El Pentium II usa el protocolo de coherencia de cachés MESI, que describiremos cuando lleguemos a los multiprocesadores en la sección 8.3.2.

La unidad de Búsqueda/Decodificación

La unidad de Búsqueda/Decodificación usa filas de procesamiento intensivamente, con siete etapas rotuladas IFU0 a ROB en la figura 4-47 (Las unidades de Despacho/Ejecución y de Retiro tienen otras cinco, lo que hace que las filas de procesamiento tengan 12 etapas en total.) Las instrucciones ingresan en la fila de procesamiento en la etapa IFU0, donde se cargan líneas de 32 bytes enteras de la caché I. Siempre que el buffer interno está vacío, se introduce otra línea de caché. El registro NEXT IP guía el proceso de búsqueda.

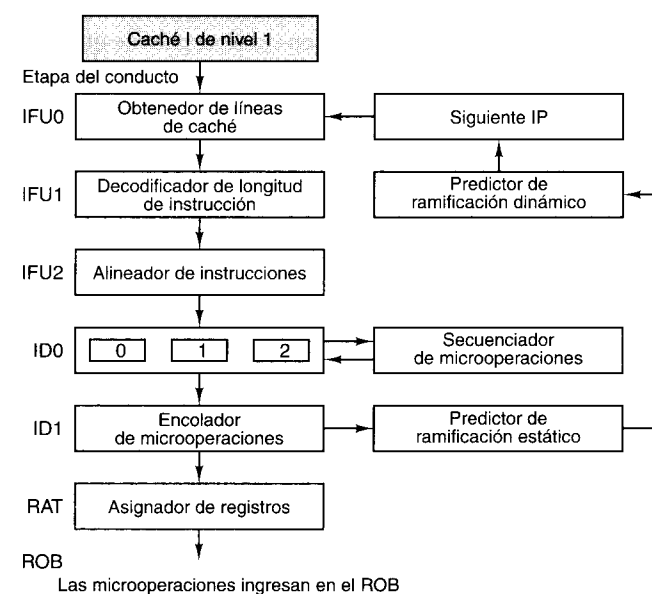


Figura 4-47. Estructura interna de la unidad de Búsqueda/Decodificación (simplificada).

Puesto que el conjunto de instrucciones Intel, conocido como **IA-32** tiene instrucciones de longitud variable con muchos formatos, la siguiente etapa de la fila de procesamiento, IFU1, analiza el flujo de bytes para localizar el principio de cada instrucción. Si es necesario, IFU1 puede examinar hasta 30 instrucciones IA-32 por adelantado. Lo malo es que adelantarse tanto implica encontrar cuatro o cinco ramificaciones condicionales en promedio, no todas las cuales pueden predecirse correctamente, por lo que el valor de poder ver tan lejos en el futuro es dudoso. La etapa IFU2 alinea las instrucciones para que la siguiente etapa pueda decodificarlas con facilidad.

La decodificación inicia en la etapa ID0. Decodificar en el Pentium II consiste en convertir cada instrucción IA-32 en una o más microoperaciones, como hacía el Mic-4. Las instrucciones IA-32 más sencillas, como las transferencias de registro a registro, se pueden traducir a una microoperación. Las más complejas pueden requerir hasta cuatro microoperaciones. Unas cuantas instrucciones extremadamente complejas pueden requerir aún más, y utilizar la ROM del secuenciador de microoperaciones para determinar la secuencia.

La etapa ID0 tiene tres decodificadores internos. Dos de ellos son para instrucciones sencillas; el tercero maneja las demás. Lo que sale de la etapa ID0 es una secuencia de microoperaciones. Cada una contiene un código de operación, dos registros fuente y un registro de destino.

Las microoperaciones colocadas en la cola de espera en la etapa ID1, análoga a la unidad de cola de espera de la figura 4-35. Esta etapa también detecta ramificaciones. Primero viene una predicción estática, por si acaso. La predicción depende de varios factores, pero en el caso de ramificaciones relativas a la instrucción en curso se predice que las ramas hacia atrás se tomarán y las ramas hacia adelante no. Después viene el predictor de ramificación dinámico que emplea un algoritmo basado en el historial, como el de la figura 4-42, sólo que en lugar de usar sólo 2 bits de historial usa 4. Debe ser evidente que con una fila de procesamiento de 12 etapas el castigo por predecir mal es enorme, de ahí el uso de tantos bits de historial. Si la ramificación no está en la tabla histórica, se usa la predicción estática.

Para evitar dependencias WAR y WAW, el Pentium II usa cambio de nombres de registro, como vimos en la figura 4-44. Los registros reales nombrados en las instrucciones IA-32 se pueden sustituir en las microoperaciones por cualesquiera de 40 registros de borrador internos ubicados en el ROB. Este cambio de nombres se efectúa en la etapa RAT.

Por último, las microoperaciones se depositan en el ROB a razón de tres por ciclo de reloj. Los operandos también se reúnen ahí, si están disponibles. Si los operandos y el registro de resultado de una microoperación están disponibles, y la unidad de ejecución está libre, la microoperación es elegible para emitirse; si no, permanece en el ROB hasta que se han adquirido todos sus recursos.

La unidad de Despacho/Ejecución

Ahora llegamos a la unidad de Despacho/Ejecución, que se ilustra en la figura 4-48. Esta unidad planifica y ejecuta las microoperaciones, resolviendo dependencias y conflictos de recursos. Aunque sólo se pueden decodificar tres instrucciones ISA por ciclo de reloj (en ID0), se pueden emitir hasta cinco microoperaciones para ejecutarse en un ciclo, una por cada puerto. Este ritmo no puede sostenerse porque excede la capacidad de la unidad de Retiro. Las microoperaciones se pueden emitir en desorden, pero la unidad de Retiro las retira en orden. Se emplea un marcador complejo para llevar la contabilidad de las microoperaciones pendientes, registros y unidades de ejecución. Cuando una microoperación es elegible para ejecutarse, se le puede iniciar, aunque otras que se colocaron antes en el ROB no estén listas. Cuando hay varias microoperaciones elegibles para ejecutarse en la misma unidad de ejecución, un algoritmo complejo escoge la que más conviene emitir a continuación. Por ejemplo,

es mucho más importante que se ejecute una ramificación que una instrucción aritmética, ya que la primera afecta el flujo del programa.

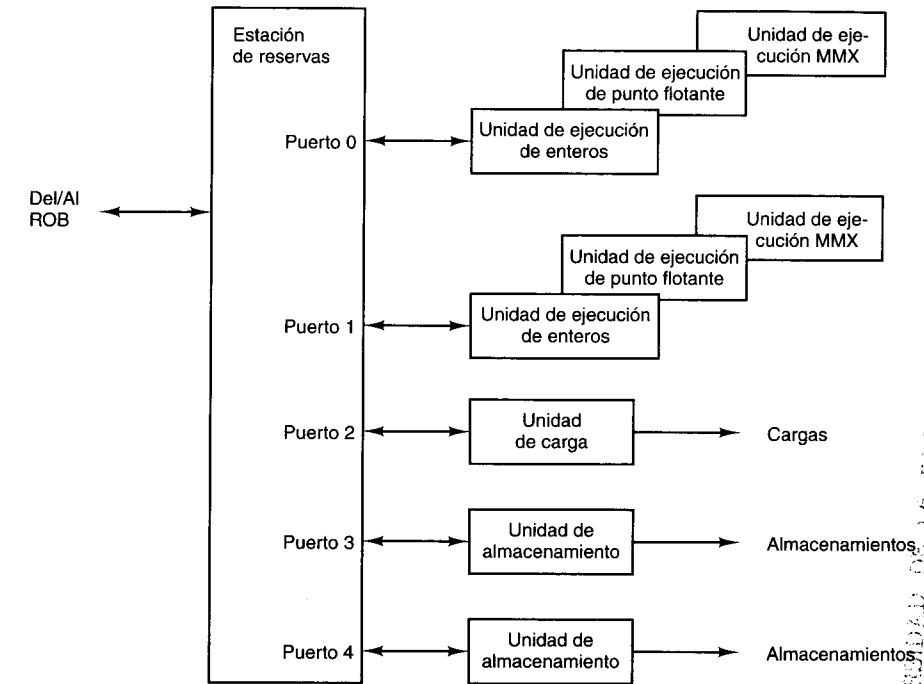


Figura 4-48. La unidad de Despacho/Ejecución.

La unidad de Despacho/Ejecución contiene una estación de reservas y unidades de ejecución conectadas a cinco puertos. La estación de reservas es una cola con 20 entradas para las microoperaciones que ya tienen todos sus operandos. Éstas esperan su turno en la estación de reservas hasta que la unidad de ejecución que necesitan se desocupa.

Hay cinco puertos que van de la estación de reserva a las unidades de ejecución. Algunas unidades de ejecución comparten un mismo puerto, como se muestra. Las unidades de carga y almacenamiento establecen la información apropiada para las operaciones de carga y almacenamiento, respectivamente. Hay dos puertos para los almacenamientos. Puesto que sólo puede emitirse una microoperación por puerto por ciclo, si dos microoperaciones pendientes necesitan pasar por el mismo puerto una de ellas tendrá que esperar.

La unidad de Retiro

Una vez que se ha ejecutado una microoperación, regresa a la estación de reservas y luego al ROB para esperar que se le retire. La unidad de Retiro se encarga de enviar los resultados al

lugar apropiado: al registro correcto, pero también a otras estaciones de la unidad de Despacho/Ejecución que están esperando ese valor. La unidad de Búsqueda/Decodificación mantiene los registros “oficiales”, es decir, los valores producidos por las instrucciones que ya terminaron. La unidad de Retiro mantiene un conjunto de registros pendientes, es decir, los valores que se calcularon en una instrucción que no ha terminado porque instrucciones previas todavía no terminan.

El Pentium II apoya plenamente la ejecución especulativa, por lo que algunas instrucciones se habrán ejecutado en vano y no se retirarán. Aquí es donde reside la capacidad de reversión. Si se determina que alguna microoperación provino de una instrucción IA-32 que no debió haberse ejecutado, sus resultados se desechan. Corresponde a la unidad de Retiro seguir la pista a todo esto. Sólo las instrucciones ejecutadas “oficialmente” pueden retirarse, y deben retirarse en el orden del programa, aunque se hayan ejecutado fuera de orden.

4.6.2 La microarquitectura de la CPU UltraSPARC II

La serie UltraSPARC es la implementación hecha por Sun de la arquitectura SPARC Versión 9. Los diversos modelos son muy similares, y difieren principalmente en su desempeño y su precio. No obstante, para evitar confusiones nos referiremos al “UltraSPARC II” en toda esta sección y describiremos sus características en las áreas en que difieren.

El UltraSPARC II es una máquina cabal de 64 bits, con registros de 64 bits y una trayectoria de datos de 64 bits, aunque por razones de compatibilidad hacia atrás con SPARC Versión 8 (es decir, de 32 bits), también puede manejar operandos de 32 bits y, de hecho, ejecutar software para SPARC de 32 bits sin modificación. Aunque la arquitectura interna es de 64 bits, el bus de memoria tiene una anchura de 128 bits, análogo a la arquitectura de 32 bits del Pentium II con bus de memoria de 64 bits, pero una generación más adelante de la CPU misma. En la figura 3-47 se muestra el corazón del sistema UltraSPARC II.

Toda la serie SPARC ha sido un diseño RISC desde que nació. Casi todas las instrucciones tienen dos registros fuente y un registro de destino, por lo que se prestan a la ejecución en filas de procesamiento en un ciclo. No hay necesidad de descomponer instrucciones CISC antiguas en microoperaciones RISC, como tiene que hacer el Pentium II.

El UltraSPARC II es una máquina superescalar que puede emitir cuatro instrucciones por ciclo de reloj de forma sostenida indefinidamente. Las instrucciones se emiten en orden pero pueden terminar fuera de orden. No obstante, las interrupciones son precisas (es decir, no hay ambigüedad respecto a dónde estaba exactamente la máquina cuando ocurrió una interrupción). Existe apoyo de hardware para cargas especulativas en forma de una instrucción PREFETCH (prebúsqueda) que no causa un fallo cuando no encuentra lo que busca en la caché. De hecho, ni siquiera se bloquean los accesos subsiguientes a la memoria. Por tanto, un compilador puede plantar uno o más PREFETCH en el flujo de código mucho antes del punto en el que se necesitan con la esperanza de tener los datos ahí cuando se necesiten, pero sin ser castigado en caso de ocurrir un fallo de caché.

Generalidades del UltraSPARC II

En la figura 4-49 se presenta un diagrama de bloques del UltraSPARC II. Todos los componentes que se muestran están en el chip de la CPU excepto la caché de nivel 2, que es totalmente externa. La caché I es una caché asociativa de 16 KB de dos vías con una línea de 32 bytes, pero que puede devolver media línea de caché. Media línea (16 bytes) contiene exactamente cuatro instrucciones, todas las cuales pueden emitirse en un solo ciclo de reloj. La caché D es de 16 KB, con mapeo directo, escritura a través y sin asignación, que también usa líneas de 32 bytes divididas en dos sublíneas de 16 bytes.

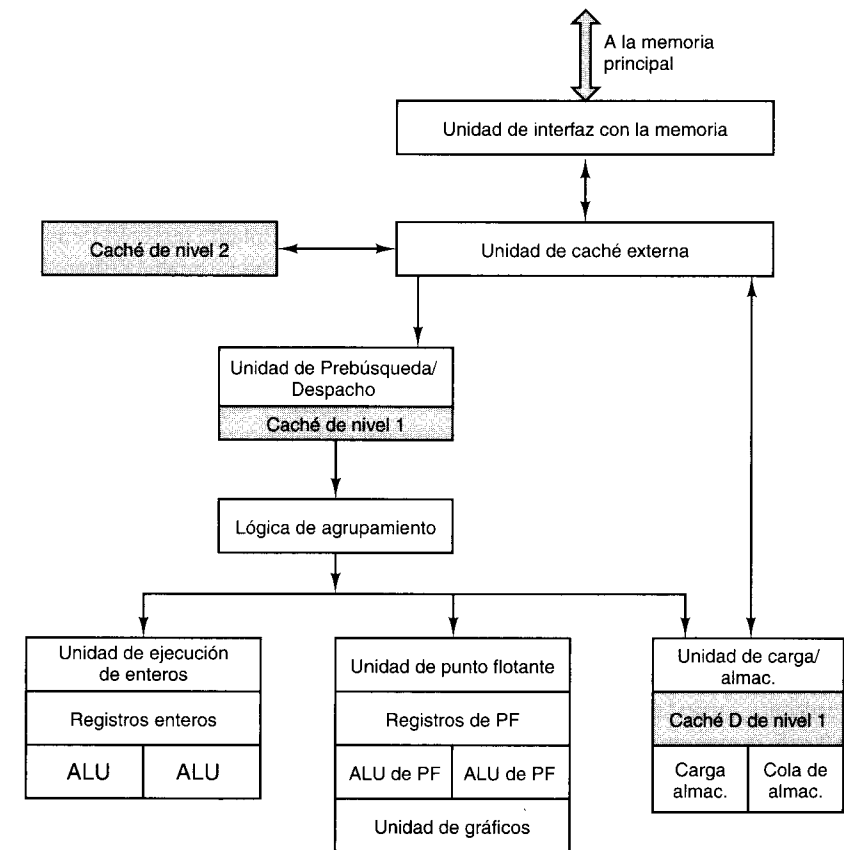


Figura 4-49. La microarquitectura de UltraSPARC II.

La unidad de caché externa maneja los fallos de caché de nivel 1 buscando la línea requerida en la caché externa (de nivel 2). Si la búsqueda tiene éxito, la línea se copia en la caché de nivel 1 apropiado. Si no, la unidad de caché externa pide a la unidad de interfaz con la memoria que busque la línea de la memoria principal.

Veamos ahora las unidades funcionales del UltraSPARC II. La unidad de Prebúsqueda/Despacho es similar a la unidad de Búsqueda/Decodificación del Pentium II (vea la figura 4-46). Sin embargo, su tarea es más fácil porque las instrucciones que llegan ya son microoperaciones de tres registros, así que no necesitan desglosarse. La unidad puede buscar instrucciones de la caché de nivel 1 o de la caché de nivel 2 sin pérdida de tiempo. Se traen cuatro instrucciones por ciclo de reloj.

Para reducir el efecto de las ramificaciones, cada grupo de cuatro instrucciones de la caché I tiene una dirección que indica cuál media línea debe tomarse a continuación. Además, hay un predictor de ramificaciones completo dentro de la unidad de Prebúsqueda/Despacho que usa un algoritmo de predicción de 2 bits similar al de la figura 4-42. Además, el UltraSPARC II tiene un conjunto de instrucciones de ramificación que permiten al compilador decirle al hardware qué predicción debe hacer, ya que en muchos casos tiene una muy buena idea de qué rama se tomará. Las instrucciones prebuscadas se almacenan en una cola de 12 entradas que alimenta a la lógica de agrupamiento.

La lógica de agrupamiento es una unidad que selecciona de la cola cuatro instrucciones a la vez para emitirlas. La cuestión es encontrar cuatro que se puedan emitir simultáneamente. La unidad de ejecución de enteros tiene dos ALU discretas, lo que le permite ejecutar dos instrucciones en paralelo. De igual manera, la unidad de punto flotante también tiene dos ALU de PF. Por tanto, un grupo podría contener dos de cada tipo, pero no cuatro de un mismo tipo. Para que el agrupamiento sea óptimo, las instrucciones se deben emitir en desorden, lo cual está permitido. El retiro también es entonces en desorden.

Las unidades de enteros y de punto flotante contienen sus respectivos registros, así que las instrucciones recogen sus operandos dentro de la unidad y dejan sus resultados ahí también. Los registros de enteros y de punto flotante son distintos, por lo que nunca hay transferencia de valores de una unidad a la otra. La unidad de punto flotante contiene además una unidad de gráficos que ejecuta instrucciones especiales para procesar imágenes 2D y 3D, audio y video, análogas a las instrucciones MMX del Pentium II.

La unidad de Carga/Almacenamiento maneja las instrucciones LOAD y STORE. Si hay un acierto en la caché D de nivel 1, la ejecución es inmediata; si no, se pide a la unidad de caché externa que obtenga la línea de la caché de nivel 2 o de la memoria principal (en un fallo de LOAD) o escriba la palabra ahí (en un fallo de STORE). Si la caché D hubiera sido de asignación de escritura, en un fallo de STORE la línea de caché requerida se habría traído de la caché de nivel 2 o de la memoria principal. En cambio, la palabra que se va a almacenar simplemente se escribe a través de la caché de nivel 2 o, si no acierta ahí, en la memoria principal. Para evitar un bloqueo causado por un fallo de la caché D, la unidad de Carga/Almacenamiento mantiene colas (individuales) de instrucciones LOAD y STORE pendientes para poder seguir procesando nuevas instrucciones mientras espera que las anteriores terminen.

La fila de procesamiento del UltraSPARC II

El UltraSPARC II tiene una fila de procesamiento de nueve etapas, algunas de las cuales son diferentes para las instrucciones con enteros y de punto flotante, como se ilustra en la figura 4-50. La primera etapa busca instrucciones de la caché I (si es posible). En condiciones

ideales (sin fallos de caché, sin errores de predicción, sin instrucciones difíciles, la combinación perfecta de instrucciones, etc.), esta etapa puede seguir trayendo y emitiendo cuatro instrucciones por ciclo indefinidamente. La etapa de decodificación añade algunos bits extra a cada instrucción antes de depositarla en la cola de instrucciones de 12 entradas que mencionamos antes. Estos bits agilizan el procesamiento subsecuente (por ejemplo, indicando la unidad de ejecución apropiada).

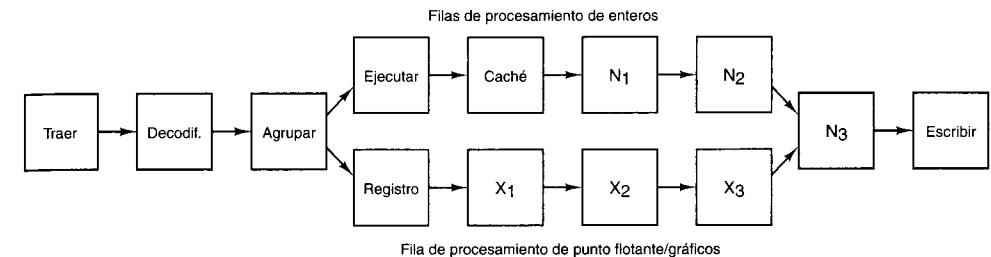


Figura 4-50. Fila de procesamiento del UltraSPARC II.

La etapa de agrupamiento corresponde a la unidad de lógica de agrupamiento que vimos antes. Esta etapa examina las instrucciones decodificadas en busca de grupos de cuatro que sean compatibles para emitirse simultáneamente.

En este punto las filas de procesamiento para enteros y para punto flotante se dividen. La etapa 4 de la unidad de enteros ejecuta casi todas las instrucciones directamente en un ciclo. Sin embargo, las instrucciones LOAD y STORE requieren procesamiento adicional en la etapa de caché. Las etapas N_1 y N_2 no hacen nada en el caso de instrucciones de registros pero se incluyen para mantener las dos filas de procesamiento sincronizadas. Que las instrucciones con enteros terminen unos cuantos nanosegundos más tarde es un precio pequeño que se paga por mantener un flujo continuo en las filas de procesamiento.

La unidad de punto flotante tiene cuatro etapas propias que inician aquí. La primera es para acceder a los registros de punto flotante; las tres siguientes son para ejecutar la instrucción. Todas las instrucciones de punto flotante se ejecutan en tres ciclos, con excepción de la división (12 ciclos) y la raíz cuadrada (22 ciclos), por lo que una serie larga de las demás instrucciones de punto flotante no frena la fila de procesamiento.

La etapa N_3 , que es común a ambas unidades, sirve para resolver condiciones de excepción que podrían haber ocurrido, como la división entre cero. La última etapa es en la que se escriben los resultados en los registros. Esta etapa es comparable a la unidad de Retiro del Pentium II en cuanto a que una vez que una instrucción ha pasado por esta etapa, está confirmada.

4.6.3 La microarquitectura de la CPU picoJava II

El picoJava II puede ejecutar programas JVM binarios sin (mucho) interpretación en software. Casi todas las instrucciones JVM son ejecutadas directamente por el hardware en un solo ciclo de reloj. Cerca de 30 instrucciones JVM están microprogramadas. Unas pocas

instrucciones no pueden ser ejecutadas por el hardware de picoJava II y causan saltos a un intérprete en software. Estas instrucciones por lo regular tienen que ver con características de JVM que no hemos estudiado, como crear y manejar objetos de software complejos.

Generalidades del picoJava II

En la figura 4-51 se muestra un diagrama de bloques de la microarquitectura del picoJava II. Hay una caché de nivel 1 dividido en el chip. La caché I es opcional y puede tener tamaños de 1 KB, 2 KB, 4 KB, 8 KB o 16 KB si está presente; es de mapeo directo, con un tamaño de línea de 16 bytes. La caché D también es opcional y también puede tener tamaños de 1 KB, 2 KB, 4 KB, 8 KB o 16 KB si está presente; es asociativa por conjuntos de dos vías, y también tiene un tamaño de línea de 16 bytes. La caché D usa asignación de escritura y reescritura. Cada caché tiene una trayectoria de 32 bits de anchura al bus de memoria. El microJava 701 tiene ambas cachés presentes, cada una con los 16 KB completos. Una unidad de punto flotante opcional también forma parte del diseño del picoJava II.

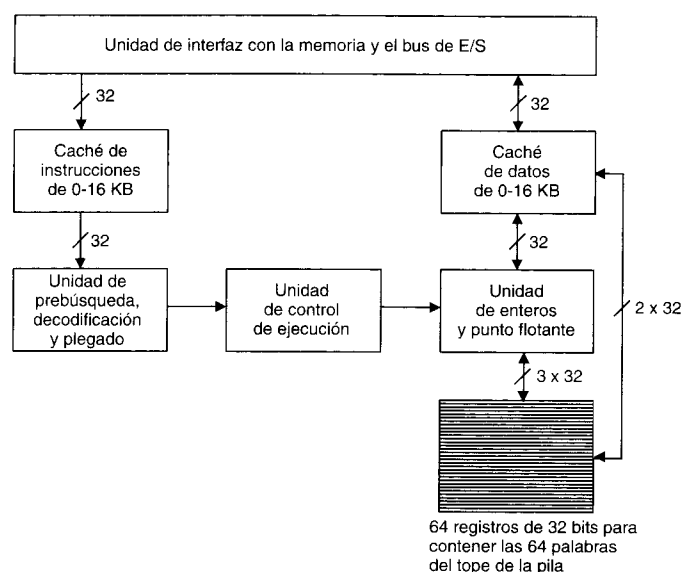


Figura 4-51. Diagrama de bloques del picoJava II con ambas cachés de nivel 1 y la unidad de punto flotante. Ésta es la configuración del microJava 701.

La caché I alimenta la unidad de búsqueda, decodificación y plegado de 8 en 8 bytes. Esta unidad, a su vez, está conectada al controlador de la ejecución y a la trayectoria de datos principal (unidad de enteros/punto flotante). La trayectoria de datos tiene una anchura de 32 bits para las operaciones con enteros, pero también puede manejar punto flotante IEEE 754 con precisión sencilla y doble.

La parte más interesante de la figura 4-51 es un archivo de registros que consiste en 64 registros de 32 bits. Estos registros pueden contener las 64 palabras superiores de la pila de la máquina JVM, lo que agiliza considerablemente el acceso a las palabras de la pila. Tanto la pila de operandos como la pila de variables locales que está inmediatamente abajo de ella se pueden guardar en el archivo de registros. Los accesos al archivo de registros son "gratuitos" (es decir, ocurren sin causar retraso, mientras que los accesos a la caché D requieren un ciclo extra). La trayectoria entre la unidad de enteros/punto flotante tiene 96 bits de anchura y puede manejar dos lecturas de 32 bits de la pila y una escritura de 32 bits en la pila en un solo ciclo.

Por ejemplo, si la pila de operandos consiste en dos palabras, puede haber hasta 62 palabras de variables locales en el archivo de registros. Desde luego, si se mete otra palabra en la pila, surge un problema. Lo que sucede es que una o más palabras del fondo de la pila se reescriben a la memoria por un proceso llamado **goteo**. De igual manera, si se sacan varias palabras del tope de la pila de operandos, quedará espacio disponible en el archivo de registros, y podrán volverse a cargar en él algunas palabras de las profundidades de la pila. Registros especiales del chip determinan qué tan lleno tiene que estar el archivo de registros antes de que algunas palabras del fondo se escriban en la memoria, y qué tan vacío tiene que estar antes de que se vuelva a cargar desde la memoria. Para facilitar el goteo sin copiado, el archivo de registros opera como un buffer circular, con apuntadores a la palabra válida más baja y a la más alta. El goteo es automático, y ocurre siempre que el archivo de registros está demasiado lleno o demasiado vacío.

La fila de procesamiento del picoJava II

El picoJava II tiene una fila de procesamiento de seis etapas que se ilustra en la figura 4-52. La primera etapa busca instrucciones de la caché I de 8 en 8 bytes y las coloca en un buffer de instrucciones cuya capacidad es de 16 bytes. La siguiente etapa decodifica las instrucciones y las pliega (combina) de ciertas maneras que describiremos en breve. Lo que sale de la unidad decodificadora es una secuencia de microoperaciones, cada una de las cuales contiene un código de operación y tres números de registro (dos registros fuente y un registro de destino). En este sentido, el picoJava II es similar al Pentium II: ambas máquinas aceptan una serie de instrucciones CISC que se desglosan internamente para dar una secuencia de microoperaciones que se alimentan a un núcleo RISC con filas de procesamiento. Sin embargo, a diferencia del Pentium II, el picoJava II no es superescalar y las microoperaciones se emiten y ejecutan en el orden en que se emitieron. Si hay un fallo de caché, si hay que buscar un operando de la memoria principal, la CPU se para y lo espera.

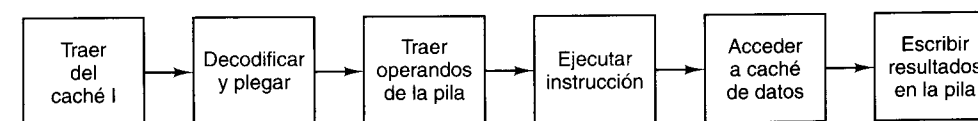


Figura 4-52. El picoJava II tiene una fila de procesamiento de seis etapas.

La tercera etapa de la fila de procesamiento busca operandos de la pila (en realidad del archivo de registros) para que estén listos para la cuarta etapa, la unidad de ejecución, donde

se llevan a cabo las instrucciones. La quinta etapa accede a la caché de datos si es necesario; por ejemplo, para guardar resultados ahí. Por último, la sexta etapa escribe resultados de vuelta en la pila.

Plegado de instrucciones

Como dijimos antes, la unidad decodificadora puede **plegar** varias instrucciones para juntarlas. Como ejemplo de este proceso, consideremos la evaluación del enunciado

$n = k + m;$

La traducción a (I)JVM podría ser

```
ILOAD 7
ILOAD 1
IADD
ISTORE 3
```

suponiendo que *k*, *m* y *n* son las variables locales 7, 1 y 3, respectivamente, la ejecución de estas cuatro instrucciones sería como se muestra en la figura 4-53(a).

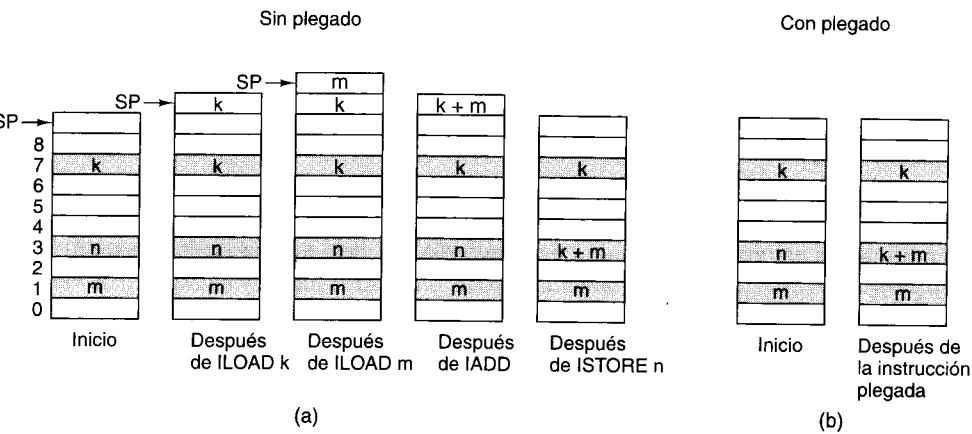


Figura 4-53. (a) Ejecución de una secuencia de cuatro instrucciones para calcular $n = k + m$. (b) La misma secuencia plegada en una instrucción.

Suponiendo que las tres variables están lo bastante alto en la pila como para que todas estén contenidas en el archivo de registros, no es necesario hacer ninguna referencia a la memoria para ejecutar esta secuencia de instrucciones. La primera instrucción, ILOAD 7 copia en el tope de la pila la palabra que está en la séptima variable local e incrementa el apuntador de la pila. Igualmente, ILOAD 1 es también una copia de registro a registro. IADD suma dos registros e ISTORE copia un registro en la porción de variables locales del archivo de registros. Ya de por sí, ahorrarse todas las referencias a la memoria es una importante optimización del desempeño.

Sin embargo, el picoJava II hace mucho más. Lo que la secuencia de cuatro instrucciones en realidad hace es sumar dos registros y guardar el resultado en un tercer registro. La unidad decodificadora detecta esta condición y emite una sola microoperación: una instrucción ADD de tres registros. Así, en lugar de tener cuatro instrucciones JVM que hacen un total de nueve referencias a la memoria, tenemos una sola microoperación de tres registros para efectuar una suma, como se muestra en la figura 4-53(b). El resultado neto es que, aunque instrucciones CISC con muchas referencias a la memoria ingresaron en la fila de procesamiento, sólo se ejecutó realmente una microoperación sencilla. Por tanto, el picoJava II puede ejecutar programas Java compilados para JVM con la misma rapidez que si se compilaran para el lenguaje máquina de una máquina RISC de extremo superior.

Como acabamos de ver, la capacidad para plegar varias instrucciones JVM en una microoperación es la clave para la ejecución de alto desempeño con filas de procesamiento. Por ello, vale la pena examinar brevemente cómo la unidad decodificadora realiza el plegado. Para fines de plegado, las instrucciones se dividen en seis grupos, que se enumeran en la figura 4-54. El primer grupo contiene instrucciones que no se pliegan. El segundo contiene cargas de variables locales. IJVM tiene una variable de este tipo, ILOAD, pero JVM tiene otras además. El tercer grupo contiene almacenamientos, como por ejemplo ISTORE. El cuarto y quinto grupos son para ramificaciones, con uno y dos operandos, respectivamente. El último grupo consiste en instrucciones que sacan dos operandos de la pila, realizan algún cálculo con ellos, y meten el resultado otra vez en la pila.

Grupo	Descripción	Ejemplo
NF	Instrucciones no plegables	GOTO
LV	Meter una palabra en la pila	ILOAD
MEM	Sacar una palabra y guardarla en la memoria	ISTORE
BG1	Operaciones que usan un operando de la pila	IFEQ
BG2	Operaciones que usan dos operandos de la pila	IF_CMPEQ
OP	Cálculos con dos operandos y un resultado	IADD

Figura 4-54. Grupos de instrucciones JVM para fines de plegado.

La unidad decodificadora emite microoperaciones de 74 bits a la máquina de ejecución (vía la unidad de búsqueda de operandos). Casi todas éstas contienen un código de operación y tres registros y se pueden ejecutar en un solo ciclo. A medida que la unidad decodificadora trae instrucciones JVM del buffer de instrucciones, las convierte en sucesiones de microoperaciones, pero al mismo tiempo busca ciertos patrones de secuencias de instrucciones JVM que puedan plegarse en una sola microoperación. Los patrones pueden tener hasta cuatro instrucciones de longitud. Cuando se encuentra un patrón, se emite la microoperación correspondiente y se desechan las instrucciones originales.

En la figura 4-55 se presenta una tabla con algunas secuencias JVM comunes que se pueden plegar. Cuando la unidad decodificadora encuentra una de estas secuencias, sustituye la descomposición de instrucciones acostumbrada por una sola microoperación que realiza el

trabajo de toda la secuencia en un ciclo. Por ejemplo, una secuencia de cuatro instrucciones se convierte en una sola microoperación **ADD** de tres registros, como vimos en la figura 4-53. El plegado sólo funciona cuando las variables locales a las que se hace referencia están lo bastante cerca del tope de la pila como para estar contenidas en el archivo de registros.

Secuencia de instrucciones				Ejemplo
LV	LV	OP	MEM	ILOAD, ILOAD, IADD, ISTORE
LV	LV	OP		ILOAD, ILOAD, IADD
LV	LV	BG2		ILOAD, ILOAD, IF_CMPEQ
LV	BG1			ILOAD, IFEQ
LV	BG2			ILOAD, IF_CMPEQ
LV	MEM			ILOAD, ISTORE
OP	MEM			IADD, ISTORE

Figura 4-55. Algunas secuencias de instrucciones JVM que se pueden plegar.

El plegado ocurre con suficiente frecuencia como para que una fracción sustancial de un programa JVM se pueda ejecutar como si se hubiera compilado directamente para un procesador RISC con filas de procesamiento, lo que le confiere la transportabilidad de JVM combinada con el alto desempeño de una máquina RISC. Mediciones realizadas muestran que el picoJava II ejecuta programas en Java hasta cinco veces más rápidamente que los mismos programas compilados a lenguaje máquina en un Pentium con la misma frecuencia de reloj, y hasta 15 veces más rápidamente que si se ejecutaran interpretados en un Pentium.

El picoJava II tiene un algoritmo de predicción de ramas muy primitivo: siempre predice que no se tomará la rama. La justificación para esto fue mantener el chip sencillo y económico en lugar de dedicar una porción sustancial del área del chip a circuitos de predicción. Además, debido a la longitud del conducto (seis etapas, en lugar de 12 en el Pentium II), el castigo por predecir erróneamente es de sólo tres ciclos.

4.6.4 Comparación del Pentium, UltraSPARC y picoJava

Nuestros tres ejemplos son muy diferentes pero es asombroso lo mucho que tienen en común en el fondo, y esto dice algo acerca de la forma óptima de diseñar una computadora. El Pentium II tiene un conjunto de instrucciones CISC prehistórico que a los ingenieros de Intel les encantaría tirar a la bahía de San Francisco, sólo que ello violaría las leyes contra la contaminación del agua de California. El UltraSPARC II es un diseño RISC puro, con instrucciones magras y potentes. El picoJava II es una máquina de pila, con instrucciones de longitud variable que hacen una enormidad de referencias a memoria (que en principio son costosas).

A pesar de estas grandes diferencias, las tres máquinas tienen unidades de ejecución muy similares. Todas las unidades de ejecución aceptan microoperaciones que contienen

un código de operación, dos registros fuente y un registro de destino. Todas ellas pueden ejecutar una microoperación en un ciclo; todas ellas tienen filas de procesamiento profundas y predicción de ramificaciones, y todas ellas tienen cachés I y D divididos, con 16 KB por caché.

Esta similitud interna no es accidental y ni siquiera se debe al incesante saltar de una empresa a otra de los ingenieros del Valle del Silicio. Como vimos con nuestros ejemplos Mic-3 y Mic-4, es fácil y natural construir una trayectoria de datos con filas de procesamiento que tome dos registros fuente, los pase por una ALU y guarde el resultado en un registro. La figura 4-34 muestra esta fila de procesamiento gráficamente. Con la tecnología actual, éste es el diseño más eficaz.

La diferencia primordial entre el Pentium II, el UltraSPARC II y el picoJava II es la forma como llegan de su conjunto de instrucciones de ISA a la unidad de ejecución. El Pentium II tiene que descomponer sus instrucciones CISC para ponerlas en el formato de tres registros que la unidad de ejecución necesita. De eso se trata la figura 4-47: partir instrucciones grandes en microoperaciones simples y precisas. El picoJava II tiene el problema opuesto: cómo combinar varias instrucciones para obtener microoperaciones simples y claras. Eso es lo que hace el plegado. El UltraSPARC II no tiene que hacer nada porque sus instrucciones nativas ya son microoperaciones. Es por esto que casi todas las ISA nuevas son del tipo RISC, a menos que tengan algún motivo ulterior (como bajar programas en Java de Internet y ejecutarlos en una máquina arbitraria).

Aprenderemos algo si comparamos nuestro diseño final, el Mic-4, con estos tres ejemplos del mundo real. El Mic-4 se parece más al Pentium II. Ambos tienen que interpretar un conjunto de instrucciones ISA no RISC. Ambos lo hacen descomponiendo las instrucciones ISA en microoperaciones que tienen un código de operación, dos registros fuente y un registro de destino. En ambos casos, las microoperaciones se depositan en una cola para ejecutarse posteriormente. El Mic-4 tiene un diseño estricto de emisión en orden, ejecución en orden y retiro en orden, mientras que el Pentium II tiene una política de emisión en orden, ejecución en desorden y retiro en orden. La estructura interna de la fila de procesamiento del Pentium II, sobre todo la parte que se muestra en la figura 4-47, también tiene algo en común con el diseño del Mic-4.

Comparemos ahora el Mic-4 con el picoJava II. Aunque parece lógico que las dos arquitecturas sean muy similares —después de todo, interpretan el mismo conjunto de instrucciones ISA— no son en realidad tan parecidas. La razón es que las unidades decodificadoras tienen estrategias diametralmente opuestas. El Mic-4 toma cada instrucción IJVM que llega e inmediatamente la descompone en microoperaciones, mientras que el picoJava II trata de combinar (plegar) varias instrucciones JVM en una sola microoperación. La sencilla asignación $i = j + k$ tarda 14 ciclos en el Mic-4 y un ciclo en el picoJava II. En este caso, el plegado gana por un factor de 14. Obviamente, el segundo método da pie a una ejecución mucho más rápida, pero la complejidad del proceso de plegado es sustancial.

El Mic-4 y el UltraSPARC II no se pueden comparar realmente porque el segundo tiene instrucciones RISC (es decir, microoperaciones de tres registros) como conjunto de instrucciones ISA. No hay necesidad de descomponerlas ni de combinarlas; se pueden ejecutar tal cual, cada una en un solo ciclo de la trayectoria de datos.

Las cuatro máquinas hacen uso intensivo de filas de procesamiento. El Pentium II tiene 12 etapas, el UltraSPARC II tiene nueve, el picoJava II tiene seis y el Mic-4 tiene siete. El Pentium II tiene unas pocas más que los otros porque sus instrucciones no son fáciles de descomponer. El UltraSPARC II tiene más de las que realmente necesita porque la fila de procesamiento de enteros se alargó artificialmente en dos etapas para hacer que las operaciones con enteros tardaran el mismo tiempo que las de punto flotante. La conclusión aquí es que con la tecnología moderna y un conjunto de instrucciones razonable, lo más recomendable es una fila de procesamiento de seis o siete etapas que procese microoperaciones de tres registros. El Mic-4 nos da una buena idea de cómo podría funcionar una fila de procesamiento semejante (al menos con un conjunto de instrucciones no RISC).

4.7 RESUMEN

El corazón de una computadora es la trayectoria de datos. Éste contiene algunos registros, dos o tres buses y una o más unidades funcionales como ALU y desplazadores. El ciclo de ejecución principal consiste en buscar algunos operandos de los registros y enviarlos por los buses a la ALU y otras unidades funcionales para efectuar la ejecución. Luego, los resultados se vuelven a guardar en registros.

La trayectoria de datos puede controlarse con un secuenciador que busca microinstrucciones de un almacén de control. Cada microinstrucción contiene bits que controlan la trayectoria de datos durante un ciclo. Dichos bits especifican qué operandos hay que seleccionar, cuál operación debe efectuarse y qué debe hacerse con los resultados. Además, cada microinstrucción especifica su sucesora, por lo regular de forma explícita al contener su dirección. Algunas microinstrucciones modifican esta dirección base calculando el OR de ciertos bits y la dirección antes de usarla.

La máquina IJVM es una máquina de pila con códigos de operación de un byte que almacena palabras en la pila, sacan palabras de la pila y combinan (por ejemplo, suman) palabras en la pila. Se dio una implementación microprogramada para la microarquitectura Mic-1. Mediante la adición de una unidad para buscar las instrucciones y precargar los bytes en el flujo de instrucciones, fue posible eliminar muchas referencias al contador de programa y hacer la máquina mucho más rápida.

Hay muchas maneras de diseñar el nivel de microarquitectura. Existen muchos puntos de equilibrio, como los diseños de dos buses *versus* tres buses, campos de microinstrucción codificados *versus* decodificados, presencia o ausencia de prebúsqueda, filas de procesamiento profundas o someras, y mucho más. El Mic-1 es una máquina sencilla, controlada por software, con ejecución secuencial y sin paralelismo. En contraste, el Mic-4 tiene una microarquitectura altamente paralela y una fila de procesamiento de siete etapas.

Existen varias formas de mejorar el desempeño. La memoria caché es uno de los más importantes. Las cachés con mapeo directo y de conjunto asociativo se usan comúnmente para agilizar las referencias a la memoria. La predicción de ramificaciones, tanto estática como dinámica, es importante, lo mismo que la ejecución fuera de orden y la ejecución especulativa.

Nuestras tres máquinas de ejemplo, el Pentium II, el UltraSPARC II y el picoJava II, son muy diferentes en lo exterior pero tienen unidades de ejecución sorprendentemente simila-

res. El Pentium II toma un conjunto de instrucciones tipo CISC y descompone las instrucciones en microoperaciones que se procesan en un núcleo superescalar con predicción de ramificaciones, ejecución en desorden y especulación. El UltraSPARC II es una CPU moderna de 64 bits con instrucciones tipo RISC que también tiene predicción de ramificaciones, ejecución en desorden y especulación. El picoJava II es una máquina más sencilla diseñada para aparatos de bajo costo, por lo que carece de algunas funciones de extremo superior como predicción dinámica de ramificaciones. No obstante, gracias a que pliega las instrucciones, puede ejecutar instrucciones JVM casi como si fueran instrucciones RISC orientadas a registros. Las tres máquinas tienen una unidad de ejecución similar que maneja microoperaciones de tres registros que fluyen por una fila de procesamiento profunda.

PROBLEMAS

1. En la figura 4-2 se muestra una forma de producir A como salida de la ALU. Sugiera otra forma.
2. En el Mic-1 se requiere 1 ns para preparar MIR, 1 ns para colocar el contenido de un registro en el bus B, 3 ns para operar la ALU y el desplazador, y 1 ns para que los resultados se propaguen de vuelta a los registros. El ancho del pulso de reloj es de 2 ns. ¿Esta máquina puede operar a 100 MHz? ¿Y a 150 MHz?
3. En la figura 4-6 el registro del bus B se codifica en un campo de 4 bits, pero el bus C se representa como mapa de bits. ¿Por qué?
4. En la figura 4-6 hay un cuadro rotulado "Bit alto". Dibuje el diagrama de circuito correspondiente.
5. Cuando el campo JMPC de una microinstrucción está habilitado, se calcula el OR de MBR y NEXT_ADDRESS para formar la dirección de la siguiente microinstrucción. ¿Hay alguna situación en la que sea lógico que NEXT_ADDRESS sea 0x1FF y se use JMPC?
6. Suponga que en el ejemplo de la figura 4-14(a) se añade el enunciado

$i = 0;$

después del enunciado if. ¿Qué haría el nuevo código de ensamblador? Suponga que el compilador es optimante.

7. Dé dos traducciones a IJVM diferentes del siguiente enunciado en Java:

$i = j + k + 4;$

8. Dé el enunciado en Java que produjo el siguiente código IJVM:

```
ILOAD j
ILOAD k
ISUB
BIPUSH 6
ISUB
DUP
IADD
ISTORE i
```

9. En el texto mencionamos que al traducir el enunciado

if (Z) goto L1; else goto L2

a binario, L2 tiene que estar en las 256 palabras inferiores del almacén de control. ¿No sería igualmente posible tener L1 en, digamos 0x40 y L2 en 0x140? Explique su respuesta.

10. En el microprograma del Mic-1, en if_icmpeq3, MDR se copia en H y en la siguiente línea se le resta TOS. Seguramente sería mejor tener un solo enunciado aquí

if+cmpeq3 Z = MDR - TOS; rd

¿Por qué no se hace así?

11. ¿Cuánto tarda un Mic-1 de 200 MHz en ejecutar el enunciado Java

i = j + k;

Calcule su respuesta en nanosegundos.

12. Repita la pregunta anterior, sólo que ahora para un Mic-2 de 200 MHz. Con base en este cálculo, ¿cuánto tardaría en ejecutarse en el Mic-2 un programa que tarda 100 s en el Mic-1?
13. En la máquina JVM completa hay códigos de operación especiales de un byte para cargar las variables locales 0 a 3 en la pila en lugar de usar la instrucción ILOAD general. ¿Cómo debería modificarse IJVM para aprovechar al máximo estas instrucciones?
14. La instrucción ISHR (desplazamiento aritmético entero hacia la derecha) existe en JVM pero no en IJVM. ISHR usa los dos valores del tope de la pila y los sustituye por un solo valor, el resultado. La segunda palabra desde el tope de la pila es el operando a desplazar. Su contenido se desplaza hacia la derecha entre 0 y 31 bits, inclusive, dependiendo del valor de los cinco bits menos significativos de la palabra que está en el tope de la pila (los otros 27 bits de esa palabra no se toman en cuenta). El bit del signo se repite a la izquierda tantos bits como el contenido se desplazó. El código de operación de ISHR es 122 (0x7A).
- ¿Qué operación aritmética equivale a desplazar a la derecha con una cuenta de 2?
 - Extienda el microcódigo para incluir esta instrucción como parte de IJVM.
15. La instrucción ISHL (desplazamiento entero hacia la izquierda) existe en JVM pero no en IJVM. ISHL usa los dos valores del tope de la pila y los sustituye por un solo valor, el resultado. La segunda palabra desde el tope de la pila es el operando a desplazar. Su contenido se desplaza hacia la izquierda entre 0 y 31 bits, inclusive, dependiendo del valor de los cinco bits menos significativos de la palabra que está en el tope de la pila (los otros 27 bits de esa palabra no se toman en cuenta). Se introducen por la derecha tantos ceros como bits se desplazó el contenido. El código de operación de ISHL es 120 (0x78).
- ¿Qué operación aritmética equivale a desplazar a la izquierda con una cuenta de 2?
 - Extienda el microcódigo para incluir esta instrucción como parte de IJVM.
16. La instrucción INVOKEVIRTUAL de JVM necesita saber cuántos parámetros tiene. ¿Por qué?
17. Escriba microcódigo para implementar la instrucción POPTWO de JVM en el Mic-1. Esta instrucción obtiene dos palabras del tope de la pila.
18. Implemente la instrucción DLOAD de JVM en el Mic-2. La instrucción tiene un índice de un byte y almacena en la pila la variable local que está en esa localidad. Luego almacena también en la pila la siguiente palabra más alta.

19. Dibuje una máquina de estados finitos para llevar la puntuación en un partido de tenis. Las reglas son las siguientes. Para ganar se necesitan al menos cuatro puntos y es preciso tener al menos dos puntos más que el contrincante. Parta de un estado (0, 0), que indica que nadie tiene puntos todavía. Luego añada un estado (1, 0), que indica que A anotó un punto. Rotule la flecha de (0, 0) a (1, 0) con A. Ahora añada un estado (0, 1) que indique que B anotó, y rotule la flecha de (0, 0) con B. Siga agregando estados y flechas hasta incluir todos los posibles estados.
20. Reconsidere el problema anterior. ¿Hay estados que puedan colapsarse sin alterar el resultado del juego? Si los hay, ¿cuáles son equivalentes?
21. Dibuje una máquina de estados finitos para predicción de ramificaciones que sea más tenaz que la figura 4-42. La predicción sólo deberá cambiar después de tres predicciones equivocadas consecutivas.
22. El registro de desplazamiento de la figura 4-27 tiene una capacidad máxima de 6 bytes. ¿Podría construirse una versión más económica de la IFU con un registro desplazador de 5 bytes? ¿Y con uno de 4 bytes?
23. Después de examinar las IFU más económicas en la pregunta anterior, examinemos IFU más costosas. ¿Tendría sentido incluir un registro de desplazamiento mucho más grande en la IFU, digamos de 12 bytes? ¿Por qué sí o por qué no?
24. En el microprograma para el Mic-2, el código de if_icmpeq6 salta a T cuando Z se pone en 1. Sin embargo, el código en T es igual al de goto1. ¿Habría sido posible ir a goto1 directamente? ¿La máquina sería más rápida si así se hiciera?
25. En el Mic-4, la unidad decodificadora establece una correspondencia entre el código de operación IJVM y el índice en ROM en el que están almacenadas las microoperaciones correspondientes. Podría parecer más sencillo omitir la etapa de decodificación y alimentar el código de operación IJVM directamente a la unidad de cola de espera. Ésta podría usar el código de operación IJVM como índice para el ROM, igual que se hace en el Mic-1. ¿Qué defecto tiene este plan?
26. Una computadora tiene una caché de dos niveles. Suponga que el 80% de las referencias a la memoria aciertan en la caché de nivel 1, el 15% aciertan en la caché de segundo nivel y el 5% fallan. Los tiempos de acceso son de 5 ns, 15 ns y 60 ns, respectivamente, y los tiempos para la caché de nivel 2 y la memoria principal comienzan a contar en el momento en que se sabe que se necesitan (por ejemplo, no se inicia un acceso a la caché de nivel 2 sino hasta después de que ocurre un fallo de caché de nivel 1). Calcule el tiempo de acceso promedio.
27. Al final de la sección 4.5.1 dijimos que la asignación de escritura es mejor sólo si es probable que vayan a efectuarse varias escrituras consecutivas en la misma línea de caché. ¿Qué sucedería en el caso de una escritura seguida de varias lecturas? ¿No habría también una ganancia apreciable?
28. En el primer borrador de este libro, la figura 4-39 mostraba una caché asociativa de tres vías en lugar de una de cuatro vías. Uno de los revisores se molestó mucho, asegurando que ello crearía una confusión terrible entre los estudiantes porque tres no es una potencia de dos y las computadoras hacen todo en binario. Puesto que el cliente siempre tiene la razón, se cambió la figura a una caché asociativa de cuatro vías. ¿Tenía razón el revisor? Comente su respuesta.
29. Una computadora con una fila de procesamiento de cinco etapas maneja las ramificaciones condicionales parando durante los siguientes tres ciclos después de encontrarse con una. ¿Qué tanto perjudica esto al desempeño si el 20% de todas las instrucciones son ramificaciones condicionales? No tome en cuenta otras cosas que podrían parar la máquina, sólo las ramificaciones condicionales.

30. Suponga que una computadora prebusca hasta 20 instrucciones por adelantado. Sin embargo, cuatro de ellas en promedio son ramificaciones condicionales, cada una con una probabilidad del 90% de ser predicha correctamente. Calcule la probabilidad de que la prebúsqueda siga el camino correcto.
31. Suponga que se modifica el diseño de la máquina de la figura 4-43 de modo que tenga 16 registros en lugar de ocho. Luego modificamos I6 de modo que use R8 como destino. ¿Qué sucede en los ciclos a partir del sexto?
32. Normalmente, las dependencias causan problemas en las CPU con filas de procesamiento. ¿Existen optimaciones que puedan efectuarse con las dependencias WAW y que mejoren realmente la situación? ¿Cuáles son?
33. Reescriba el intérprete de Mic-1 de modo que ahora LV apunte a la primera variable local en lugar de al apuntador de enlace.
34. Escriba un simulador de una caché con mapeo directo de una vía. Haga que el número de entradas y el tamaño de línea sean parámetros de la simulación. Experimente con ella y prepare un informe de sus hallazgos.

5

UNIVERSIDAD DE LA REPUBLICA
FACULTAD DE INGENIERIA
DEPARTAMENTO DE
DOCUMENTACION Y BIBLIOTECA
MONTEVIDEO - URUGUAY

EL NIVEL DE ARQUITECTURA DEL CONJUNTO DE INSTRUCCIONES

En este capítulo estudiaremos con detalle el nivel de arquitectura del conjunto de instrucciones (ISA, *Instruction Set Architecture*). Este nivel se ubica entre el nivel de microarquitectura y el del sistema operativo, como vimos en la figura 1-2. Históricamente, este nivel se desarrolló antes que cualquiera de los otros niveles y, de hecho, originalmente era el único nivel. Aun hoy no es raro oír hablar de este nivel simplemente como “la arquitectura” de una máquina o a veces (incorrectamente) como “lenguaje ensamblador”.

El nivel ISA tiene una importancia especial que lo hace crucial para los arquitectos de sistemas: es la interfaz entre el software y el hardware. Si bien sería posible hacer que el hardware ejecute directamente programas escritos en C, C++, FORTRAN 90 o algún otro lenguaje de alto nivel, no sería una buena idea. Se perdería la ventaja en términos de desempeño de la compilación sobre la interpretación. Además, para ser útiles en la práctica, casi todas las computadoras deben ser capaces de ejecutar programas escritos en varios lenguajes, no sólo en uno.

La estrategia que adoptan casi todos los diseñadores de sistemas es hacer que programas en diversos lenguajes de alto nivel se traduzcan a una forma intermedia común —el nivel ISA— y construir hardware que pueda ejecutar directamente programas en el nivel ISA. El nivel ISA define la interfaz entre los compiladores y el hardware; es el lenguaje que ambos tienen que entender. La relación entre los compiladores, el nivel ISA y el hardware se muestra en la figura 5-1.

Idealmente, cuando los arquitectos diseñan una máquina nueva hablan tanto con los escritores de compiladores como con los ingenieros de hardware para averiguar qué caracte-

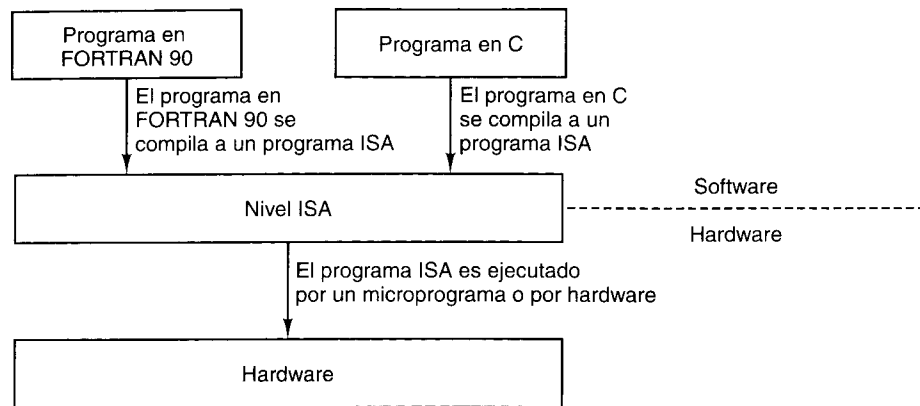


Figura 5-1. El nivel ISA es la interfaz entre los compiladores y el hardware.

rísticas quieren que se incluyan en el nivel ISA. Si los escritores de compiladores quieren alguna característica que los ingenieros no pueden implementar de forma económica (por ejemplo, una instrucción de ramificar y hacer la nómina), no se incluye. Asimismo, si la gente de hardware tiene una maravillosa característica nueva que quiere incluir (digamos una memoria en la que las palabras cuyas direcciones son números primos son superrápidas) pero la gente de software no puede pensar en un código que pueda aprovecharla, se quedará en los planos. Después de muchas negociaciones y simulaciones, surgirá un nivel ISA perfectamente optimado para los lenguajes de programación objetivo, y se implementará.

Eso es la teoría. Ahora viene la triste realidad. Cuando llega una nueva máquina, la primera pregunta que todos los clientes en potencia hacen es: “¿Es compatible con su predecesora?” La segunda es: “¿Puedo ejecutar mi sistema operativo antiguo en ella?” La tercera es: “¿Ejecutará todos mis programas de aplicación existentes sin modificación?” Si cualquiera de las respuestas es “no”, los diseñadores tendrán mucho que explicar. A los clientes generalmente no les gusta tirar a la basura todo su software viejo y comenzar otra vez desde cero.

Esta actitud somete a los arquitectos de computadoras a una presión enorme en favor de mantener inalterada la ISA al cambiar de modelo, o al menos hacerla **compatible con modelos anteriores**. Con esto queremos decir que la nueva máquina debe ejecutar programas viejos sin modificación. Sin embargo, es perfectamente aceptable que la nueva máquina tenga nuevas instrucciones y otras características que aprovechar, además del software nuevo. En términos de la figura 5-1, en tanto los diseñadores hagan a la ISA compatible con los modelos anteriores, gozan de libertad más o menos completa para hacer lo que quieran con el hardware, pues a casi nadie le interesa el verdadero hardware (o siquiera saber qué hace). Los diseñadores pueden cambiar de un diseño microprogramado a uno de ejecución directa, o añadir filas de procesamiento o recursos superescalares o cualquier otra cosa que se les ocurra, siempre que mantengan la compatibilidad con la ISA anterior. La meta es asegurarse de que los programas viejos funcionen en la máquina nueva. El reto entonces es construir mejores máquinas sujetas a la restricción de compatibilidad hacia atrás.

Con lo anterior no queremos implicar que el diseño de ISA carezca de importancia. Una buena ISA es mucho mejor que una mala, sobre todo en términos de potencia de cómputo bruta *versus* costo. Si dos diseños son equivalentes en todo lo demás, diferentes ISA podrían representar una diferencia de hasta un 25% en el desempeño. Lo que queremos destacar es que las fuerzas del mercado hacen que sea muy difícil (aunque no imposible) desechar una ISA antigua e introducir una nueva. No obstante, de vez en cuando surge una nueva ISA de aplicación general, y en mercados especializados (como sistemas incorporados o procesadores multimedia) se dan con mucha mayor frecuencia. Por tanto, entender el diseño de la ISA es importante.

¿Qué hace que una ISA sea buena? Hay dos factores primordiales. Primero, una buena ISA debe definir un conjunto de instrucciones que se pueda implementar con eficiencia en las tecnologías actuales y futuras y permita crear diseños económicos durante varias generaciones. Un diseño deficiente es más difícil de implementar; podría requerir muchas más compuertas para implementar el procesador y más memoria para ejecutar los programas. También podría ser más lento porque la ISA no permite distinguir oportunidades para traslapar operaciones, y requiere diseños mucho más complicados para alcanzar un desempeño equivalente. Un diseño que aprovecha las peculiaridades de una tecnología dada podría ser algo efímero, que ofrece una sola generación de implementaciones económicas y luego es superado por unas ISA más progresivas.

Segundo, una buena ISA debe ofrecer un objetivo claro para el código compilado. La regularidad y exhaustividad de una gama de opciones son rasgos importantes que no siempre están presentes en una ISA. Éstas son propiedades cruciales para un compilador, el cual podría tener problemas para escoger la mejor opción entre alternativas limitadas, sobre todo cuando la ISA no permite algunas alternativas aparentemente obvias. En pocas palabras, dado que la ISA es la interfaz entre el hardware y el software, debe complacer a los diseñadores de hardware (debe ser fácil de implementar con eficiencia) y a los de software (debe ser fácil generar buen código para ella).

5.1 GENERALIDADES DEL NIVEL ISA

Iniciemos nuestro estudio del nivel ISA preguntándonos qué es. Ésta podría parecer una pregunta sencilla, pero tiene más complicaciones de las que podríamos imaginar. En la sección que sigue presentaremos algunos de estos aspectos; luego examinaremos modelos de memoria, registros e instrucciones.

5.1.1 Propiedades del nivel ISA

En principio, el nivel ISA está definido por el aspecto que la máquina presenta al programador de lenguaje de máquina. Puesto que ya ninguna persona (en sus cabales) programa mucho en lenguaje de máquina, modifiquemos la definición para decir que el código en el nivel ISA es lo que un compilador produce (haciendo caso omiso por el momento de las llamadas al

sistema operativo y del lenguaje ensamblador simbólico). Para producir código en el nivel ISA, el escritor de compiladores tiene que conocer el modelo de memoria, los registros disponibles, los tipos de datos e instrucciones con que se cuenta, etc. El conjunto de toda esta información es lo que define el nivel ISA.

Según esta definición, cuestiones como si la microarquitectura está microprogramada o no, si usa filas de procesamiento o no, si es superescalar o no, etc., no forman parte del nivel ISA porque el escritor de compiladores no las percibe. Sin embargo, esta afirmación no es del todo correcta porque algunas de esas propiedades afectan el desempeño, y el escritor de compiladores sí percibe eso. Consideremos, por ejemplo, un diseño superescalar que puede emitir instrucciones una tras otra en el mismo ciclo siempre que una manipule enteros y la otra sea de punto flotante. Si el compilador alterna instrucciones de enteros con instrucciones de punto flotante, logrará un desempeño claramente mejor que si no lo hace. Por tanto, los detalles del funcionamiento superescalar *sí son* visibles en el nivel ISA, y la separación entre las capas no es tan nítida como podría parecer a primera vista.

En algunas arquitecturas el nivel ISA se especifica con un documento de definición formal, a menudo producido por un consorcio de la industria. En otras no hay tal documento. Por ejemplo, tanto SPARC V9 (la versión 9 de SPARC) como la JVM tienen definiciones oficiales (Weaver y Germond, 1994; y Lindholm y Yellin, 1997, respectivamente). El propósito de un documento reglamentado es que diferentes implementadores puedan construir la máquina y que todas ellas ejecuten exactamente el mismo software y obtengan exactamente los mismos resultados.

En el caso de SPARC, lo que se busca es que diversos fabricantes de chips produzcan chips SPARC que sean funcionalmente idénticos, con diferencias sólo en el desempeño y el precio. Para que esta idea funcione, los fabricantes de chips tienen que saber qué se supone que hace un chip SPARC (en el nivel ISA). Por tanto, el documento reglamentado especifica el modelo de memoria, los registros presentes, lo que hacen las instrucciones, etc., pero no la microarquitectura.

Tales documentos reglamentados contienen secciones **normativas**, que imponen requisitos, y secciones **informativas**, que ayudan al lector pero no forman parte de la definición formal. Las secciones normativas usan mucho palabras como *debe*, *no puede* y *debería*, para requerir, prohibir y sugerir aspectos de la arquitectura, respectivamente. Por ejemplo, un enunciado como

La ejecución de un código de operación reservado debe causar una trampa.

dice que si un programa ejecuta un código de operación que no está definido, debe causar una trampa (una interrupción en software) y no hacer caso omiso de él. Una estrategia alternativa podría ser dejar este aspecto abierto, en cuyo caso el enunciado podría ser

El efecto de ejecutar un código de operación reservado está definido por la implementación.

Esto significa que el escritor de compiladores no puede confiar en un comportamiento específico, pues los diferentes implementadores están en libertad de tomar diferentes decisiones. Casi todas las especificaciones de arquitectura van acompañadas por juegos de programas de prueba que verifican si una implementación que asegura cumplir con la especificación realmente lo hace.

Es evidente por qué el SPARC V9 tiene un documento que define su nivel ISA: para que todos los chips SPARC V9 ejecuten el mismo software. Por lo mismo, existe un documento reglamentado para JVM: a fin de que todos los intérpretes (o chips como el picoJava II) puedan ejecutar cualquier programa JVM válido. No existe un documento de definición formal para el nivel ISA del Pentium II porque Intel no quiere que para otros fabricantes sea fácil producir chips Pentium II. De hecho, Intel ha acudido a las cortes para tratar de impedir que otros vendedores produzcan clones de sus chips.

Otra propiedad importante del nivel ISA es que en casi todas las máquinas hay al menos dos modos. El **modo de Kernel** sirve para ejecutar el sistema operativo y permite la ejecución de todas las instrucciones. El **modo de usuario** sirve para ejecutar programas de aplicación y no permite que se ejecuten ciertas instrucciones peligrosas (como las que manipulan directamente la caché). En este capítulo nos concentraremos primordialmente en las instrucciones y propiedades del modo de usuario.

5.1.2 Modelos de memoria

Todas las computadoras dividen la memoria en celdas que tienen direcciones consecutivas. El tamaño de celda más común actualmente es de 8 bits, pero se han usado celdas de 1 bit hasta 60 bits (véase la figura 2-10). Una celda de 8 bits se llama **byte**. La razón para usar bytes es que los caracteres ASCII tienen 7 bits, de modo que un carácter ASCII y un bit de paridad caben en un byte. Si UNICODE llega a dominar la industria en el futuro, las computadoras podrían basarse en unidades de 16 bits numeradas consecutivamente. Después de todo, 2^4 es un número más bonito que 2^3 , ya que 4 es una potencia de 2, y 3 no lo es.

Los bytes generalmente se agrupan en palabras de 4 bytes (32 bits) u 8 bytes (64 bits), y se proporcionan instrucciones para manipular palabras enteras. Muchas arquitecturas exigen que las palabras estén alineadas respecto a sus fronteras naturales, de modo que, por ejemplo, una palabra de 4 bytes puede comenzar en la dirección 0, 4, 8, etc., pero no en la dirección 1 ni en la 2. Asimismo, una palabra de 8 bytes puede comenzar en la dirección 0, 8, 16, etc., pero no en la dirección 4 ni en la 6. La alineación de palabras de 8 bytes se ilustra en la figura 5-2.

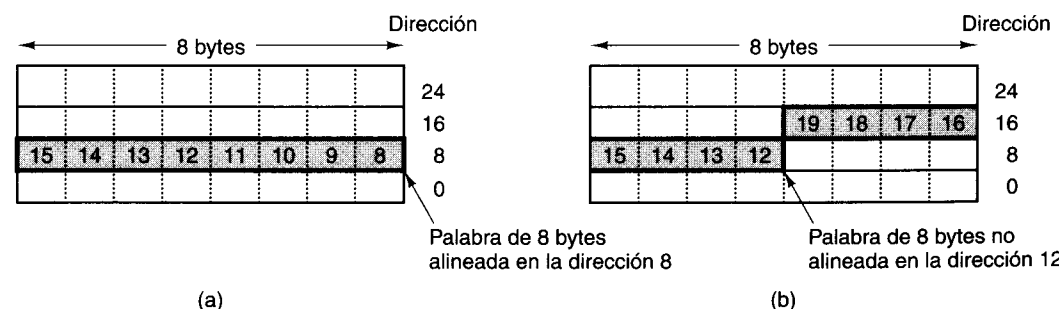


Figura 5-2. Palabra de 8 bytes en una memoria little-endian. (a) Alineada. (b) No alineada. Algunas máquinas requieren que las palabras estén alineadas en la memoria.

Normalmente se exige la alineación porque las palabras operan con mayor eficiencia así. El Pentium II, por ejemplo, que obtiene grupos de 8 bytes de la memoria, usa direcciones físicas de 36 bits, pero tiene sólo 33 bits de dirección, como se muestra en la figura 3-44. Así, el Pentium II no podría hacer una referencia no alineada a la memoria aunque quisiera, porque los tres bits de orden bajo no se especifican explícitamente; siempre son ceros, lo que obliga a todas las direcciones de memoria a ser múltiplos de 8 bytes.

Sin embargo, este requisito de alineación a veces causa problemas. En el Pentium II, los programas ISA pueden hacer referencia a palabras que comienzan en cualquier dirección, propiedad que se remonta al 8088, que tenía un bus de datos de un byte de anchura (y por ende ningún requisito de alineación de las referencias a la memoria según fronteras de 8 bytes). Si un programa del Pentium II lee una palabra de 4 bytes en la dirección 7, el hardware tiene que hacer una referencia a la memoria para obtener los bytes del 0 al 7 y una segunda referencia a la memoria para obtener los bytes del 8 al 15. Luego la CPU tiene que extraer los 4 bytes requeridos de los 16 bytes que se leyeron de la memoria y armarlos en el orden correcto para formar una palabra de 4 bytes.

La capacidad para leer palabras en direcciones arbitrarias requiere lógica adicional en el chip, lo que lo hace más grande y más costoso. A los ingenieros de diseño les encantaría deshacerse de esa capacidad y simplemente exigir a todos los programas que hagan referencias a la memoria alineadas por palabra. El problema es que cada vez que los ingenieros dicen “¿A quién le interesa ejecutar programas apolillados para el 8088 que no hacen referencia a la memoria como debe ser?” la gente de comercialización tiene una respuesta adecuada: “A nuestros clientes”.

Casi todas las máquinas tienen un solo espacio de direcciones lineal en el nivel ISA, que se extiende desde la dirección 0 hasta algún máximo, que suele ser 2^{32} bytes o 2^{64} bytes. Sin embargo, unas cuantas máquinas tienen espacios de direcciones distintos para las instrucciones y para los datos, de modo que una obtención de instrucción en la dirección 8 acude a un espacio de direcciones distinto que una obtención de datos en la dirección 8. Este esquema es más complejo que tener un solo espacio de direcciones, pero tiene dos ventajas. Primera, es posible tener 2^{32} bytes de programa y otros 2^{32} bytes de datos usando sólo direcciones de 32 bits. Segunda, como todas las escrituras se efectúan automáticamente en el espacio de datos, es imposible sobrecribir accidentalmente el programa, con lo que se elimina una fuente de errores de programa.

Cabe señalar que tener espacios de direcciones distintos para instrucciones y datos no es lo mismo que tener una caché de nivel 1 dividida. En el primer caso, la cantidad total de espacio de direcciones se duplica y las lecturas a una dirección dada producen resultados diferentes, dependiendo de si se está leyendo una palabra de instrucciones o de datos. Con una caché dividida, sigue habiendo un solo espacio de direcciones, sólo de diferentes cachés guardan diferentes partes de ese espacio.

Otro aspecto del modelo de memoria en el nivel ISA es la semántica de memoria. Es natural esperar que una instrucción **LOAD** que ocurre después de una instrucción **STORE** y que hace referencia a la misma dirección devuelva el valor que se acaba de guardar. Sin embargo, como vimos en el capítulo 4, en muchos diseños las microinstrucciones se reordenan, y existe el peligro de que la memoria no tenga el comportamiento esperado. El problema empeora en un multiprocesador, donde cada una de las CPU envía un flujo de solicitudes de lectura y escritura (posiblemente reordenadas) a una memoria compartida.

Los diseñadores de sistemas pueden adoptar varias estrategias para resolver este problema. En un extremo, todas las solicitudes de memoria se pueden colocar en serie, de modo que una se finalice antes de que se emita la siguiente. Esta estrategia perjudica el desempeño pero simplifica al máximo la semántica de memoria (todas las operaciones se ejecutan en el orden estricto del programa).

En el otro extremo, no se ofrecen en general garantías de ningún tipo. Para forzar un ordenamiento de la memoria, el programa debe ejecutar una instrucción **SYNC**, que bloquea la emisión de operaciones de memoria nuevas hasta que todas las anteriores hayan terminado. Este diseño somete a los compiladores a un gran esfuerzo porque tienen que entender los detalles del funcionamiento de la microarquitectura subyacente, pero ofrece a los diseñadores de hardware el máximo de libertad para optimar el uso de la memoria.

También puede haber modelos de memoria intermedios, en los que el hardware bloquea automáticamente la emisión de ciertas referencias a la memoria (por ejemplo, las que implican una dependencia RAW o WAR) pero no otras. Si bien es molesto que todas estas peculiaridades causadas por la microarquitectura queden expuestas en el nivel ISA (al menos para los escritores de compiladores y los programadores en lenguaje ensamblador), es la tendencia actual. Dicha tendencia se debe a las implementaciones subyacentes, como el reordenamiento de microinstrucciones, las filas de procesamiento de gran capacidad, las cachés con múltiples niveles, y demás. Veremos más ejemplos de tales efectos artificiales más adelante en este capítulo.

5.1.3 Registros

Todas las computadoras tienen algunos registros que son visibles en el nivel ISA. Su función es controlar la ejecución del programa, almacenar resultados temporales y otras cosas. En general, los registros visibles en el nivel de microarquitectura, como **TOS** y **MAR** en la figura 4-1, no son visibles en el nivel ISA. Sin embargo, unos cuantos de ellos, como el contador de programa y el apuntador de la pila, son visibles en ambos niveles. Por otra parte, los registros visibles en el nivel ISA siempre son visibles en el nivel de microarquitectura porque ahí es donde se implementan.

Los registros del nivel ISA se pueden dividir en dos categorías amplias: registros de propósito especial y registros de propósito general. Los primeros incluyen cosas como el contador de programa y el apuntador a la pila, además de otros registros con funciones específicas. En contraste, los registros de propósito general sirven para guardar variables locales clave y los resultados intermedios de los cálculos. Su función principal es proporcionar acceso rápido a datos que se usan continuamente (básicamente para evitar accesos a la memoria). Las máquinas RISC, con sus CPU rápidas y memorias (relativamente) lentas, casi siempre tienen por lo menos 32 registros de propósito general, y la tendencia en los nuevos diseños de CPU es a tener aún más.

En algunas máquinas los registros de propósito general son totalmente simétricos e intercambiables. Si todos los registros son equivalentes, un compilador puede usar **R1** para contener un resultado temporal, pero igual podría usar **R25**. El registro escogido no importa.

En cambio, en otras máquinas algunos de los registros de propósito general podrían tener algo de especial. Por ejemplo, en el Pentium II hay un registro llamado **EDX** que se

puede usar como registro general, pero que también recibe la mitad del producto en una multiplicación y que contiene la mitad del dividendo en una división.

Aun si los registros de propósito general son totalmente intercambiables, es común que el sistema operativo o los compiladores adopten convenciones sobre su uso. Por ejemplo, algunos registros podrían contener parámetros de procedimientos invocados y otros podrían usarse como registros de borrador. Si un compilador coloca una variable local importante en R1 y luego invoca a un procedimiento de biblioteca que cree que R1 es un registro de borrador que puede usar, cuando el procedimiento regrese, R1 podría contener basura. Si existen convenciones en el nivel de sistema sobre el uso de registros, es recomendable que los compiladores y los programadores en lenguaje ensamblador se ajusten a ellas.

Además de los registros en el nivel ISA que los programas de usuario pueden ver, siempre hay una cantidad sustancial de registros de propósito especial que sólo son accesibles en modo de kernel. Estos registros controlan las diversas cachés, memoria, dispositivos de E/S y otras características de hardware de la máquina. Sólo el sistema operativo los usa, así que los compiladores y los usuarios no tienen que saber de ellos.

Un registro de control que es una especie de híbrido kernel/usuario es el **registro de banderas o palabra de estado del programa (PSW, Program Status Word)**. Este registro contiene diversos bits que la CPU necesita. Los bits más importantes son los **códigos de condición**. Estos bits se ajustan en cada ciclo de la ALU y reflejan la situación del resultado de la operación más reciente. Entre los bits de condición más comunes están

N – Se enciende si el resultado fue negativo

Z – Se enciende si el resultado fue cero

V – Se enciende si el resultado causó un desbordamiento

C – Se enciende si el resultado causó un acarreo de salida del bit de la extrema izquierda

A – Se enciende si hubo un acarreo de salida del bit 3 (acarreo auxiliar)

P – Se enciende si el resultado tuvo paridad par

Los códigos de condición son importantes porque las instrucciones de comparación y ramificación condicional (es decir, las instrucciones de salto condicional) los usan. Por ejemplo, la instrucción **CMP** generalmente resta dos operandos y establece los códigos de condición con base en la diferencia. Si los operandos son iguales, la diferencia será cero y se encenderá el bit de código de condición Z de la PSW. Una instrucción **BEQ** (ramificar si son iguales, *Branch Equal*) subsecuente prueba el bit Z y toma la rama si está encendido.

La PSW contiene más que sólo los códigos de condición, pero el contenido total varía de una máquina a otra. Otros campos comunes indican el modo de la máquina (es decir, de kernel o de usuario), un bit de rastreo (que sirve para depurar), el nivel de prioridad de la CPU y la **habilitación** de interrupciones. En muchos casos es posible leer la PSW en modo de usuario, pero algunos de los campos sólo pueden modificarse en modo de kernel (por ejemplo, el bit de modo de usuario/kernel).

5.1.4 Instrucciones

La característica principal del nivel de ISA es su conjunto de instrucciones de máquina. Éstas controlan lo que la máquina puede hacer. Siempre hay instrucciones **LOAD** y **STORE** (en una forma o en otra) para trasladar datos entre la memoria y los registros, e instrucciones **MOVE** para copiar datos entre los registros. Siempre se cuenta con instrucciones aritméticas, lo mismo que instrucciones booleanas e instrucciones para comparar datos y tomar una rama u otra según los resultados. Ya hemos visto algunas instrucciones ISA representativas (vea la figura 4-11) y estudiaremos muchas más en este capítulo.

5.1.5 Generalidades del nivel ISA del Pentium II

En este capítulo veremos tres ISA muy diferentes: la IA-32 de Intel, encarnada en el Pentium II, la arquitectura SPARC versión 9, implementada en los procesadores UltraSPARC, y JVM, implementada en el picoJava II. La intención no es ofrecer una descripción exhaustiva de cualquiera de las ISA, sino mostrar los aspectos importantes de una ISA y cómo dichos aspectos pueden variar de una ISA a otra. Comencemos con el Pentium II.

El procesador Pentium II ha evolucionado a través de muchas generaciones, y su linaje se remonta a algunos de los primeros microprocesadores jamás construidos, como vimos en el capítulo 1. Si bien la ISA básica mantiene apoyo completo para la ejecución de programas escritos para los procesadores 8086 y 8088 (que tenían la misma ISA), contiene restos del 8080, un procesador de 8 bits popular en la década de los setenta. El 8080, a su vez, tuvo una marcada influencia de restricciones de compatibilidad con el 8008, más antiguo aún, que se basó en el 4004, un chip de 4 bits que se usaba cuando los dinosaurios dominaban el planeta.

Desde el punto de vista del software, el 8086 y el 8088 eran máquinas sencillas de 16 bits (aunque el 8088 tenía un bus de datos de 8 bits). Su sucesor, el 80286, también fue una máquina de 16 bits. Su ventaja principal fue un mayor espacio de direcciones, aunque pocos programas lo llegaron a usar porque consistía en 16,384 segmentos de 64K en lugar de una memoria lineal de 2^{24} bytes.

El 80386 fue la primera máquina de 32 bits de la familia Intel. Todas las máquinas subsecuentes (80486, Pentium, Pentium Pro, Pentium II, Celeron y Xeon) han tenido prácticamente la misma arquitectura de 32 bits que el 80386, llamada **IA-32**, por lo que es ésta arquitectura en la que nos concentraremos aquí. El único cambio arquitectónico importante desde el 80386 fue la introducción de las instrucciones MMX en versiones posteriores del Pentium, y su subsecuente inclusión en el Pentium II y otros procesadores.

El Pentium II tiene tres modos de operación, dos de los cuales hacen que actúe como un 8088. En **modo real**, todas las funciones que se han añadido desde el 8088 se desactivan y el Pentium II se comporta como un simple 8088. Si cualquier programa hace algo indebido, toda la máquina se cae. Si Intel hubiera diseñado a los seres humanos, habría incluido un bit que los hiciera revertir a modo de chimpancé (casi todo el cerebro inhabilitado, no habla, come casi únicamente plátanos, etc.).

Un paso más arriba está el **modo 8086 virtual**, que permite ejecutar programas para 8088 viejos con protección. En este modo, un verdadero sistema operativo controla toda la

máquina. Para ejecutar un programa para 8088, el sistema operativo crea un entorno aislado especial que actúa como un 8088, excepto que si su programa se cae se notifica al sistema operativo en lugar de hacer que la máquina se caiga. Cuando un usuario de Windows inicia una ventana MS-DOS, el programa que se ejecuta ahí se inicia en modo 8086 virtual para proteger a Windows de programas MS-DOS mal comportados.

El último modo es el modo protegido, en el que el Pentium II realmente actúa como Pentium II y no como un 8088 muy costoso. Hay cuatro niveles de privilegio controlados por bits de la PSW. El nivel 0 corresponde al modo de kernel en otras computadoras y tiene pleno acceso a la máquina. El sistema operativo usa este nivel. El nivel 3 es para programas de usuario; bloquea el acceso a ciertas instrucciones y registros de control críticos para evitar que un programa de usuario desbocado haga caer a toda la máquina. Los niveles 1 y 2 casi nunca se usan.

El Pentium II tiene un espacio de direcciones enorme, con la memoria dividida en 16,384 segmentos, cada uno de los cuales va desde la dirección 0 hasta la dirección $2^{32} - 1$. Sin embargo, casi todos los sistemas operativos (incluidos UNIX y todas las versiones de Windows) sólo reconocen un segmento, así que la generalidad de los programas de aplicación ven un espacio de direcciones lineal de 2^{32} bytes, y a veces una parte de éste está ocupado por el sistema operativo. Cada byte del espacio de direcciones tiene su propia dirección, y las palabras son de 32 bits. El formato para guardar las palabras es little endian (el byte de orden más bajo tiene la dirección más baja).

Los registros del Pentium II se muestran en la figura 5-3. Los primeros cuatro registros, EAX, EBX, ECX y EDX son registros de 32 bits de propósito más o menos general, aunque cada uno tiene sus propias peculiaridades. EAX es el principal registro aritmético; EBX es bueno para contener apuntadores (direcciones de memoria); ECX desempeña un papel en los ciclos; EDX se necesita para la multiplicación y la división donde, junto con EAX, guarda productos y dividendos de 64 bits. Cada uno de estos registros contiene un registro de 16 bits en los 16 bits de orden bajo y un registro de 8 bits en los 8 bits de orden bajo. Estos registros facilitan la manipulación de cantidades de 16 y 8 bits, respectivamente. El 8088 y el 80286 sólo tenían los registros de 8 y 16 bits. Los registros de 32 bits se añadieron en el 80386, junto con el prefijo E, que significa Extendido.

Los siguientes tres registros también son de propósito más o menos general, pero con más peculiaridades. Los registros ESI y EDI se incluyeron para contener apuntadores a la memoria, sobre todo para las instrucciones de manipulación de cadenas en hardware, en las que ESI apunta a la cadena de origen y EDI apunta a la cadena de destino. El registro EBP también es para apuntadores; generalmente se usa para apuntar a la base del marco de pila vigente, como hace LV en IJVM. Cuando un registro (como EBP) sirve para apuntar a la base del marco de pila local, suele llamársele **apuntador de cuadro**. Por último, ESP es el apuntador de la pila.

El siguiente grupo de registros, CS a GS, son registros de segmento. En gran medida, se trata de trilobites electrónicos, fósiles que tienen que ver con la forma en que el 8088 trataba de direccionar 2^{20} bytes de memoria usando direcciones de 16 bits. Baste decir que cuando el Pentium II se configura para usar un solo espacio de direcciones lineal de 32 bits, podemos hacer caso omiso de ellos sin peligro. Luego viene el EIP, que es el contador de programa

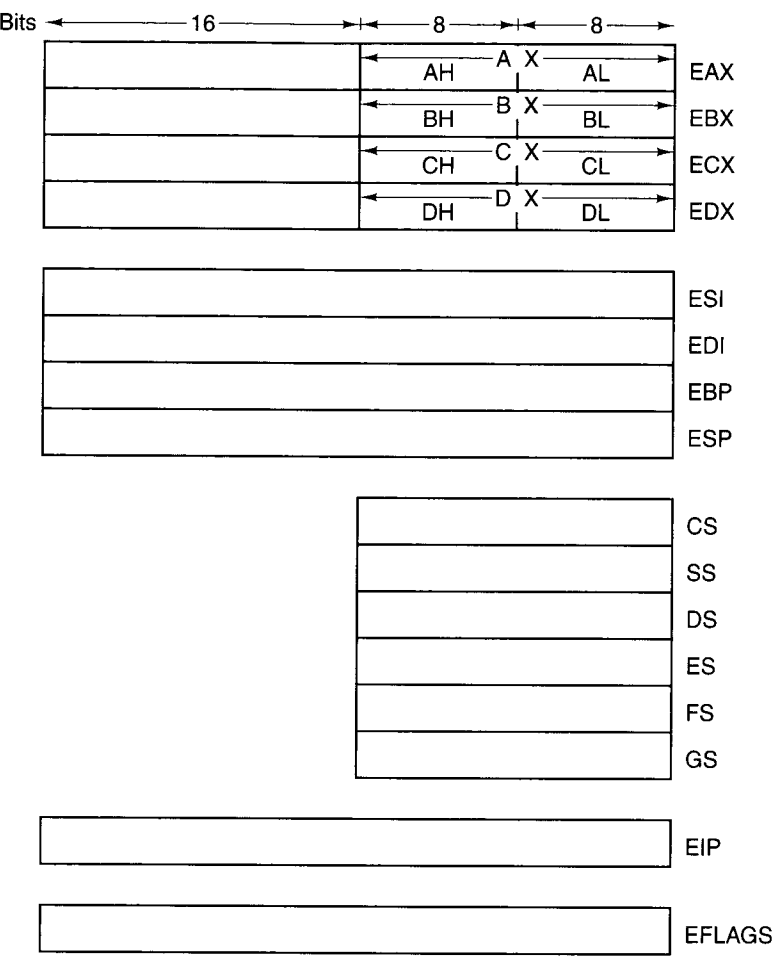


Figura 5-3. Los registros primarios del Pentium II.

(Extended Instruction Pointer, apuntador a instrucciones extendido). Por último, llegamos a EFLAGS, que es la PSW.

5.1.6 Generalidades del nivel ISA del UltraSPARC II

Sun Microsystems introdujo la arquitectura SPARC en 1987. SPARC fue una de las primeras arquitecturas comerciales calificadas como RISC, y se basó en gran medida en las investigaciones realizadas en Berkeley a fines de la década de los ochenta (Patterson, 1985; Patterson y Séquin, 1982). El SPARC original era una arquitectura de 32 bits, pero el UltraSPARC II es una máquina de 64 bits basada en la versión 9 de la arquitectura, y es la que describiremos en este capítulo. Por congruencia con el resto del libro, nos referiremos al UltraSPARC II aquí, pero en el nivel ISA todos los UltraSPARC son idénticos.

La estructura de memoria del UltraSPARC II es sencilla y clara: la memoria direccionable es una formación lineal de 2^{64} bytes. Lo malo es que esta memoria es tan grande (18,446,744,073,709,551,616 bytes) que ninguna máquina actual puede implementarla. Las implementaciones actuales limitan el tamaño de las direcciones a las que pueden acceder (2^{44} bytes en el UltraSPARC II), pero esto aumentará en modelos futuros. El orden por omisión de los bytes es big endian, pero puede convertirse en little endian encendiendo un bit en la PSW.

Es importante que la ISA tenga un límite más alto del que necesitan las implementaciones, porque las implementaciones futuras con toda seguridad necesitarán aumentar el tamaño de la memoria a la que el procesador puede acceder. Uno de los problemas más graves que surgen en las arquitecturas que tienen éxito ha sido que su ISA limita la cantidad de memoria direccionable. En computación, el único error que no se puede subsanar es la falta de suficientes bits. Un día sus nietos le preguntarán cómo las computadoras lograban hacer algo con direcciones de sólo 32 bits y sólo 4 GB de memoria real.

El ISA de SPARC es claro, aunque la organización de los registros es un tanto compleja en un intento por hacer a las llamadas a procedimientos más eficientes. La experiencia muestra que la organización de los registros no vale el trabajo que se invierte en ella, pero la regla de la compatibilidad con modelos anteriores hizo imposible deshacerse de ella.

El UltraSPARC II tiene dos grupos de registros: 32 registros de propósito general de 64 bits y 32 registros de punto flotante. Los registros de propósito general se llaman R0 a R31 aunque en ciertos contextos se usan otros nombres. Los nombres alternativos y las funciones de los registros se muestran en la figura 5-4.

Registro	Nombre alt.	Función
R0	G0	Conectado a 0. Escribir en él no hace nada
R1 – R7	G1 – G7	Contiene variables globales
R8 – R13	O0 – O5	Contiene parámetros al procedimiento invocado
R14	SP	Apuntador a la pila
R15	O7	Registro de borrador
R16 – R23	L0 – L7	Contiene variables locales para el procedimiento actual
R24 – R29	I0 – I5	Contiene parámetros entrantes
R30	FP	Apuntador a la base del marco de pila vigente
R31	I7	Contiene dirección de retorno del procedimiento actual

Figura 5-4. Los registros generales del UltraSPARC II.

Todos los registros de propósito general tienen una capacidad de 64 bits y, con excepción de R0 que siempre es 0, pueden ser leídos o escritos por diversas instrucciones de carga y

almacenamiento. Los usos que se dan en la figura 5-4 se basan en parte en la convención, pero también en cómo el hardware los trata. En general, no es prudente darles un uso distinto del que se indica en la figura 5-4 a menos que usted sea un Gurú SPARC Cinta Negra y de veras sepa perfectamente lo que está haciendo. Es responsabilidad del compilador o del programador asegurarse de que el programa accese a los registros correctamente y realice los tipos correctos de operaciones aritméticas con ellos. Por ejemplo, es muy fácil cargar números de punto flotante en los registros generales y luego realizar suma de enteros con ellos, operación que producirá resultados sin sentido pero que la CPU realizará de buena gana si se le pide.

Las variables globales sirven para retener constantes, variables y apuntadores que se necesitan en todos los procedimientos, aunque pueden guardarse y volverse a cargar al salir de un procedimiento y entrar en él si es necesario. Los registros Ix y Ox se usan para pasar parámetros a procedimientos y así evitar referencias a la memoria. Más adelante explicaremos cómo funciona esto.

Tres registros dedicados se usan para cosas especiales. Los registros FP y SP delimitan el cuadro vigente. El primero apunta a la base del cuadro vigente y sirve para direccionar variables locales, exactamente igual como hacía LV en la figura 4-10. El segundo indica el tope de pila actual y fluctúa a medida que se meten palabras en la pila y se sacan de ella. En cambio, FP sólo cambia cuando se invoca un procedimiento o cuando se regresa de un procedimiento. El tercer registro de propósito especial es R31; se usa en llamadas a procedimientos para guardar la dirección de retorno.

El UltraSPARC II en realidad tiene más de 32 registros de propósito general, aunque sólo 32 son visibles para el programa en un momento dado. Esta característica llamada **ventana de registros**, se incluyó para apoyar de forma eficiente las llamadas a procedimientos, y se ilustra en la figura 5-5. La idea básica es emular una pila, pero usando registros. Es decir, en realidad hay varios conjuntos de registros, así como hay varios cuadros en una pila. Exactamente 32 registros generales son visibles en un instante dado. El registro CWP (apuntador a la ventana vigente, *Current Window Pointer*) indica cuál conjunto de registros se está usando actualmente.

La instrucción de llamada a procedimiento oculta el conjunto anterior de registros y proporciona un conjunto nuevo para que lo use el procedimiento invocado decrementando CWP. Sin embargo, algunos registros se perpetúan del procedimiento invocador al procedimiento invocado, lo que ofrece un mecanismo eficiente para pasar parámetros entre procedimientos. Esta técnica opera cambiando el nombre de algunos de los registros: después de la llamada al procedimiento, los antiguos registros de salida, R8 a R15, siguen siendo visibles, pero ahora son los registros de entrada, R24 a R31. Sin embargo, los ocho registros globales no cambian, es decir, siempre son el mismo conjunto de registros.

A diferencia de la memoria, que es casi infinita (al menos en lo que toca a la pila), cuando los procedimientos se anidan con demasiada profundidad la máquina se queda sin ventanas de registros desocupadas. En ese punto el conjunto de mayor antigüedad se vacía a la memoria para desocupar un conjunto. Asimismo, después de muchos retornos de procedimientos, podría ser necesario traer un conjunto de registros de la memoria. En general, esta complejidad es una lata y probablemente no vale la pena. Sólo ayuda cuando las llamadas no se anidan con demasiada profundidad.

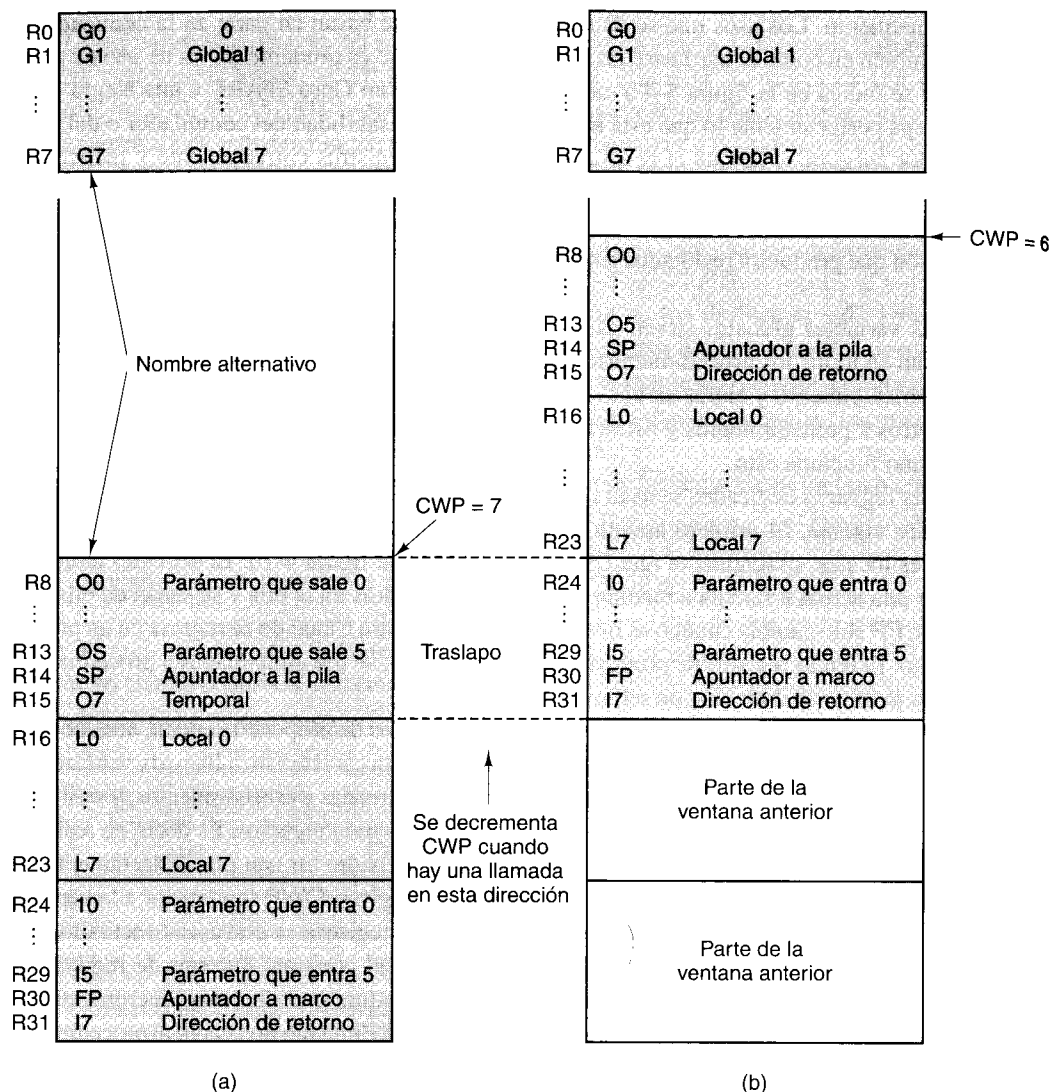


Figura 5-5. Operación de las ventanas de registros del UltraSPARC II.

El UltraSPARC II también tiene 32 registros de punto flotante, que pueden contener valores de 32 bits (precisión sencilla) o bien de 64 bits (doble precisión). También es posible usar pares de estos registros para manejar valores de 128 bits (cuádruple precisión).

La arquitectura del UltraSPARC II es una **arquitectura de carga/almacenamiento**. Es decir, las únicas operaciones que accesan a la memoria directamente son los **LOAD** y **STORE**, instrucciones para trasladar datos entre los registros y la memoria. Todos los operandos para instrucciones aritméticas y lógicas deben provenir de registros o ser proporcionados por la instrucción (no la memoria), y todos los resultados se deben guardar en un registro (no en la memoria).

5.1.7 Generalidades del nivel ISA de la Máquina Virtual Java (JVM)

El nivel ISA de la JVM es poco común, pero limpio y sencillo. Ya lo vimos en su mayor parte cuando estudiamos la JVM. El modelo de memoria JVM es igual al modelo IJVM que presentamos en el capítulo 4 e ilustramos en la figura 4.10, pero con una región adicional, que presentaremos en breve. El orden de los bytes es big endian.

La memoria tiene cuatro regiones principales: el marco de variables locales, la pila de operandos, el área de métodos y la reserva de constantes. Recuerde que las implementaciones Mic-x de IJVM tienen los registros LV, SP, PC y CPP que apuntan a estas regiones. Todos los accesos a la memoria deben efectuarse como una distancia respecto a uno de estos registros; nunca se usan apuntadores ni direcciones de memoria absolutas. Aunque JVM no requiere estos registros (ni los menciona), es probable que la mayor parte de las implementaciones los tengan o tengan algo muy parecido.

La ausencia de apuntadores para acceder a variables locales y constantes no es un accidente; es crucial para alcanzar una de las principales metas de diseño de Java: que los usuarios puedan bajar un programa JVM (binario) de Internet y ejecutarlo de forma segura, sin que el programa pueda espiar o dañar la máquina en la que se está ejecutando. Al limitar el uso de apuntadores, puede aumentarse la seguridad.

Recuerde que ninguna de las regiones de memoria definidas en IJVM podía ser muy grande. El área de métodos sólo podía tener 64 KB. Asimismo, el espacio ocupado por las variables locales no podía ser mayor que 64 KB. También la reserva de constantes está limitada a 64 KB. JVM tiene exactamente las mismas restricciones y por la misma razón exactamente: las distancias que se usan como índices para direccionar estas regiones están limitadas a 16 bits.

Desde luego, el área del marco de variables locales es distinta para cada procedimiento, de modo que cada procedimiento recién invocado tiene 64 KB para sus propias variables locales. Asimismo, la reserva de constantes es distinta para cada clase, de modo que cada una puede tener sus propios 64 KB. Aun así, no hay lugar para guardar arreglos grandes o estructuras de datos que se crean dinámicamente como las listas o los árboles. Para resolver este problema, JVM contiene una región extra de memoria, el **montículo** (*heap*), que se puede asignar para guardar objetos dinámicos o muy grandes. Cuando el compilador de Java ve un enunciado como

```
int a[] = new int[4096]
```

genera una llamada al asignador de memoria de tiempo de ejecución, que asigna el espacio en el montículo y devuelve un apuntador a él. Así pues, sí se usan apuntadores en JVM, pero de una forma muy controlada para que los programadores no puedan manipularlos directamente.

Si se crean suficientes estructuras de datos nuevas en el montículo, llegará un momento en el que éste se quedará sin espacio. Cuando el sistema de tiempo de ejecución de Java detecta que el montículo está (casi) lleno, invoca un programa llamado **recolector de basura** que busca en el montículo objetos que ya no se están usando. Se emplea un algoritmo sofisticado (generacional). Luego, los objetos no utilizados se desechan para desocupar espacio en el montículo. La asignación con **new** y la liberación con el recolector de basura automático constituyen un esquema de gestión de memoria totalmente diferente del uso de una pila para variables locales. Las dos técnicas son complementarias; cada una tiene su sitio.

JVM no tiene registros de propósito general que puedan cargarse o almacenarse bajo control del programa; es una máquina de pila pura. Si bien ésta es una decisión poco común en estos días, el resultado es una ISA sencilla y elegante, para la cual es fácil compilar.

El problema principal de las máquinas de pila es el gran número de referencias a la memoria que se requieren. Sin embargo, como vimos en la sección 4.6.3, un diseño ingenioso puede eliminar una gran cantidad de ellas mediante el plegado de varias instrucciones JVM. Además, la sencillez de la arquitectura implica que se puede implementar en una cantidad de silicio pequeña, lo que deja la mayor parte del área del chip disponible para una enorme caché de nivel 1, lo que reduce aún más el número de referencias a la memoria. (El picoJava II tiene un caché de cuando más 16 KB + 16 KB porque la meta era construir un chip de muy bajo costo más que uno muy veloz.)

5.2 TIPOS DE DATOS

Todas las computadoras necesitan datos. De hecho, la tarea principal de muchos sistemas de computadora es procesar datos financieros, comerciales, científicos, de ingeniería o de otro tipo. Los datos tienen que representarse de alguna forma específica dentro de la computadora. En el nivel ISA se emplean diversos tipos de datos, que explicaremos a continuación.

Una cuestión fundamental es si existe o no apoyo de hardware para un tipo de datos en particular. Apoyo de hardware significa que una o más instrucciones esperan datos en un formato dado, y el usuario no puede escoger un formato distinto. Por ejemplo, los contadores tienen el peculiar hábito de escribir los números negativos con el signo menos a la derecha del número, en lugar de a la izquierda, donde lo ponen los especialistas en computación. Suponga que, en un intento por impresionar a su jefe, el encargado del centro de cómputo de un bufete contable cambia todos los números de todas las computadoras de modo que el bit de la extrema derecha (en lugar del de la extrema izquierda) haga las veces de bit de signo. No cabe duda de que el jefe quedaría muy impresionado... porque todo el software dejaría de funcionar correctamente. El hardware espera cierto formato para los enteros y no funciona correctamente si recibe algo distinto.

Consideremos ahora otro bufete contable, uno que acaba de obtener un contrato para verificar la deuda federal (cuánto debe el gobierno federal a sus acreedores). No sería factible usar aritmética de 32 bits aquí porque los números en cuestión son mayores que 2^{32} (unos 4000 millones). Una solución es usar dos enteros de 32 bits para representar cada número, lo que da 64 bits en total. Si la máquina no reconoce este tipo de números de **doble precisión**, todas las operaciones aritméticas que se realicen con ellos se tendrán que efectuar en software, pero las dos partes pueden estar en cualquier orden porque al hardware no le importa. Éste es un ejemplo de tipo de datos sin apoyo de hardware y por tanto sin una representación obligatoria en hardware. En las secciones que siguen examinaremos los tipos de datos que tienen apoyo de hardware y que por ello requieren formatos específicos.

5.2.1 Tipos de datos numéricos

Los tipos de datos se pueden dividir en dos categorías: numéricos y no numéricos. El principal de los tipos numéricos es el tipo entero. Los enteros tienen diversas longitudes, como 8, 16, 32 y 64 bits. Los enteros cuentan cosas (por ejemplo, el número de destornilladores que una ferretería tiene en existencia), identifican cosas (por ejemplo, números de cuenta bancarios) y mucho más. Casi todas las computadoras modernas almacenan los enteros en notación binaria de complemento a dos, aunque se han llegado a usar otros sistemas. Los números binarios se tratan en el Apéndice A.

Algunas computadoras reconocen enteros sin signo además de enteros con signo. En el caso de los enteros sin signo, no hay bit de signo y todos los bits contienen datos. Este tipo de datos tiene la ventaja de contar con un bit extra de modo que, por ejemplo, una palabra de 32 bits puede contener un entero sin signo dentro del intervalo de 0 a $2^{32} - 1$, inclusive. En cambio, un entero de 32 bits con signo en formato de complemento a dos sólo puede manejar números hasta $2^{31} - 1$ aunque, por supuesto, también puede manejar números negativos.

Si un número no se puede expresar como un entero, digamos 3.5, se usan números de punto flotante, los cuales se tratan en el Apéndice B. Su longitud es de 32, 64 o a veces 128 bits. Casi todas las computadoras cuentan con instrucciones para efectuar operaciones aritméticas de punto flotante, y muchas cuentan con registros distintos para contener operandos enteros y operandos de punto flotante.

Algunos lenguajes de programación, entre los que sobresale COBOL, permiten números decimales como tipo de datos. Las máquinas diseñadas para ejecutar con frecuencia programas en COBOL suelen manejar números decimales en hardware, por lo regular codificando un número decimal en 4 bits y empaquetando dos números decimales en cada byte (formato decimal codificado en binario). Sin embargo, la aritmética no funciona correctamente con números decimales empaquetados, por lo que se requieren instrucciones especiales para corregir la aritmética decimal. Dichas instrucciones necesitan conocer el acarreo de salida del bit 3. Es por ello que el código de condición a menudo incluye un bit de acarreo auxiliar. Por cierto, el tristemente famoso problema del año 2000 (Y2K) fue causado por programadores en COBOL que decidieron que sería más económico representar el año como dos dígitos decimales que como un número binario de 16 bits. Vaya optimización.

5.2.2 Tipos de datos no numéricos

Aunque las primeras computadoras en su mayoría se ganaban la vida triturando números, las computadoras modernas a menudo se usan para aplicaciones no numéricas, como procesamiento de textos o gestión de bases de datos. Para estas aplicaciones se requieren otros tipos de datos que en muchos casos cuentan con apoyo de instrucciones en el nivel ISA. Es evidente que los caracteres son importantes aquí, aunque no todas las computadoras ofrecen apoyo de hardware para ellos. Los códigos de caracteres más comunes son ASCII y UNICODE, que manejan caracteres de 7 bits y de 16 bits, respectivamente. Ambos se describieron en el capítulo 2.

No es raro que el nivel ISA cuente con instrucciones especiales diseñadas para manejar cadenas de caracteres, es decir, series consecutivas de caracteres. Estas cadenas en ocasiones

se delimitan con un carácter especial al final. Como alternativa, puede incluirse un campo de longitud de cadena para saber dónde termina ésta. Las instrucciones pueden realizar funciones de copiado, búsqueda, edición y otras con cadenas.

Los valores booleanos también son importantes. Un valor booleano puede adoptar uno de dos valores: verdadero (*true*) o falso (*false*). En teoría, un solo bit puede representar un valor booleano, con 0 como falso y 1 como verdadero (o viceversa). En la práctica se usa un byte o una palabra para cada valor booleano porque los bits individuales de un byte no tienen cada uno su propia dirección y por tanto el acceso a ellos no es fácil. Un sistema común sigue la convención de que 0 significa falso y cualquier otra cosa significa verdadero.

La única situación en la que un valor booleano normalmente se representa con un bit es cuando existe un arreglo de ellos, de modo que una palabra de 32 bits puede contener 32 valores booleanos. Semejante estructura de datos se denomina **mapa de bits** y se presenta en muchos contextos. Por ejemplo, se puede usar un mapa de bits para seguir la pista a los bloques libres de un disco. Si el disco tiene n bloques, el mapa de bits tiene n bits.

Nuestro último tipo de datos es el apuntador, que es sólo una dirección de máquina. Ya hemos visto muchos apuntadores. En las máquinas Mic-x, SP, PC, LV y CPP son ejemplos de apuntadores. Accesar a una variable que está a una distancia fija de un apuntador, que es como funciona ILOAD, es de lo más común en todas las máquinas.

5.2.3 Tipos de datos en el Pentium II

El Pentium II reconoce enteros con signo en complemento a dos, enteros sin signo, números decimales codificados en binario y números de punto flotante IEEE 754, como se indica en la figura 5-6. A causa de sus orígenes como una humilde máquina de 8 bits/16 bits, el Pentium II maneja también registros con estas longitudes y cuenta con muchas instrucciones para realizar operaciones aritméticas, booleanas y de comparación. Los operandos no tienen que estar alineados en la memoria, pero se obtiene un mejor desempeño si las direcciones de palabra son múltiplos de 4 bytes.

Tipo	8 bits	16 bits	32 bits	64 bits	128 bits
Entero con signo	×	×	×		
Entero sin signo	×	×	×		
Entero decimal codificado en binario	×				
Punto flotante			×	×	

Figura 5-6. Tipos de datos numéricos del Pentium II. Los tipos que se manejan se indican con ×.

El Pentium II también es bueno para manipular caracteres ASCII de 8 bits: hay instrucciones especiales para copiar y buscar cadenas de caracteres. Estas instrucciones se pueden usar tanto con cadenas cuya longitud se conoce desde antes, como con cadenas cuyo final está marcado; se usan a menudo en bibliotecas de manipulación de cadenas.

5.2.4 Tipos de datos en el UltraSPARC II

El UltraSPARC II reconoce una amplia gama de formatos de datos, como se muestra en la figura 5-7. En el caso de enteros, el UltraSPARC II reconoce operandos de 8, 16, 32 y 64 bits, tanto con signo como sin signo. Los enteros con signo se expresan como complemento a dos. Se manejan operandos de punto flotante de 32, 64 y 128 bits, y se ajustan a la norma IEEE 754 (para los números de 32 y 64 bits). No se reconocen los números decimales codificados en binario. Todos los operandos deben estar alineados en la memoria.

Tipo	8 bits	16 bits	32 bits	64 bits	128 bits
Entero con signo	×	×	×	×	
Entero sin signo	×	×	×	×	
Entero decimal codificado en binario					
Punto flotante			×	×	×

Figura 5-7. Tipos de datos numéricos del UltraSPARC II.

El UltraSPARC II tiene una fuerte orientación hacia los registros y casi todas las instrucciones operan con registros de 64 bits. Los tipos de datos de caracteres y cadenas no cuentan con apoyo de instrucciones especiales en hardware.

5.2.5 Tipos de datos en la Máquina Virtual Java

Java es un lenguaje con tipos fuertes, lo que significa que todo operando tiene un tipo y tamaño específicos que se conocen en el momento de la compilación. Estos tipos se reflejan en los tipos de tiempo de ejecución reconocidos por JVM. JVM reconoce los tipos numéricos que se indican en la figura 5-8. Los enteros con signo se representan como complemento a dos. El lenguaje Java no incluye enteros sin signo y JVM no los reconoce, como tampoco reconoce los números decimales codificados en binario.

Tipo	8 bits	16 bits	32 bits	64 bits	128 bits
Entero con signo	×	×	×	×	
Entero sin signo					
Entero decimal codificado en binario					
Punto flotante			×	×	

Figura 5-8. Tipos de datos numéricos de la JVM.

JVM reconoce caracteres, pero como caracteres UNICODE en lugar de los caracteres ASCII de 8 bits más tradicionales. El apoyo para apuntadores está limitado en su mayor parte al uso interno del compilador y del sistema de tiempo de ejecución, pues los programas de usuario no pueden acceder directamente a apuntadores, los cuales se usan principalmente para hacer referencia a objetos.

5.3 FORMATOS DE INSTRUCCIONES

Una instrucción consiste en un código de operación generalmente acompañado por información adicional como la dirección de los operandos y el destino de los resultados. El tema general de especificar dónde están los operandos (es decir, sus direcciones) se llama **direccionamiento** y nos ocuparemos de él ahora.

La figura 5-9 muestra varios posibles formatos para instrucciones de nivel 2. Las instrucciones siempre tienen un código de operación que indica lo que hace la instrucción. Puede haber cero, una, dos o tres direcciones presentes.

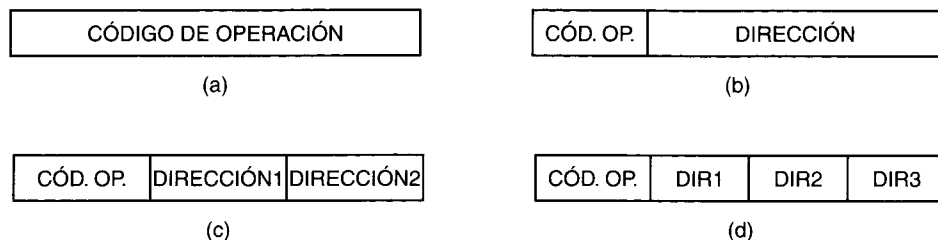


Figura 5-9. Cuatro formatos de instrucción comunes: (a) Instrucción con cero direcciones. (b) Instrucción con una dirección. (c) Instrucción con dos direcciones. (d) Instrucción con tres direcciones.

En algunas máquinas todas las instrucciones tienen la misma longitud; en otras podría haber muchas longitudes distintas. La longitud de las instrucciones puede ser menor, igual o mayor que la de una palabra. Exigir que todas las instrucciones tengan la misma longitud es más sencillo y facilita la decodificación, pero a menudo desperdicia espacio, ya que todas las instrucciones tienen que ser tan largas como la más larga. En la figura 5-10 se muestran algunas posibles relaciones entre la longitud de las instrucciones y de las palabras.

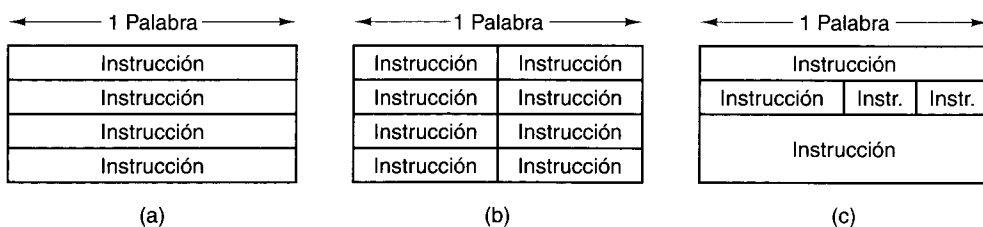


Figura 5-10. Algunas posibles relaciones entre la longitud de las instrucciones y de las palabras.

5.3.1 Criterios de diseño para los formatos de instrucciones

Cuando un equipo de diseño de computadoras tiene que escoger los formatos de instrucciones para su máquina, debe considerar varios factores. No debemos subestimar la dificultad de esta decisión, la cual debe tomarse en una etapa temprana del diseño de una computadora nueva. Si la computadora tiene éxito comercial, el conjunto de instrucciones podría sobrevi-

vir 20 años o más. La capacidad para añadir instrucciones nuevas y aprovechar otras oportunidades que surgen durante un tiempo largo tiene gran importancia, pero sólo si la arquitectura —y la compañía que la construye— sobreviven el tiempo suficiente para que la arquitectura sea un éxito.

La eficiencia de una ISA dada depende en gran medida de la tecnología con que una computadora se va a implementar. Con los años, esta tecnología sufrirá grandes cambios, y en retrospectiva se verá que algunas de las decisiones de la ISA no fueron las mejores. Por ejemplo, si los accesos a la memoria son rápidos, un diseño basado en pila (como JVM) es bueno, pero si son lentos entonces lo mejor es tener muchos registros (como en el UltraSPARC II). Los lectores que creen que esta decisión es fácil podrían tomar una hoja de papel y anotar sus predicciones para (1) una velocidad de reloj de CPU típica y (2) un tiempo de acceso a RAM típico, para las computadoras de 20 años en el futuro. Doble bien la hoja de papel y guárdela 20 años. Luego desdóblela y léala. Quienes carezcan de humildad pueden olvidarse de la hoja de papel y simplemente publicar sus predicciones en Internet ahora.

Desde luego, aun los diseñadores con gran visión pueden tomar decisiones subóptimas. E incluso si pudieran tomar todas las decisiones correctas, también tienen que enfrentar el corto plazo. Si esta ISA elegante es un poco más costosa que sus competidoras actuales mucho menos atractivas, la compañía podría no sobrevivir el tiempo suficiente para que el mundo aprecie la elegancia de la ISA.

Si todo lo demás es igual, las instrucciones cortas son mejores que las largas. Un programa que consiste en n instrucciones de 16 bits ocupa la mitad del espacio de memoria que ocupan n instrucciones de 32 bits. En vista de la tendencia descendente de los precios de las memorias, este factor podría ser menos importante en el futuro, si no fuera por el hecho de que el software crece a un ritmo más rápido que aquél con que bajan los precios de la memoria.

Además, minimizar el tamaño de las instrucciones podría hacerlas más difíciles de decodificar o de traslapar. Por tanto, la reducción del tamaño de las instrucciones al mínimo debe compararse contra el tiempo que se requiere para decodificar y ejecutar las instrucciones.

Otra razón para minimizar la longitud de las instrucciones ya es importante y se vuelve más importante a medida que aumenta la rapidez de los procesadores: el ancho de banda de la memoria (el número de bits/s que la memoria puede proporcionar). El impresionante aumento en la rapidez de los procesadores durante la última década no ha ido acompañado por un aumento comparable en el ancho de banda de las memorias. Una de las restricciones a las que cada vez están más sujetos los procesadores tiene su origen en la incapacidad del sistema de memoria para proporcionar instrucciones y operandos al mismo ritmo que el procesador puede consumirlos. Cada memoria tiene un ancho de banda determinado por su tecnología y su diseño de ingeniería. El cuello de botella del ancho de banda aplica no sólo a la memoria principal, sino también a todas las cachés.

Si el ancho de banda de una caché de instrucciones es de t bps y la longitud media de las instrucciones es de r bits, la caché podrá suministrar cuando más t/r instrucciones por segundo. Cabe señalar que éste es un *límite superior* de la rapidez con que el procesador puede ejecutar instrucciones, aunque se están realizando investigaciones cuya meta es superar esta barrera al parecer infranqueable. Es evidente que la rapidez con que se pueden ejecutar las instrucciones (es decir, la velocidad del procesador) podría estar limitada por la longitud de

las instrucciones. El hecho de que las instrucciones sean más cortas implica un procesador más rápido. Dado que los procesadores modernos pueden ejecutar varias instrucciones en cada ciclo de reloj, es imperativo traer varias instrucciones en cada ciclo de reloj. Este aspecto de la caché de instrucciones hace que el tamaño de las instrucciones sea un criterio de diseño importante.

Un segundo criterio de diseño es suficiente espacio en el formato de las instrucciones para expresar todas las operaciones deseadas. No puede haber una máquina con 2^n operaciones e instrucciones de menos de n bits. Simplemente no habría suficiente espacio en el código de operación para indicar cuál instrucción se necesita. Además, la experiencia ha demostrado una y otra vez la insensatez de no dejar libre un número sustancial de códigos de operación para adiciones futuras al conjunto de instrucciones.

Un tercer criterio tiene que ver con el número de bits de un campo de dirección. Considere el diseño de una máquina con caracteres de 8 bits y una memoria principal que debe contener 2^{32} caracteres. Los diseñadores podrían optar por asignar direcciones consecutivas a unidades de 8, 16, 24 o 32 bits, además de otras posibilidades.

Imagine qué sucedería si el equipo de diseño degenerara en dos facciones opuestas, una que propone que el byte de 8 bits sea la unidad básica de la memoria, y otra partidaria de usar la palabra de 32 bits como unidad básica de memoria. El primer grupo propondría una memoria de 2^{32} bytes, numerados 0, 1, 2, 3, ..., 4,294,967,295. El segundo grupo propondría una memoria de 2^{30} palabras numeradas 0, 1, 2, 3, ..., 1,073,741,823.

El primer grupo señalaría que para comparar dos caracteres en la organización de 32 bits el programa no sólo tendría que obtener las palabras que contienen los caracteres, sino que también tendría que extraer cada carácter de su palabra para poder compararlos. Ello costaría instrucciones adicionales y por tanto desperdiciaría espacio. La organización de 8 bits, en cambio, asigna una dirección a cada carácter, lo que facilita mucho las comparaciones.

Los partidarios de las palabras de 32 bits contraatacarían señalando que su propuesta sólo requiere 2^{30} direcciones distintas, lo que implica direcciones de sólo 30 bits, mientras que la propuesta de bytes de 8 bits requiere 32 bits para direccionar la misma memoria. Direcciones más cortas implican instrucciones más cortas, lo que además de ocupar menos espacio requiere menos tiempo para la obtención. Como alternativa, podrían conservarse las direcciones de 32 bits para hacer referencia a una memoria de 16 GB en lugar de una miserable memoria de 4 GB.

Este ejemplo demuestra que para tener una definición de memoria más fina hay que pagar el precio de direcciones más largas y por tanto instrucciones más largas. Lo máximo en definición es una organización de memoria en la que es posible direccionar cada bit individualmente (como en la Burroughs B1700). En el otro extremo está una memoria que consiste en palabras muy largas (como la serie Cyber de CDC que tenía palabras de 60 bits).

Los sistemas de cómputo modernos han llegado a un término medio que, en cierto sentido, captura lo peor de ambos enfoques. Esta organización requiere todos los bits necesarios para direccionar bytes individuales, pero todos los accesos a la memoria leen una, dos o a veces cuatro palabras a la vez. Por ejemplo, la lectura de un byte de la memoria en el UltraSPARC II trae por lo menos 16 bytes (véase la figura 3-47) y probablemente toda una línea de caché de 64 bytes.

5.3.2 Expansión de códigos de operación

En la sección anterior vimos cómo puede establecerse un equilibrio entre la cortedad de las instrucciones y la definición de la memoria. En esta sección examinaremos equilibrios en los que intervienen tanto los códigos de operación como las direcciones. Considere una instrucción de $(n + k)$ bits con un código de operación de k bits y una sola dirección de n bits. Esta instrucción permite 2^k operaciones distintas y 2^n celdas de memoria direccionables. O bien, los mismos $(n + k)$ bits podrían dividirse en un código de operación de $(k - 1)$ bits y una dirección de $(n + 1)$ bits, lo que implica la mitad de las instrucciones pero el doble de memoria direccionable o bien la misma cantidad de memoria pero con el doble de definición. Un código de operación de $(k + 1)$ bits y direcciones de $(n - 1)$ bits implica más operaciones pero a expensas de reducir el número de celdas direccionables o bien la definición de la memoria. Es posible establecer equilibrios muy complejos entre los bits del código de operación y los de las direcciones, además de los equilibrios sencillos que acabamos de describir. El esquema que analizaremos en los párrafos que siguen se llama **expansión de código de operación**.

La forma más clara de explicar la expansión de códigos de operación es con un ejemplo. Consideremos una máquina en la que las instrucciones tienen 16 bits y las direcciones tienen 4 bits, como se muestra en la figura 5-11. Esta situación podría ser razonable para una máquina que tiene 16 registros (y por tanto direcciones de registro de 4 bits) en los que se efectúan todas las operaciones aritméticas. Un diseño sería un código de operación de 4 bits y tres direcciones en cada instrucción, para dar instrucciones de 16 bits.

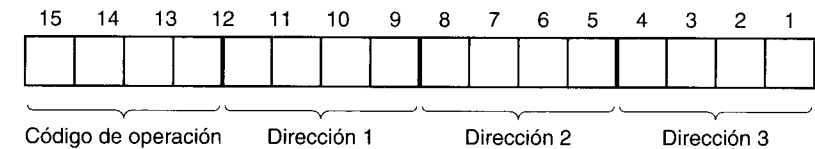


Figura 5-11. Instrucción con código de operación de 4 bits y tres campos de dirección de 4 bits.

Sin embargo, si los diseñadores necesitan 15 instrucciones de tres direcciones, 14 instrucciones de dos direcciones, 31 instrucciones de una dirección y 16 instrucciones que no requieren una dirección, pueden usar los códigos de operación del 0 al 14 como instrucciones de tres direcciones pero interpretar el código de operación 15 de forma distinta (vea la figura 5-12).

El código 15 indica que el código de operación está contenido en los bits 8 a 15, no en los bits 12 a 15. Los bits 0 a 3 y 4 a 7 forman dos direcciones, como siempre. Las 14 instrucciones de dos direcciones tienen 1111 en los 4 bits de la izquierda, y números del 0000 al 1101 en los bits 8 a 11. Las instrucciones que tienen 1111 en los bits de la izquierda y 1110 o bien 1111 en los bits 8 a 11 reciben un trato especial: se tratan como si sus códigos de operación estuvieran en los bits 4 a 15. El resultado es 32 códigos de operación nuevos. Puesto que sólo se necesitan 31, el código de operación 1111111111 se interpreta como señal de que el código de operación real está en los bits 0 a 15, lo que da 16 instrucciones que no usan ninguna dirección.

A medida que avanzaba nuestra explicación, los códigos de operación iban creciendo: las instrucciones de tres direcciones tienen un código de operación de 4 bits; las de dos

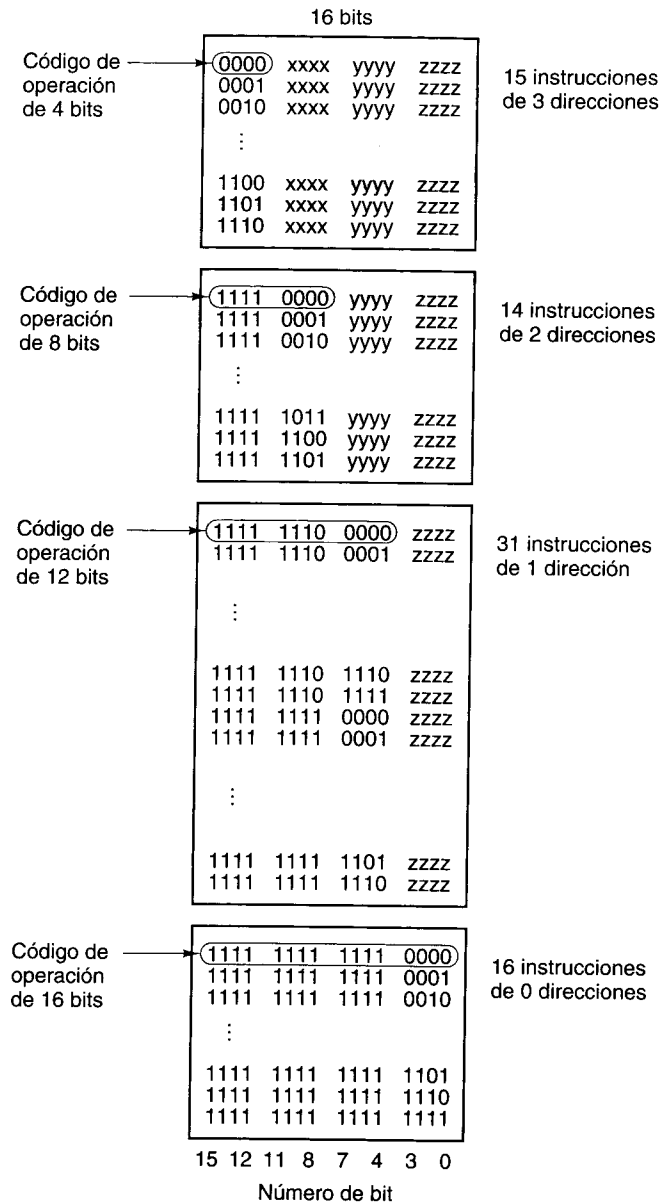


Figura 5-12. Código de operación expansible que permite tener 15 instrucciones de tres direcciones, 14 de dos direcciones, 31 de una dirección y 16 de cero direcciones. Los campos marcados xxxx, yyyy y zzzz son campos de dirección de 4 bits.

direcciones tienen un código de operación de 8 bits, las de una dirección tienen un código de operación de 12 bits, y las de cero instrucciones tienen un código de operación de 16 bits.

La idea de expandir los códigos de operación ilustra un equilibrio entre el espacio para códigos de operación y el espacio para otra información. En la práctica, la expansión de códigos de operación no son tan nítidas y regulares como en nuestro ejemplo. De hecho, la capacidad para usar códigos de operación con tamaño variable se puede aprovechar de una de dos maneras. Primera, puede hacerse que todas las instrucciones tengan la misma longitud, asignando los códigos de operación más cortos a las instrucciones que necesitan más bits para especificar otras cosas. Segunda, se puede minimizar el tamaño *medio* de las instrucciones escogiendo códigos de operación más cortos para las instrucciones comunes y más largos para las instrucciones menos usadas.

Llevando la idea de los códigos de operación de longitud variable al extremo, es posible minimizar la longitud promedio de las instrucciones codificando cada instrucción de modo tal que el número de bits requerido sea el mínimo. El problema es que entonces las instrucciones tendrían diversos tamaños que ni siquiera estarían alineados respecto a las fronteras de bytes. Aunque han existido algunas ISA con esta propiedad (como el malhadado Intel 432), la importancia de la alineación es tan grande para decodificar rápidamente las instrucciones que tal grado de optimización es con toda seguridad contraproducente. No obstante, es común aplicarlo en el nivel de bytes. Más adelante examinaremos la ISA de JVM, por ejemplo, para ver cómo se han escogido cuidadosamente los formatos de las instrucciones a fin de minimizar el tamaño de los programas.

5.3.3 Los formatos de instrucciones del Pentium II

Los formatos de instrucción del Pentium II son muy complejos e irregulares, con hasta seis campos de longitud variable, cinco de los cuales son opcionales. El patrón general se muestra en la figura 5-13. Se llegó a tal patrón porque la arquitectura evolucionó a lo largo de muchas generaciones e incluyó algunas decisiones poco afortunadas en etapas tempranas. En aras de la compatibilidad con modelos anteriores, se respetaron dichas decisiones en procesadores posteriores. En general, para las instrucciones de dos operandos, si un operando está en la memoria, el otro podría no estar en la memoria. Por ello existen instrucciones que suman dos registros, que suman un registro a un valor contenido en la memoria, y que suman un valor de la memoria a un registro, pero no que suman una palabra de la memoria a otra.

En las primeras arquitecturas Intel todos los códigos de operación eran de un byte, aunque se usaba mucho el concepto de byte prefijo para modificar algunas instrucciones. Un **byte prefijo** es un código de operación extra antepuesto a una instrucción para modificar su acción. La instrucción WIDE de IJVM y JVM es un ejemplo de byte prefijo. Lamentablemente, en algún punto durante la evolución, Intel se quedó sin códigos de operación, por lo que uno de ellos, 0xFF, se designó como **código de escape** para poder incluir un segundo byte de instrucción.

Los bits individuales de los códigos de operación del Pentium II no proporcionan mucha información acerca de la instrucción. La única estructura que tiene el campo de código de operación es el uso del bit de orden bajo en algunas instrucciones para indicar byte/palabra, y el uso del bit adyacente para indicar si la dirección de memoria (si está presente) es la fuente

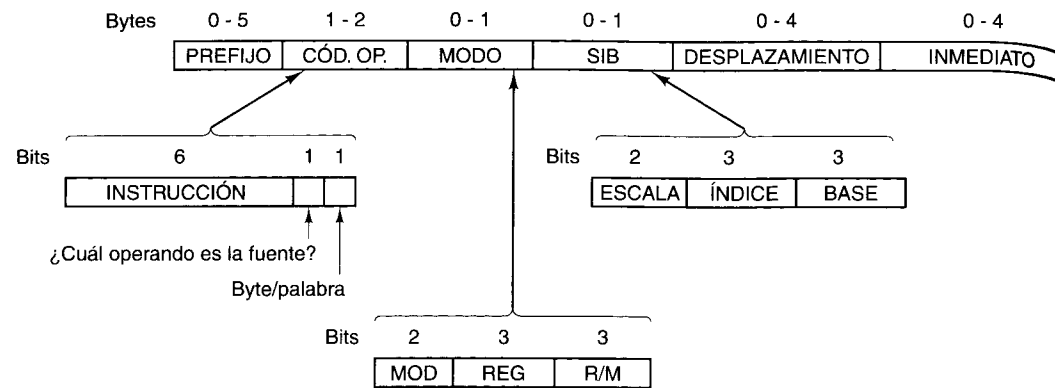


Figura 5-13. Formatos de instrucciones del Pentium II.

o el destino. Así, en general, es preciso decodificar cabalmente el código de operación para determinar qué clase de operación debe efectuarse, y por tanto qué longitud tiene la instrucción. Esto dificulta las implementaciones de alto desempeño, porque se requiere mucho trabajo de decodificación para siquiera determinar dónde comienza la siguiente instrucción.

En la mayor parte de las instrucciones que hacen referencia a un operando en la memoria, el byte de código de operación va seguido de un segundo byte que describe el operando. Estos 8 bits se dividen en un campo **MOD** de dos bits y dos campos de registro de tres bits, **REG** y **R/M**. A veces los tres primeros bits de este byte se usan como extensión del código de operación, lo que da un total de 11 bits para el código de operación. Sin embargo, el campo de modo de dos bits implica que sólo hay cuatro formas de direccionar operandos y uno de los operandos siempre debe ser un registro. Lógicamente, podría especificarse cualquiera de **EAX**, **EBX**, **ECX**, **EDX**, **ESI**, **EDI**, **EBP** o **ESP** como cualquiera de los registros, pero las reglas de codificación prohíben algunas combinaciones, por lo que se destinan a casos especiales. Algunos modos requieren un byte adicional, llamado **SIB** (**escala, índice, base**; *Scale, Index, Base*), que amplía la especificación. Este esquema no es ideal, sino un término medio dadas las demandas opuestas de compatibilidad hacia atrás y el deseo de añadir funciones nuevas que no se contemplaron originalmente.

Además de todo esto, algunas instrucciones tienen 1, 2 o 4 bytes más que especifican una dirección de memoria (desplazamiento) y tal vez otros 1, 2 o 4 bytes que contienen una constante (operando inmediato).

5.3.4 Los formatos de instrucciones del UltraSPARC II

La ISA del UltraSPARC II consiste exclusivamente en instrucciones de 32 bits, alineadas en la memoria. En general, las instrucciones son sencillas y especifican una sola acción. Una instrucción aritmética representativa especifica dos registros que proporcionan los operandos fuente y un solo registro de destino. Una variante permite a la instrucción proporcionar una constante de 13 bits con signo en lugar de uno de los registros. En el caso de un **LOAD**, se suman dos registros (o un registro y una constante de 13 bits) para especificar la dirección de memoria que se leerá. Los datos se colocan en el registro que se especifica.

El SPARC original tenía un número muy limitado de formatos de instrucciones, que se ilustran en la figura 5-14. Con el tiempo, se añadieron nuevos formatos. En el momento de escribirse esta obra, el número era 31 y en aumento. (¿Pasará mucho tiempo antes de que alguna compañía anuncie “la máquina RISC más compleja del mundo”?). Casi todas las nuevas variantes se obtuvieron recortándole unos cuantos bits a algún campo. Por ejemplo, las instrucciones de ramificación originales usaban el formato 3, con una distancia de 22 bits. Cuando se añadieron las ramificaciones predichas, se eliminaron tres de los 22 bits, pues se usaba uno para la predicción (tomar/no tomar la rama) y dos para especificar cuál conjunto de bits de código de condición se debía usar. Esto dejó un desplazamiento de 19 bits. Otro ejemplo es el de las muchas instrucciones que se añadieron para convertir un tipo de datos a otro (entero a punto flotante, etc.). Muchas de éstas usan una variante del formato 1b en la que el campo **INMEDIATO** se dividió en un campo de 5 bits que da el registro fuente y un campo de 8 bits que proporciona más bits para el código de operación. Sin embargo, la generalidad de las instrucciones sigue usando los formatos que se muestran en la figura.

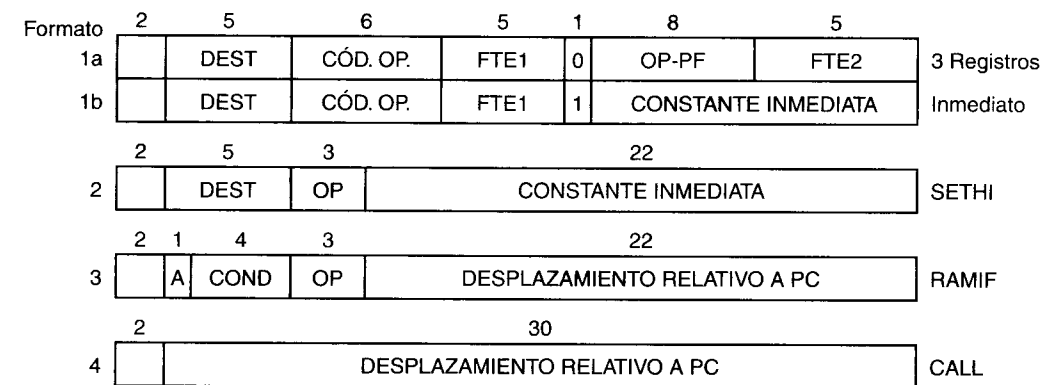


Figura 5-14. Los formatos de instrucción del SPARC original.

Los primeros dos bits de cada instrucción ayudan a determinar el formato de la instrucción y le dicen al hardware dónde puede encontrar el resto del código de operación, si hay más. En el formato 1a, las dos fuentes son registros; en el formato 1b, una fuente es un registro y la otra es una constante dentro del intervalo -4096 a $+4095$. El bit 13 selecciona cuál de las dos se usará. (El bit de la extrema derecha es el bit 0.) En ambos casos, el destino siempre es un registro. Hay suficiente espacio de codificación para un máximo de 64 instrucciones, algunas de las cuales están reservadas para uso futuro.

Con sólo instrucciones de 32 bits, no es posible incluir una constante de 32 bits en la instrucción. La instrucción **SETHI** establece 22 bits, y deja espacio para que otra instrucción establezca los otros 10. **SETHI** es la única instrucción que usa este formato.

Las ramificaciones condicionales sin predicción usan el formato 3, y el campo **COND** indica cuál condición debe probarse. El bit **A** tiene que ver con evitar ranuras de retraso en ciertas condiciones. Las ramificaciones con predicción usan el mismo formato, pero con un desplazamiento de 19 bits, como se mencionó antes.

El último formato es para la instrucción **CALL**, que sirve para invocar un procedimiento. Esta instrucción es especial porque es la única en la que se requieren 30 bits de datos para especificar una dirección. En esta ISA hay un solo código de operación de dos bits. La dirección es la dirección objetivo dividida entre 4, lo que hace que el alcance sea de aproximadamente $\pm 2^{31}$ bytes relativo a la instrucción actual.

5.3.5 Los formatos de instrucciones de la JVM

En general, los formatos de instrucciones JVM son muy sencillos. En la figura 5-15 se muestran todos ellos. Su sencillez probablemente se debe al hecho de que la JVM es nueva. Ya veremos dentro de 10 años. Todas las instrucciones inician con un código de operación de un byte. Algunos códigos van seguidos de un segundo byte que puede ser un índice (como en **ILOAD**), una constante (como en **BIPUSH**) o un indicador de tipo de datos (como en **NEWARRAY**, que crea en el montículo un arreglo unidimensional del tipo indicado). El formato 3 es prácticamente igual al segundo, sólo que usa una constante de 16 bits en lugar de una de 8 bits (por ejemplo, **WIDE ILOAD** o **GOTO**). El formato 4 sólo se usa para **IINC**. El formato 5 sólo se usa para **MULTINEWARRAY**, que crea un arreglo multidimensional en el montículo. El formato 6 sólo se usa con **INVOKEINTERFACE**, que llama a un método en ciertas circunstancias. El formato 7 sólo se usa con **WIDE IINC** para proporcionar un índice de 16 bits y una constante de 16 bits que se suman a la variable seleccionada. El formato 8 sólo se usa con las instrucciones **WIDE GOTO** y **WIDE JSR** que hacen posibles ramificaciones distantes y ciertas llamadas de métodos. El último formato sólo se usa con dos instrucciones que sirven para implementar el enunciado **switch** de Java. En síntesis, con la excepción de ocho instrucciones específicas, todas las instrucciones de la JVM usan los formatos 1, 2 o 3, que son cortos y sencillos.

De hecho, la situación es aún mejor de lo que parece. Las instrucciones de la Máquina Virtual Java se han codificado de tal modo que las instrucciones más comunes se especifican con un solo byte. De las 256 instrucciones posibles dentro de un byte, muchas de ellas son en realidad casos especiales de la forma general de una instrucción que vimos en **IJVM**.

Por ejemplo, veamos cómo se carga una variable local en Java. En realidad hay tres formas distintas de especificar esto: la más corta es la más común y la más larga cubre todos los posibles casos. JVM tiene una instrucción **ILOAD** que usa un índice de 8 bits para especificar la variable local que se debe meter en la pila. También vimos cómo el prefijo **WIDE** permite usar el mismo código de operación para especificar cualquiera de las primeras 65,536 entradas del marco de variables locales. Sin embargo, el uso de **WIDE ILOAD** requiere 4 bytes: uno para **WIDE**, uno para **ILOAD** y dos para el índice de 16 bits. Esta división se justifica por el hecho de que casi todos los **ILOAD** usan una de las primeras 256 variables locales. El prefijo **WIDE** se necesita por generalidad, pero pocas veces se usa.

Pero la JVM va más lejos aún. Puesto que los parámetros que se pasan a un método se pasan en las primeras palabras del marco de variables locales, los usos más comunes de **ILOAD** son las entradas con índices bajos. Los diseñadores de la JVM decidieron que estas posiciones podrían ser tan comunes que valdría la pena asignar códigos de operación de un byte distintos para cada una de estas combinaciones. **ILOAD_0** (0x1A) almacena la variable local 0 en la pila. Esto equivale *exactamente* a la instrucción de dos bytes **ILOAD 0**, excepto

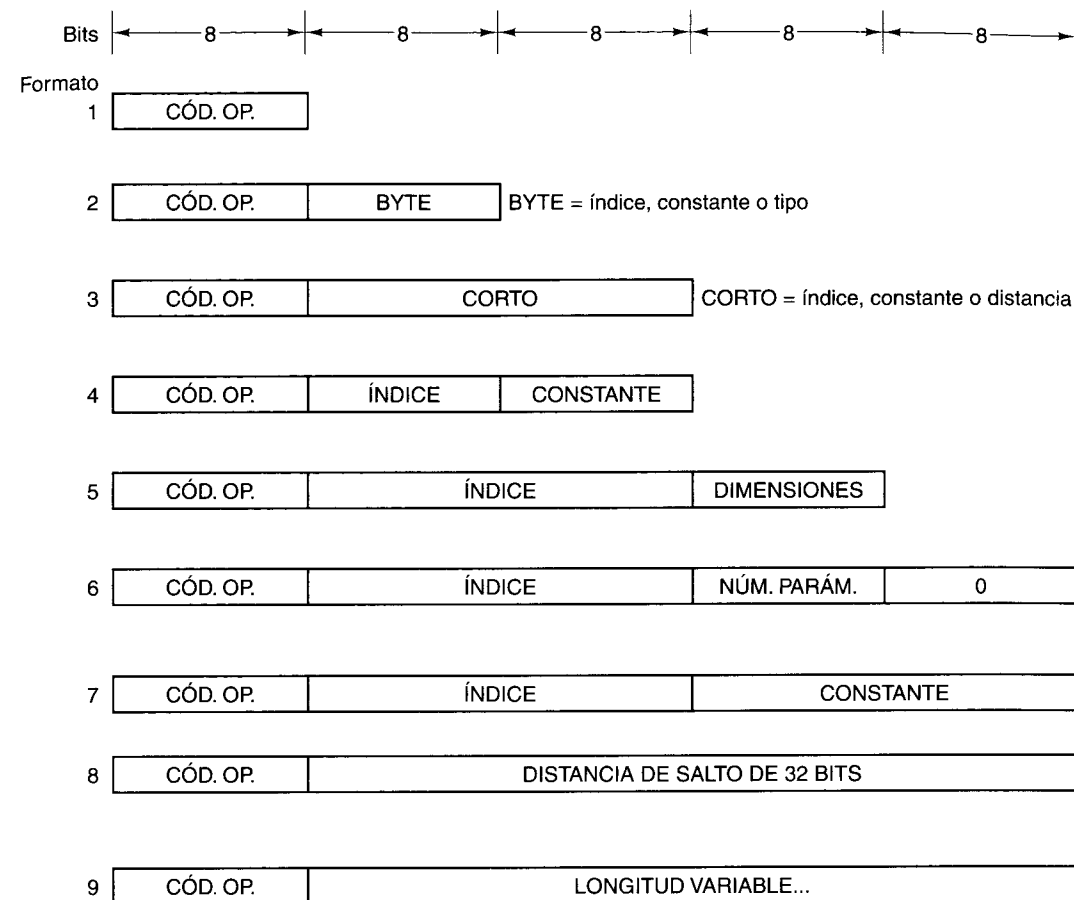


Figura 5-15. Los formatos de instrucción JVM.

que ocupa un byte en lugar de dos. Asimismo, **ILOAD_1**, **ILOAD_2** e **ILOAD_3** (códigos de operación 0x1B, 0x1C y 0x1D, respectivamente) almacenan las variables locales 1, 2 y 3 en la pila. Cabe señalar que la variable local 1, por ejemplo, se puede cargar de tres maneras: **ILOAD_1**, **ILOAD 1** o **WIDE ILOAD 1**.

Existen variaciones similares de muchas instrucciones, como instrucciones especiales que equivalen exactamente a **BIPUSH** con constantes de 0, 1, 2, 3, 4 y 5, así como -1. También hay instrucciones especiales para guardar variables de la pila en las primeras cuatro palabras del espacio de variables locales.

Cabe señalar que estas variaciones no carecen de costo. Sólo pueden especificarse 256 instrucciones únicas con un byte. La decisión de dedicar cuatro de ellas a la carga de las primeras cuatro variables locales consume cuatro de las 256 instrucciones disponibles. Y si añadimos la instrucción **ILOAD** básica vemos que ya se ocuparon cinco instrucciones. Desde luego, el prefijo **WIDE** también consume uno de los 256 valores disponibles (y ni siquiera es una instrucción, sólo un prefijo), pero es más general porque se aplica a muchos otros códigos de operación.

Los diseñadores de la JVM usaron un método un poco diferente para especificar la carga de operandos desde la reserva de constantes, tal vez porque esperaban una diferencia en la distribución de las direcciones especificadas. Incluyeron dos versiones de la instrucción: **LDC** y **LDC_W**. La segunda forma, que se incluyó en la **IJVM**, es la forma general que puede traer cualquiera de las 65,536 palabras de la reserva de constantes. La primera forma requiere un índice de sólo un byte, pero sólo puede traer cualquiera de las primeras 256 palabras. Así, la obtención de cualquiera de las primeras 256 palabras puede efectuarse con una instrucción de dos bytes, mientras que la obtención de cualquier palabra se puede efectuar con una instrucción de tres bytes. Estas opciones consumen dos de los 256 códigos de operación. Cabe señalar que el conjunto de instrucciones habría sido más limpio y regular si los diseñadores hubieran escogido la misma técnica que con **ILOAD**, es decir, el uso del prefijo **WIDE**, en lugar de la instrucción **LDC_W**. Sin embargo, de hacerse así la obtención de constantes más allá de las primeras 256 habría requerido cuatro bytes en lugar de tres.

La técnica de combinar códigos de operación e índices en un solo byte y luego asignar los 256 bytes disponibles según la frecuencia de uso fue propuesta en 1978 por el autor, pero tuvieron que pasar más de dos décadas para que se popularizara (Tanenbaum, 1978).

5.4 DIRECCIONAMIENTO

Un diseño cuidadoso de los códigos de operación es una parte importante de la ISA. Sin embargo, una porción considerable de los bits de un programa se gastan en especificar de dónde provienen los operandos, más que las operaciones que se efectuarán con ellos. Considere una instrucción **ADD** que requiere la especificación de tres operandos: dos fuentes y un destino. (Se acostumbra usar el término *operando* para referirse a los tres, pero el destino es el lugar donde se coloca el resultado.) De alguna manera, la instrucción **ADD** debe indicar dónde están los operandos y dónde debe ponerse el resultado. Si las direcciones de memoria tienen 32 bits, la especificación ingenua de esta instrucción requiere tres direcciones de 32 bits además del código de operación. Las direcciones ocupan muchos más bits que el código de operación.

Se usan dos métodos generales para reducir el tamaño de la especificación. Primero, si un operando va a usarse varias veces, puede colocarse en un registro. El uso de un registro para una variable tiene dos ventajas: el acceso es más rápido y se requieren menos bits para especificar el operando. Si hay 32 registros, cualquiera de ellos puede especificarse con sólo cinco bits. Si la instrucción **ADD** puede efectuarse usando sólo operandos en registros, bastan 15 bits para especificar los tres operandos, en lugar de los 96 que se necesitarían si todos estuvieran en la memoria.

Desde luego, el uso de registros no es gratuito. De hecho, hay ocasiones en que su uso empeora el problema. Si primero hay que cargar en un registro un operando de la memoria, se ocuparán más bits de especificación que si sólo se especificara la posición en memoria. Primero se necesita una instrucción **LOAD** para colocar el operando en un registro. Esto no sólo requiere un código de operación, sino también la dirección de memoria completa y la especi-

ficación del registro objetivo. Por tanto, si un operando sólo se va a usar una vez, no vale la pena colocarlo en un registro.

Por fortuna, años de mediciones muestran que los operandos se reutilizan mucho. Por consiguiente, casi todas las arquitecturas recientes cuentan con un gran número de registros y casi todos los compiladores hacen hasta lo imposible por mantener variables locales en esos registros, lo que elimina muchas referencias a la memoria. Esto reduce tanto el tamaño de los programas como el tiempo de ejecución.

Un segundo método para reducir el tamaño de la especificación consiste en especificar uno o más operandos implícitamente. Hay varias técnicas para hacerlo. Una de ellas es utilizar una sola especificación para un operando fuente y para el destino. Mientras que una instrucción **ADD** general de tres direcciones podría usar la forma

$$\text{DESTINO} = \text{FUENTE1} + \text{FUENTE2}$$

una instrucción de dos registros podría estar limitada a la forma

$$\text{REGISTRO2} = \text{REGISTRO2} + \text{FUENTE1}$$

Esta instrucción es destructiva en el sentido de que el contenido de **REGISTRO2** no se conserva. Si el valor original se va a necesitar posteriormente, primero habrá que copiarlo en otro registro. El equilibrio aquí es que las instrucciones de dos direcciones son más cortas, pero son menos generales. Diferentes diseñadores toman diferentes decisiones. El Pentium II, por ejemplo, usa instrucciones de dos direcciones en el nivel ISA, mientras que el UltraSPARC II usa instrucciones de tres direcciones.

Habiendo reducido el número de operandos de nuestra instrucción **ADD** de tres a dos, no nos detengamos. Las primeras computadoras tenían sólo un registro llamado **acumulador**. La instrucción **ADD**, por ejemplo, siempre sumaba una palabra de la memoria al acumulador, por lo que sólo era necesario especificar un operando (el de la memoria). Esta técnica funcionaba bien con cálculos sencillos, pero si se requerían varios resultados intermedios el acumulador tenía que escribirse en la memoria y más adelante tenía que recuperarse. Es por esto que la técnica se ha dejado de usar.

Ahora hemos pasado de una **ADD** de tres direcciones a una de dos direcciones y luego a una con una sola dirección. ¿Qué falta? ¿Cero direcciones? Sí. En el capítulo 4 vimos cómo **IJVM** usa una pila. La instrucción **IADD** de **IJVM** no tiene direcciones. Tanto la fuente como el destino están implícitos. Veremos el direccionamiento de pila un poco más adelante.

5.4.1 Modos de direccionamiento

Hasta aquí no nos hemos fijado mucho en la forma en que los bits de un campo de dirección se interpretan para encontrar el operando. Una posibilidad es que contienen la dirección en memoria del operando. Además del campo grande que se necesita para especificar la dirección en memoria completa, este método tiene la restricción adicional de que la dirección se debe determinar en el momento de la compilación. Hay otras posibilidades que permiten acortar las especificaciones y también determinar las direcciones dinámicamente.

En las secciones que siguen exploraremos algunas de estas formas, llamadas **modos de direccionamiento**.

5.4.2 Direccionamiento inmediato

La forma más sencilla de especificar un operando es que la parte de dirección de la instrucción realmente contenga el operando mismo en lugar de una dirección u otra información que describa la ubicación del operando. Semejante operando se llama **operando inmediato** porque se obtiene automáticamente de la memoria al mismo tiempo que se obtiene la instrucción misma; por tanto, está disponible inmediatamente para usarse. En la figura 5-16 se muestra una posible instrucción para cargar el registro R1 con la constante 4.

MOV	R1	4
-----	----	---

Figura 5-16. Instrucción inmediata para cargar 4 en el registro 1.

El direccionamiento inmediato tiene la virtud de no requerir una referencia extra a la memoria para obtener el operando. Su desventaja es que con esta técnica sólo se puede suministrar una constante. Además, el número de valores está limitado por el tamaño del campo. No obstante, muchas arquitecturas usan esta técnica para especificar constantes enteras pequeñas.

5.4.3 Direccionamiento directo

Un método para especificar un operando en la memoria es sencillamente dar su dirección completa. Este modo se llama **direccionamiento directo**. Al igual que el direccionamiento inmediato, el uso del direccionamiento directo tiene limitaciones: la instrucción siempre accederá a la misma localidad de memoria exactamente. Si bien el valor contenido en ella puede cambiar, la dirección no puede alterarse. Por ello, el direccionamiento directo sólo sirve para acceder variables globales cuya dirección se conoce en el momento de la compilación. Por otra parte, muchos programas tienen variables globales, por lo que este modo se usa ampliamente. Los detalles de cómo la computadora sabe cuáles direcciones son inmediatas y cuáles son directas se explicarán más adelante.

5.4.4 Direccionamiento por registro

El direccionamiento por registro es igual al directo en lo conceptual pero especifica un registro en lugar de una posición de memoria. Dada la importancia de los registros (por su acceso tan rápido y sus direcciones tan cortas) este modo de direccionamiento es el más común en la mayor parte de las computadoras. Muchos compiladores se esfuerzan por determinar a cuáles variables se accederá con mayor frecuencia (por ejemplo, el índice de un ciclo) y colocan estas variables en registros.

Este modo de direccionamiento se conoce simplemente como **modo de registro**. En las arquitecturas de carga/almacenamiento como el UltraSPARC II, casi todas las instrucciones

emplean este modo de direccionamiento exclusivamente. La única ocasión en que no se usa este modo de direccionamiento es cuando un operando se pasa de la memoria a un registro (instrucción **LOAD**) o de un registro a la memoria (instrucción **STORE**). Incluso en el caso de esas instrucciones, uno de los operandos es un registro: dónde está o dónde se colocará la palabra de memoria.

5.4.5 Direccionamiento indirecto por registro

En este modo, el operando que se especifica viene de la memoria o va a ella, pero su dirección no está fija en la instrucción, como en el direccionamiento directo. En vez de ello, la dirección está contenida en un registro. Cuando una dirección se usa de esta manera, se llama **apuntador**. Una gran ventaja del direccionamiento indirecto por registro es que puede hacer referencias a la memoria sin pagar el precio de tener una dirección de memoria completa en la instrucción. También permite usar diferentes palabras de la memoria en diferentes ejecuciones de la instrucción.

Para ver qué utilidad podría tener usar una palabra diferente en cada ejecución, imagine un ciclo que procesa los elementos de un arreglo unidimensional de enteros con 1024 elementos y calcula la suma de los elementos en el registro R1. Fuera del ciclo, puede hacerse que algún otro registro, digamos R2, apunte al primer elemento del arreglo, y que otro registro, digamos R3, apunte a la primera dirección más allá del arreglo. Con 1024 enteros de 4 bytes cada uno, si el arreglo comienza en A, la primera dirección más allá del arreglo será A + 4096. En la figura 5-17 se muestra un código de ensamblador típico para realizar este cálculo en una máquina de dos direcciones.

MOV R1,#0	; acumular la suma en R1, inicialmente 0
MOV R2,#A	; R2 = dirección del arreglo A
MOV R3,#A+1024	; R3 = dirección de la primera palabra después de A
CICLO: ADD R1,(R2)	; indirecto por registro R2 para obtener operando
ADD R2,#4	; incrementar R2 en una palabra (4bytes)
CMP R2,R3	; ¿ya terminamos?
BLT LOOP	; si R2 < R3, no hemos acabado; continuar

Figura 5-17. Programa genérico en ensamblador para calcular la suma de los elementos de un arreglo.

En este pequeño programa usamos varios modos de direccionamiento. Las primeras tres instrucciones usan el modo de registro para el primer operando (el destino) y el modo inmediato para el segundo operando (una constante indicada por el signo #). La segunda instrucción coloca la *dirección* de A en R2, no su contenido. Eso es lo que el signo # le dice al ensamblador. Asimismo, la tercera instrucción coloca en R3 la dirección de la primera palabra después del arreglo.

Lo interesante es que el cuerpo del ciclo mismo no contiene ninguna dirección de memoria. En la cuarta instrucción se usa el modo de registro y el indirecto por registro; en la quinta se usa el modo de registro y el inmediato, y en la sexta se usa dos veces el modo de registro.

La instrucción de ramificación BLT podría usar una dirección de memoria, pero lo más probable es que especifique la dirección a la que lleva la rama con una distancia de 8 bits relativa a la instrucción BLT misma. Al evitar totalmente las direcciones de memoria hemos producido un ciclo corto y rápido. Por cierto, este programa es realmente para el Pentium II, sólo que cambiamos los nombres de las instrucciones y los registros, así como la notación, para hacerlo más comprensible, ya que la sintaxis del lenguaje ensamblador estándar del Pentium II (MASM) es rarísima, una reliquia de la anterior vida de la máquina como un 8088.

Vale la pena señalar que, en teoría, hay otra forma de realizar este cálculo sin usar direccionamiento indirecto por registro. El ciclo podría haber contenido una instrucción que sumara A y $R1$, como

```
ADD R1,A
```

Luego, en cada iteración del ciclo la instrucción misma podría incrementarse en 4, de modo que después de una iteración diga

```
ADD R1,A+4
```

y así hasta terminar.

Un programa que se modifica a sí mismo de esta manera es un programa **automodificable**. La idea se le ocurrió al mismísimo John von Neumann y era razonable en las primeras computadoras, que no tenían direccionamiento indirecto por registro. Hoy día los programas automodificables se consideran de muy mal gusto y difíciles de entender; además, no pueden compartirse entre varios procesos al mismo tiempo. Otra desventaja es que ni siquiera funcionan correctamente en máquinas con caché de nivel 1 dividida, si la caché L_1 no cuenta con circuitos para efectuar escrituras de vuelta (porque los diseñadores supusieron que los programas no se modifican ellos mismos).

5.4.6 Direccionamiento indexado

En muchos casos es útil poder hacer referencia a palabras de memoria que están a una distancia conocida de un registro. Ya vimos algunos ejemplos con IJVM en los que se hace referencia a variables locales indicando su distancia respecto a LV . Direccionar la memoria dando un registro (explícito o implícito) más una distancia constante se denomina **direccionamiento indexado**.

El acceso a variables locales en IJVM utiliza un apuntador a la memoria (LV) en un registro más una distancia pequeña en la instrucción misma, como se muestra en la figura 4-19(a). Sin embargo, también es posible hacerlo del otro modo: con el apuntador a la memoria en la instrucción y la distancia pequeña en el registro. Para ver cómo funciona esto, considere el cálculo siguiente. Tenemos dos arreglos unidimensionales A y B , de 1024 palabras cada uno, y queremos calcular $A_i \text{ AND } B_i$ para todos los pares i y luego obtener el OR de estos 1024 productos booleanos para ver si hay al menos un par distinto de cero en el conjunto. Una estrategia sería colocar la dirección de A en un registro, la dirección de B en un segundo registro, y luego incrementarlas juntas, de forma análoga a como hicimos en la figura 5-17.

No cabe duda de que esto funcionaría, pero hay una forma mejor de hacerlo, como se muestra en la figura 5-18.

```

MOV R1,#0      ; acumular el OR en R1, inicialmente 0
MOV R2,#0      ; R2 = índice, i, del producto actual: A[i] AND B[i]
MOV R3,#4096   ; R3 = primer valor del índice que no se usará
CICLO MOV R4,A(R2) ; R4 = A[i]
      AND R4,B(R2) ; R4 = A[i] AND B[i]
      OR R1,R4     ; hacer OR de todos los productos booleanos y poner en R1
      ADD R2,#4    ; i = i + 4 (incremento en unidades de 1 palabra = 4 bytes)
      CMP R2,R3    ; ¿ya terminamos?
      BLT LOOP     ; si R2 < R3, no hemos terminado; continuar.

```

Figura 5-18. Programa genérico en ensamblador para calcular el OR de A_i y B_i con dos arreglos de 1024 elementos.

El funcionamiento del programa es sencillo. Necesitamos cuatro registros:

1. $R1$ – Contiene el OR acumulado de los productos booleanos
2. $R2$ – El índice i que se usa para procesar los arreglos
3. $R3$ – La constante 4096, que es el valor más bajo de i que no debemos usar
4. $R4$ – Un registro de borrador para contener cada producto a medida que se forma.

Después de inicializar los registros ingresamos en el ciclo de seis instrucciones. La instrucción que está en *CICLO* trae A_i y lo coloca en $R4$. Aquí, el cálculo de la fuente usa el modo indexado. Un registro, $R2$, y una constante, la dirección de A , se suman y la suma se usa para hacer referencia a la memoria. La suma de estas dos cantidades se envía a la memoria, pero no se guarda en ningún registro visible para el usuario. La notación

```
MOV R4,A(R2)
```

indica que el destino usa modo de registro con $R4$ como el registro y la fuente usa modo indexado, con A como la distancia y $R2$ como el registro. Si A tiene el valor de 124300, por ejemplo, la instrucción de máquina real para esto es probable que se parezca a la de la figura 5-19.

MOV	R4	R2	124300
-----	----	----	--------

Figura 5-19. Una posible representación de $\text{MOV } R4,A(R2)$.

En la primera iteración del ciclo, $R2$ es 0 (porque así se inicializó), de modo que la palabra de memoria direccionada es A_0 , en la dirección 124300. Esta palabra se carga en $R4$. En la siguiente iteración del ciclo $R2$ es 4, así que la palabra de memoria a la que se accede es A_1 , en 124304, etcétera.

Como prometimos antes, aquí la distancia en la instrucción misma es el apuntador a la memoria, y el valor que está en el registro es un entero pequeño que se incrementa durante el cálculo. Desde luego, esta forma requiere un campo de distancia en la instrucción lo bastante grande como para contener una dirección, así que es menos eficiente que la forma anterior; no obstante, en muchos casos es la mejor manera de hacerlo.

5.4.7 Direccionamiento basado indexado

Algunas máquinas tienen un modo de direccionamiento en el que la dirección en memoria se calcula sumando dos registros más una distancia (opcional). Este modo se conoce como **direccionamiento basado indexado**. Uno de los registros es la base y el otro es el índice. Un modo así habría sido útil aquí. Afuera del ciclo podríamos haber colocado la dirección de A en R5 y la dirección de B en R6. Luego podríamos haber sustituido la instrucción que está en CICLO y su sucesora por

CICLO: MOV R4,(R2+R5)
 AND R4,(R2+R6)

Si hubiera un modo de direccionamiento para acceder a la memoria de forma indirecta usando la suma de dos registros sin una distancia, habría sido ideal. Como alternativa, incluso una instrucción con una distancia de 8 bits habría sido mejor que el código original, porque podríamos asignar 0 a ambas distancias. En cambio, si las distancias siempre son de 32 bits no habremos ganado nada al usar este modo. En la práctica, las máquinas que manejan este modo suelen tener una forma con una distancia de 8 bits o de 16 bits.

5.4.8 Direccionamiento de pila

Ya mencionamos que es preferible hacer las instrucciones de máquina lo más pequeñas posible. El límite para reducir la longitud de las direcciones es no tener direcciones. Como vimos en el capítulo 4, son posibles instrucciones con cero direcciones, como IADD, si se usan con una pila. En esta sección examinaremos con mayor detalle el direccionamiento de pila.

Notación polaca inversa

Una tradición antigua en matemáticas es colocar el operador entre los operandos, como en $x + y$, y no después de los operandos, como en $x + y$. La forma en la que el operador está “adentro”, entre los operandos, se llama notación **infija**. La forma en la que el operador está después de los operandos se llama notación **posfija** o **notación polaca inversa**, por el lógico polaco J. Lukasiewicz (1958), quien estudió las propiedades de esta notación.

La notación polaca inversa tiene varias ventajas sobre la infija para expresar fórmulas algebraicas. Primera, cualquier fórmula se puede expresar sin paréntesis. Segunda, es cómoda para evaluar fórmulas en computadoras que tienen pilas. Tercera, los operadores infijos tienen precedencia, lo cual es arbitrario e indeseable. Por ejemplo, sabemos que $a \times$

$b + c$ significa $(a \times b) + c$ y no $a \times (b + c)$ porque se definió arbitrariamente que la multiplicación tiene precedencia sobre la suma. Pero, ¿el desplazamiento a la izquierda tiene precedencia sobre el AND booleano? ¿Quién sabe? La notación **polaca inversa** elimina esta molestia.

Existen varios algoritmos para convertir fórmulas infijas a notación polaca inversa. La que se da a continuación es una adaptación de una idea de E. W. Dijkstra. Suponga que una fórmula se compone de los siguientes símbolos: variables, los operadores díadicos (de dos operandos) $+ - * /$, y los paréntesis izquierdo y derecho. Para marcar el final de una fórmula, insertamos el símbolo $\perp$ después del último símbolo y antes del primero.

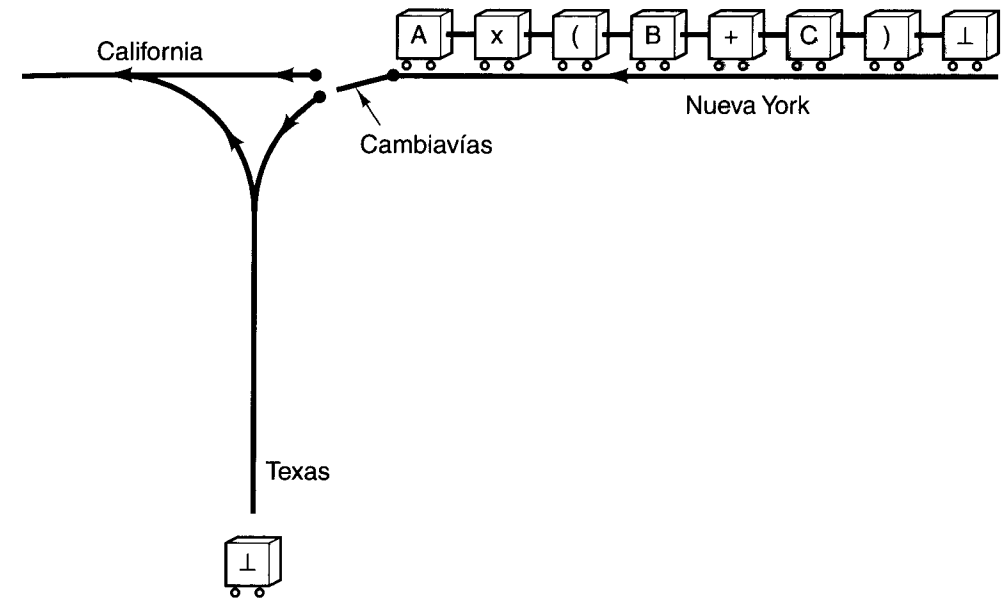


Figura 5-20. Cada vagón de ferrocarril representa un símbolo de la fórmula que se convertirá de notación infija a polaca inversa.

La figura 5-20 muestra una vía de ferrocarril de Nueva York a California, con una desviación a la mitad que lleva a Texas. Cada símbolo de la fórmula se representa con un vagón de ferrocarril. El tren se mueve hacia el poniente (a la izquierda). Cuando un vagón se acerca a un cambiavías, debe detenerse justo antes de llegar y preguntar si debe ir a California directamente o desviarse hacia Texas. Los vagones que contienen variables siempre se van directamente a California sin pasar por Texas. Los vagones que contienen cualquier otro símbolo deben indagar el contenido del vagón más cercano en la vía a Texas antes de ingresar al cambiavías.

La tabla de la figura 5-21 muestra lo que sucede, dependiendo del contenido del vagón más cercano en la vía a Texas y del vagón que está llegando al cambiavías. El primer $\perp$ siempre va a Texas. Los números se refieren a las siguientes situaciones:

- 1. El vagón en el cambiavías se dirige a Texas.
- 2. El vagón más reciente en la línea a Texas se da vuelta y se dirige a California.
- 3. Tanto el vagón que está en el cambiavías como el último que tomó la vía a Texas son secuestrados y desaparecen (es decir, ambos se eliminan).
- 4. Parar. Los símbolos que están en California representan la fórmula en notación polaca inversa leídos de izquierda a derecha.
- 5. Parar. Ocurrió un error. La fórmula original no estaba balanceada correctamente.

		Vagón en el cambiavías					
		⊥	+	-	x	/	()
Último vagón que llegó a la vía a Texas	⊥	4	1	1	1	1	5
	+	2	2	2	1	1	2
	-	2	2	2	1	1	2
	x	2	2	2	2	2	1
	/	2	2	2	2	2	1
	(	5	1	1	1	1	3

Figura 5-21. Tabla de decisión del algoritmo para convertir notación infija a polaca inversa.

Después de realizar cada acción, se hace una nueva comparación entre el vagón que está en el cambiavías, que podría ser el mismo vagón que en la comparación anterior o bien el siguiente vagón, y el vagón que ahora es el último en tomar la vía a Texas. El proceso continúa hasta llegar al paso 4. Observe que la vía a Texas se está usando como pila, en la que la desviación de un vagón a Texas equivale a la operación de meter o apilar (*push*), y hacer que un vagón que ya está en la vía a Texas se dé vuelta y se dirija a California es una operación de sacar o desapilar (*pop*).

Notación infija	Notación polaca inversa
$A + B \times C$	$A B C \times +$
$A \times B + C$	$A B \times C +$
$A \times B + C \times D$	$A B \times C D \times +$
$(A + B) / (C - D)$	$A B + C D - /$
$A \times B / C$	$A B \times C /$
$((A + B) \times C + D) / (E + F = G)$	$A B + C \times D + E F + G + /$

Figura 5-22. Ejemplos de expresiones infijas y sus equivalentes en notación polaca inversa.

El orden de las variables es el mismo en la notación infija y en la polaca inversa, pero el orden de los operadores no siempre es el mismo. En la notación polaca inversa los operadores aparecen en el orden en que se ejecutarán realmente durante la evaluación de la expresión. La figura 5-22 da ejemplos de fórmulas infijas y sus equivalentes en notación polaca inversa.

Evaluación de fórmulas en notación polaca inversa

La notación polaca inversa es ideal para evaluar fórmulas en una computadora que tiene pila. La fórmula consiste en *n* símbolos, cada uno de los cuales es un operando o bien un operador. El algoritmo para evaluar una fórmula en notación polaca inversa empleando una pila es sencillo. Se recorre la cadena de notación polaca inversa de izquierda a derecha. Cuando se encuentra un operando, se mete en la pila. Cuando se encuentra un operador, se ejecuta la instrucción correspondiente.

La figura 5-23 muestra la evaluación de

$(8 + 2 \times 5) / (1 + 3 \times 2 - 4)$

en JVM. La fórmula en notación polaca inversa es

$8\ 2\ 5\ \times\ +\ 1\ 3\ 2\ \times\ +\ 4\ -\ /\$

En la figura introdujimos IMUL e IDIV como instrucciones de multiplicación y división, respectivamente. El número que está en el tope de la pila es el operando derecho, no el izquierdo. Este punto es importante para la división (y la resta) porque el orden de los operandos es significativo (a diferencia de la suma y la multiplicación). En otras palabras, IDIV se definió cuidadosamente de modo que si primero se empila el numerador, luego se empila el denominador y luego se efectúa la operación, se obtiene el resultado correcto. Observe lo fácil que es generar código (I)JVM a partir de notación polaca inversa: basta con recorrer la fórmula y producir una instrucción por símbolo. Si el símbolo es una constante o variable, se produce una instrucción que lo almacene en la pila. Si el símbolo es un operador, se produce una instrucción que ejecute la operación.

5.4.9 Modos de direccionamiento para instrucciones de ramificación

Hasta ahora hemos examinado sólo instrucciones que operan con datos. Las instrucciones de ramificación (y las llamadas a procedimientos) también necesitan modos de direccionamiento para especificar la dirección objetivo. Los modos que ya examinamos generalmente también funcionan para las ramificaciones. El direccionamiento directo es una posibilidad obvia: la dirección objetivo simplemente se incluye completa en la instrucción.

Sin embargo, otros modos de direccionamiento también tienen sentido. El direccionamiento indirecto por registro permite al programa calcular la dirección objetivo, colocarla en un registro, y luego saltar ahí. Este modo es el que más flexibilidad ofrece porque la dirección objetivo se calcula en el momento de la ejecución, pero también es la que crea más oportunidades de incluir errores que son casi imposibles de encontrar.

Otro modo razonable es el indexado, que salta una distancia conocida desde un registro. Sus propiedades son las mismas que la del modo indirecto por registro.

Paso	Cadena restante	Instrucción	Pila
1	$825 \times +132 \times +4 - /$	BIPUSH 8	8
2	$25 \times +132 \times +4 - /$	BIPUSH 2	8, 2
3	$5 \times +132 \times +4 - /$	BIPUSH 5	8, 2, 5
4	$\times +132 \times +4 - /$	IMUL	8, 10
5	$+132 \times +4 - /$	IADD	18
6	$132 \times +4 - /$	BIPUSH 1	18, 1
7	$32 \times +4 - /$	BIPUSH 3	18, 1, 3
8	$2 \times +4 - /$	BIPUSH 2	18, 1, 3, 2
9	$\times +4 - /$	IMUL	18, 1, 6
10	$+4 - /$	IADD	18, 7
11	$4 - /$	BIPUSH 4	18, 7, 4
12	$- /$	ISUB	18, 3
13	$/$	IDIV	6

Figura 5-23. Uso de una pila para evaluar una fórmula en notación polaca inversa.

Otra opción es el direccionamiento relativo al PC. En este modo, la distancia (con signo) que está en la instrucción se suma al contador de programa para obtener la dirección objetivo. De hecho, esto no es más que el modo indexado usando PC como el registro.

5.4.10 Ortogonalidad de códigos de operación y modos de direccionamiento

Desde el punto de vista del software, las instrucciones y el direccionamiento deben tener una estructura regular, con un mínimo de formatos de instrucciones. Una estructura así permite al compilador producir fácilmente buen código. Todos los códigos de operación deben permitir todos los modos de direccionamiento si son lógicos. Además, todos los registros deberán estar disponibles para todos los modos de registro, incluidos el apuntador de marco (FP), el apuntador de pila (SP) y el contador de programa (PC).

Como ejemplo de diseño aseado para una máquina de tres direcciones, considere los formatos de instrucción de 32 bits de la figura 5-24. Se reconocen hasta 256 códigos de operación. En el formato 1, cada instrucción tiene dos registros fuente y un registro de destino. Todas las instrucciones aritméticas y lógicas usan este formato.

El campo de 8 bits no utilizado del final puede servir para diferenciar aún más las instrucciones. Por ejemplo, podría asignarse un solo código de operación para todas las operaciones de punto flotante, y el campo extra distinguiría entre ellas. Además, si el bit 23 está encendido, se usa el formato 2 y el segundo operando deja de ser un registro para convertirse en una constante inmediata de 13 bits con signo. Las instrucciones LOAD y STORE también pueden usar este formato para hacer referencia a la memoria en modo indexado.

Bits	8	1	5	5	5	8
1	CÓD. OP.	0	DEST	FUENTE1	FUENTE2	
2	CÓD. OP.	1	DEST	FUENTE1	DISTANCIA	
3	CÓD. OP.	DISTANCIA				

Figura 5-24. Diseño sencillo para los formatos de instrucción de una máquina de tres direcciones.

Se requieren unas pocas instrucciones adicionales, como ramificaciones condicionales, pero éstas podrían usar fácilmente el formato 3. Por ejemplo, se podría asignar un código de operación a cada ramificación (condicional), llamada a procedimiento, etc., lo que dejaría 24 bits para una distancia relativa al PC. Si suponemos que dicha distancia se cuenta en palabras, el alcance sería de ± 32 MB. Además, podrían reservarse unos cuantos códigos de operación para las LOAD y STORE que necesitan las distancias largas del formato 3. Éstas no tendrían plena generalidad (por ejemplo, sólo se podría cargar o almacenar RO), pero no se usarían con mucha frecuencia.

Consideremos ahora un diseño para una máquina de dos direcciones que puede usar una palabra de memoria como uno de los operandos. El diseño se muestra en la figura 5-25. Una máquina así puede sumar una palabra de memoria a un registro, sumar un registro a una palabra de memoria, sumar un registro a un registro, o sumar una palabra de memoria a una palabra de memoria. Actualmente, los accesos a la memoria son relativamente costosos, por lo que este diseño no goza de popularidad, pero si los adelantos en las tecnologías de caché o de memoria hacen que los accesos a ésta sean baratos en el futuro, éste es un diseño para el cual es muy fácil y eficiente la compilación. La PDP-11 y la VAX fueron máquinas con mucho éxito que dominaron el mundo de las minicomputadoras durante dos décadas empleando diseños similares a éste.

Bits	8	3	5	4	3	5	4
	CÓD. OP.	MODO	REG	DISTANCIA	MODO	REG	DISTANCIA
(Dirección o distancia directa de 32 bits opcional)							
(Dirección o distancia directa de 32 bits opcional)							

Figura 5-25. Diseño sencillo para los formatos de instrucción de una máquina de dos direcciones

En este diseño, tenemos otra vez un código de operación de 8 bits, pero ahora tenemos 12 bits para especificar la fuente y 12 bits para especificar el destino. Para cada operando, 3 bits dan el modo, 5 bits dan el registro y 4 bits dan la distancia. Con tres bits de modo podríamos manejar direccionamiento inmediato, directo, de registro, indirecto por registro, indexado y de pila, y tener espacio para dos modos futuros más. Éste es un diseño limpio y regular para el cual es fácil

compilar y muy flexible, sobre todo si el contador de programa, el apuntador a la pila y el apuntador a las variables locales están incluidos entre los registros generales y se puede acceder a ellos.

El único problema aquí es que con direccionamiento directo necesitamos más bits para la dirección. Lo que hicieron la PDP-11 y la VAX fue añadir una palabra extra a la instrucción, para la dirección de cada operando direccionado directamente. También podríamos usar uno de los dos modos de direccionamiento disponibles para un modo indexado con una distancia de 32 bits después de la instrucción. Así, en el peor caso, como una **ADD** de memoria a memoria en la que ambos operandos se direccionan directamente, o usando la forma indexada larga, la instrucción tendría 96 bits de largo y usaría tres ciclos de bus (uno para la instrucción, dos para datos). Por otra parte, casi todos los diseños RISC requerirían al menos 96 bits, y probablemente más, para sumar una palabra de memoria arbitraria a otra, y usaría al menos cuatro ciclos de bus.

Hay muchas posibles alternativas a la figura 5-25. En este diseño es posible ejecutar el enunciado

$i = j;$

en una instrucción de 32 bits, siempre que i y j se cuenten entre las primeras 16 variables locales. Por otra parte, para las variables después de la 16, tenemos que usar distancias de 32 bits. Una opción sería otro formato con una sola distancia de 8 bits en lugar de dos distancias de 4 bits, más una regla que diga que la fuente o el destino podrían usarla, pero no ambos. Las posibilidades y equilibrios son ilimitados, y los diseñadores de máquinas deben balancear muchos factores para obtener un buen resultado.

5.4.11 Modos de direccionamiento del Pentium II

Los modos de direccionamiento del Pentium II son altamente irregulares y difieren dependiendo de si una instrucción dada está en modo de 16 bits o en modo de 32 bits. En lo que sigue haremos caso omiso del modo de 16 bits; el modo de 32 bits ya es bastante malo. Los modos que se manejan incluyen inmediato, directo, de registro, indirecto por registro, indexado y un modo especial para direccionar elementos de un arreglo. El problema es que no todos los modos aplican a todas las instrucciones y no todos los registros se pueden usar en todos los modos. Esto hace que la tarea del escritor de compiladores sea mucho más difícil y que el código generado sea más deficiente.

El byte **MODO** de la figura 5-13 controla los modos de direccionamiento. Uno de los operandos se especifica con la combinación de los campos **MOD** y **R/M**. El otro siempre es un registro, y está dado por el valor del campo **REG**. Las 32 combinaciones que pueden especificarse con el campo **MOD** de 2 bits y el campo **R/M** de 3 bits se enumeran en la figura 5-26. Por ejemplo, si ambos campos son cero, el operando se lee de la dirección de memoria contenida en el registro **EAX**.

Las columnas 01 y 10 contienen modos en los que un registro se suma a una distancia de 8 o 32 bits que sigue a la instrucción. Si se selecciona una distancia de 8 bits, primero se extiende su signo a 32 bits antes de la suma. Por ejemplo, una instrucción **ADD** con **R/M** = 011, **MOD** = 01 y una distancia de 6 calcula la suma de **EBX** y 6 y lee la palabra de memoria que está en esa dirección para obtener uno de los operandos. **EBX** no se modifica.

La columna **MOD** = 11 permite escoger entre dos registros. Si la instrucción es de palabras, se escoge el primero; si la instrucción es de bytes, se escoge el segundo. Observe que la tabla no es del todo regular. Por ejemplo, no hay forma de direccionar indirectamente a través de **EBP** ni de sumar una distancia a **ESP**.

R/M	MOD			
	00	01	10	11
000	M[EAX]	M[EAX + DIST8]	M[EAX + DIST32]	EAX o AL
001	M[ECX]	M[ECX + DIST8]	M[ECX + DIST32]	ECX o CL
010	M[EDX]	M[EDX + DIST8]	M[EDX + DIST32]	EDX o DL
011	M[EBX]	M[EBX + DIST8]	M[EBX + DIST32]	EBX o BL
100	SIB	SIB con DIST8	SIB con DIST32	ESP o AH
101	Directo	M[EBP + DIST8]	M[EBP + DIST32]	EBP o CH
110	M[ESI]	M[ESI + DIST8]	M[ESI + DIST32]	ESI o DH
111	M[EDI]	M[EDI + DIST8]	M[EDI + DIST32]	EDI o BH

Figura 5-26. Modos de direccionamiento de 32 bits del Pentium II. $M[x]$ es la palabra de memoria que está en x .

En algunos modos un byte adicional, llamado **SIB** (**escala, índice, base**) sigue al byte **MODO** (véase la figura 5-13). El byte **SIB** especifica un factor de escala y dos registros. Cuando está presente un byte **SIB**, la dirección del operando se calcula multiplicando el registro índice por 1, 2, 4 u 8 (dependiendo de **ESCALA**), sumándolo al registro base, y por último tal vez sumándole un desplazamiento de 8 o 32 bits, dependiendo de **MOD**. Casi todos los registros pueden usarse como índice o como base.

Los modos **SIB** son útiles para acceder a elementos de un arreglo. Por ejemplo, considere el enunciado en Java

```
for (i = 0; i < n; i++) a[i] = 0;
```

donde a es un arreglo de enteros de 4 bytes local respecto al procedimiento en curso. Por lo regular, se usa **EBP** para apuntar a la base del marco de pila que contiene las variables locales y arreglos, como se muestra en la figura 5-27. El compilador podría mantener i en **EAX**. Para acceder a $a[i]$, usaría un modo **SIB** en el que la dirección del operando es la suma de $4 \times \mathbf{EAX}$, **EBP** y 8. Esta instrucción podría guardar un valor en $a[i]$ con una sola instrucción.

¿Vale la pena este modo? Es difícil saberlo. No hay duda de que esta instrucción, si se usa correctamente, ahorra unos cuantos ciclos. La frecuencia con que se use depende del compilador y de la aplicación. El problema es que esta instrucción ocupa un área dentro del chip que podría haber tenido otro uso si la instrucción no estuviera presente. Por ejemplo, la caché nivel 1 podría haber sido más grande, o el chip podría haber sido más pequeño, lo que tal vez habría permitido una velocidad de reloj un poco más alta.

Éstos son los tipos de decisiones que los diseñadores enfrentan todo el tiempo. Por lo regular se efectúan simulaciones extensas antes de producir circuitos en silicio, pero esas simu-

laciones requieren tener una buena idea de la magnitud de la carga de trabajo. Podemos asegurar que los diseñadores del 8088 no incluyeron un navegador de Web en su conjunto de prueba. A pesar de ello, no pocos de los descendientes de ese producto se usan ahora primordialmente para explorar la Web, y es posible que las decisiones tomadas hace 20 años sean totalmente inadecuadas para las aplicaciones actuales. No obstante, en aras de la compatibilidad con modelos anteriores, una vez que se incorpora una característica, es imposible deshacerse de ella.

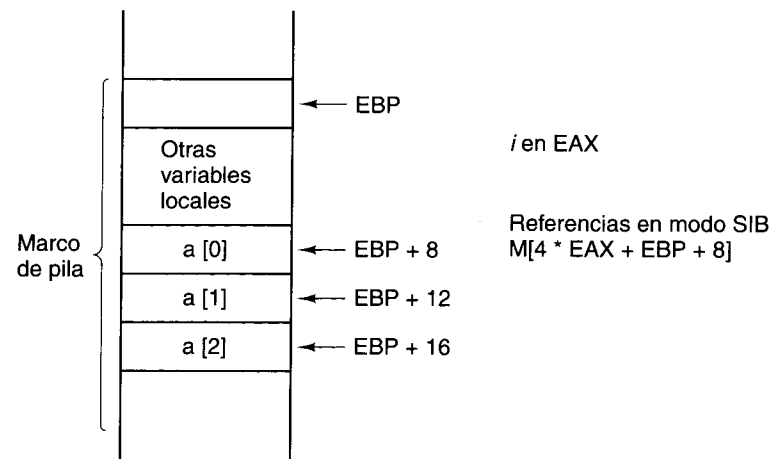


Figura 5-27. Acceso a $a[i]$.

5.4.12 Modos de direccionamiento del UltraSPARC II

En la ISA UltraSPARC todas las instrucciones usan direccionamiento inmediato o de registro con excepción de las que direccionan memoria. En el modo de registro, los 5 bits simplemente indican cuál registro usar. En el modo inmediato, una constante de 13 bits (con signo) proporciona los datos. No hay otros modos para las instrucciones aritméticas, lógicas y similares.

Tres clases de instrucciones direccionan memoria: las **LOAD**, las **STORE** y una instrucción de sincronización de multiprocesadores. Las instrucciones **LOAD** y **STORE** tienen dos modos para direccionar la memoria. El primer modo calcula la suma de dos registros y luego la usa para acceso indirecto. El otro es una indexación tradicional, con una distancia de 13 bits (con signo).

5.4.13 Modos de direccionamiento de la JVM

La JVM no tiene modos de direccionamiento generales en el sentido de que toda instrucción tiene unos cuantos bits que le indican cómo calcular la dirección (como en el Pentium II). En vez de ello, cada instrucción tiene asociado un modo de direccionamiento específico. Puesto que la JVM no tiene registros visibles, los modos de direccionamiento de registro e indirecto por registro ni siquiera son posibles. Unas cuantas instrucciones, como **BIPUSH**, usan direccionamiento inmediato. El único otro modo disponible es el modo indexado, empleado por **ILOAD**, **ISTORE**, **LDC_W** y varias otras instrucciones para especificar una variable

relativa a algún registro implícito, por lo regular **LV** o **CPP**. Las instrucciones de ramificación también usan el modo indexado, con **PC** como registro implícito.

5.4.14 Análisis de los modos de direccionamiento

Ya hemos estudiado una buena cantidad de modos de direccionamiento. Los que usan el Pentium II, el UltraSPARC II y JVM se resumen en la figura 5-28. Sin embargo, como ya explicamos, no todas las instrucciones pueden usar todos los modos.

Modo de direccionamiento	Pentium II	UltraSPARC II	JVM
Inmediato	x	x	x
Directo	x		
Registro	x	x	
Indirecto por registro	x		
Indexado	x	x	x
Basado-indexado		x	
Pila			x

Figura 5-28. Comparación de modos de direccionamiento.

En la práctica, no se necesitan muchos modos de direccionamiento para tener una ISA eficaz. Puesto que casi todo el código que se escribe en este nivel es generado por compiladores, el aspecto más importante de los modos de direccionamiento de una arquitectura es que las opciones sean pocas y claras, y que los costos (en términos de tiempo de ejecución y tamaño del código) puedan calcularse con facilidad. Lo que eso implica generalmente es que una máquina debe adoptar una postura extrema: o bien debe ofrecer todas las opciones posibles, o sólo debe ofrecer una. Cualquier término medio implica que el compilador enfrentará decisiones que tal vez no tenga los suficientes conocimientos o sofisticación para tomar.

Así, las arquitecturas más limpias por lo regular sólo tienen un número muy reducido de modos de direccionamiento, con límites estrictos para su uso. En la práctica, tener los modos inmediato, directo, de registro e indexado es suficiente para casi todas las aplicaciones. Además, todos los registros (incluidos el apuntador a variables locales, el apuntador a la pila y el contador de programa) deberán poder usarse cuando se requiera un registro. Los modos de direccionamiento más complejos tal vez reduzcan el número de instrucciones, pero a expensas de introducir secuencias de operaciones que no se pueden ejecutar fácilmente en paralelo con otras operaciones secuenciales.

Ya terminamos nuestro estudio de los diversos equilibrios que puede haber entre códigos de operación y direcciones, y las distintas formas de direccionamiento. Al estudiar una computadora nueva, es recomendable examinar las instrucciones y los modos de direccionamiento no sólo para ver cuáles están disponibles, sino también para entender por qué se tomaron esas decisiones y qué consecuencias habrían tenido otras decisiones.

5.5 TIPOS DE INSTRUCCIONES

Las instrucciones en el nivel ISA pueden dividirse aproximadamente en una media docena de grupos que son relativamente similares entre las distintas máquinas, aunque podrían diferir en los detalles. Además, toda computadora tiene unas cuantas instrucciones poco comunes, que se añadieron para mantener la compatibilidad con modelos previos o porque el arquitecto tuvo una idea brillante o tal vez porque una dependencia del gobierno le pagó al fabricante para que la incluyera. Trataremos de cubrir brevemente todas las categorías comunes a continuación, sin tratar de ser exhaustivos.

5.5.1 Instrucciones de movimiento de datos

Copiar datos de un lugar a otro es la más fundamental de todas las operaciones. Con copiar queremos decir crear un nuevo objeto con un patrón de bits idéntico al del original. Este uso de la palabra “movimiento” es un tanto diferente de su uso normal en español. Cuando decimos que las 14 toneladas de maíz que estaban en la Bodega 23 se movieron al molino, no queremos implicar que se creó una copia idéntica del maíz en el molino y que las 14 toneladas originales todavía están en la Bodega 23. Cuando decimos que el contenido de la localidad de memoria 2000 se trasladó o pasó a algún registro, siempre queremos decir que se creó en el registro una copia idéntica y que el original todavía está en la localidad 2000. Sería mejor llamar a las instrucciones de movimiento de datos instrucciones de “duplicación de datos”, pero el término “traslado o movimiento de datos” ya está establecido.

Hay dos razones para copiar datos de un lugar a otro. Una es fundamental: la asignación de valores a variables. La asignación

$$A = B$$

se implementa copiando el valor que está en la dirección de memoria *B* en la posición *A* porque el programador dijo que se hiciera. La segunda razón para copiar datos es prepararlos para un acceso y uso eficientes. Como hemos visto, muchas instrucciones sólo pueden acceder a variables si están disponibles en un registro. Puesto que hay dos posibles fuentes para un dato (memoria o registro), y hay dos posibles destinos para un dato (memoria o registro), hay cuatro posibles tipos de copiado. Algunas computadoras tienen cuatro instrucciones, una para cada caso; otras tienen una instrucción para los cuatro casos. Otras más usan **LOAD** para copiar de la memoria a un registro, **STORE** para copiar de un registro a la memoria, **MOVE** para copiar de un registro a otro, y ninguna para copiar de una localidad de memoria a otra.

Las instrucciones de movimiento de datos deben indicar de alguna manera la cantidad de datos que deben copiarse. En algunas ISA existen instrucciones para mover cantidades variables de datos que van desde un bit hasta toda la memoria. En las máquinas con longitud de palabra fija la cantidad que se traslada suele ser exactamente una palabra. Para mover una cantidad mayor o menor se requiere una rutina en software que usa desplazamiento y fusión. Algunas ISA ofrecen capacidades adicionales para copiar menos de una palabra (generalmente en incrementos de bytes) y también varias palabras. Copiar múltiples palabras no es

fácil, sobre todo si el número máximo de palabras es grande, porque una operación así puede tardar mucho, y podría tenerse que interrumpir a la mitad. Algunas máquinas con longitud de palabra variable tienen instrucciones que especifican sólo las direcciones de origen y de destino pero no la cantidad. El movimiento continúa hasta que se encuentra un campo de fin de datos.

5.5.2 Operaciones diádicas

Las operaciones diádicas son las que combinan dos operandos para producir un resultado. Todas las ISA tienen instrucciones para sumar y restar enteros. La multiplicación y división de enteros también son estándar en muchos casos. Suponemos que no es necesario explicar por qué las computadoras están equipadas con instrucciones aritméticas.

Otro grupo de operaciones diádicas incluye las instrucciones booleanas. Aunque existen 16 funciones booleanas de dos variables, casi ninguna máquina, o ninguna, tiene instrucciones para las 16. Por lo regular están presentes **AND**, **OR** y **NOT**; a veces también se incluyen el **OR EXCLUSIVO**, el **NOR** y el **NAND**.

Un uso importante de **AND** es la extracción de bits de una palabra. Consideremos, por ejemplo, una máquina con palabras de 32 bits en la que se almacenan cuatro caracteres de 8 bits en cada palabra. Supongamos que es necesario separar el segundo carácter de los otros tres para imprimirlo; es decir, es necesario crear una palabra que contenga ese carácter en los 8 bits de la derecha (es decir, que esté **justificado a la derecha**), con ceros en los 24 bits de la izquierda.

Para extraer el carácter, se calcula el **AND** de la palabra que lo contiene y una constante, llamada **máscara**. El resultado de esta operación es que los bits no deseados se convierten en ceros —es decir, se enmascaran— como se muestra a continuación.

```
10110111 10111100 11011011 10001011 A
00000000 11111111 00000000 00000000 B (máscara)
00000000 10111100 00000000 00000000 A AND B
```

Después, el resultado se desplazará 16 bits a la derecha para aislar el carácter en el extremo derecho de la palabra.

Un uso importante de **OR** es empaquetar bits en una palabra, donde “empaquetar” es lo contrario de “extraer”. Si queremos modificar los 8 bits de la derecha de una palabra de 32 bits sin alterar los otros 24 bits, primero enmascaramos los 8 bits no deseados y luego introducimos el nuevo carácter con un **OR**, como se muestra a continuación.

```
10110111 10111100 11011011 10001011 A
11111111 11111111 11111111 00000000 B (máscara)
10110111 10111100 11011011 00000000 A AND B
00000000 00000000 00000000 01010111 C
10110111 10111100 11011011 01010111 (A AND B) OR C
```

La operación **AND** tiende a eliminar unos, porque nunca hay más unos en el resultado que en cualquiera de los operandos. La operación **OR** tiende a insertar unos, porque siempre hay al menos tantos unos en el resultado como en el operando que más unos tiene. El **OR**

EXCLUSIVO, en cambio, es simétrico, y en promedio no tiende a insertar ni eliminar unos. Esta simetría respecto a los unos y ceros a veces es útil, por ejemplo al generar números aleatorios.

Casi todas las computadoras actuales también manejan una serie de instrucciones de punto flotante, que corresponden aproximadamente a las operaciones aritméticas con enteros. Casi todas las máquinas ofrecen al menos dos tamaños de números de punto flotante, los más cortos para efectuar operaciones rápidas y los más largos para aquellas ocasiones en que se necesitan muchos dígitos de precisión. Si bien hay muchas variaciones de formatos de punto flotante posibles, se ha adoptado casi universalmente una sola norma: IEEE 754. Los números de punto flotante y la norma se describen en el Apéndice B.

5.5.3 Operaciones monádicas

Las operaciones monádicas tienen un operando y producen un resultado. Dado que se tiene que especificar una dirección menos que en las operaciones diádicas, las instrucciones a veces son más cortas, aunque en muchos casos también hay que especificar otra información.

Las instrucciones para desplazar o someter a rotación el contenido de una palabra o byte son muy útiles y a menudo se incluyen con variaciones. Los desplazamientos son operaciones en las que los bits se recorren hacia la izquierda o la derecha, y los bits que son expulsados por un extremo se pierden. Las rotaciones son desplazamientos en los que los bits que salen por un extremo se retroalimentan en el otro extremo. A continuación ilustramos la diferencia entre un desplazamiento y una rotación.

```
00000000 00000000 00000000 01110011 A
00000000 00000000 00000000 00011100 A desplazado 2 bits a la derecha
11000000 00000000 00000000 00011100 A girado 2 bits a la derecha
```

Son útiles los desplazamientos y rotaciones tanto a la derecha como a la izquierda. Si una palabra de n bits se gira k bits a la izquierda, el resultado es el mismo que si se hubiera girado a la derecha $n - k$ bits.

Los desplazamientos a la derecha a menudo se realizan con extensión del signo. Esto implica que las posiciones que se desocupan en el extremo izquierdo de la palabra se llenan con el bit de signo original, 0 o 1. Es como si el bit de signo se arrastrara a la derecha. Entre otras cosas, esto implica que un número negativo seguirá siendo negativo. A continuación ilustramos esta situación para desplazamientos de 2 bits a la derecha.

```
11111111 11111111 11111111 11110000 A
00111111 11111111 11111111 11111100 A desplazado sin extensión de signo
11111111 11111111 11111111 11111100 A desplazado con extensión de signo
```

Un uso importante del desplazamiento es la multiplicación y división por potencias de 2. Si un entero positivo se desplaza k bits a la izquierda, el resultado, siempre que no haya habido desbordamiento, es el número original multiplicado por 2^k . Si un entero positivo se desplaza a la derecha k bits, el resultado es el número original dividido entre 2^k .

Los desplazamientos pueden servir para agilizar ciertas operaciones aritméticas. Consideremos, por ejemplo, el cálculo de $18 \times n$ para algún entero positivo n . Puesto que $18 \times n =$

$16 \times n + 2 \times n$, podemos obtener $16 \times n$ desplazando 4 bits a la izquierda una copia de n . Podemos obtener $2 \times n$ desplazando n un bit a la izquierda. La suma de estos dos números es $18 \times n$. Realizamos la multiplicación con un movimiento, dos desplazamientos y una suma, lo cual muchas veces es más rápido que una multiplicación. Desde luego, el compilador sólo puede usar este truco cuando un factor es una constante.

El desplazamiento de números negativos, aun con extensión de signo, da resultados muy distintos. Consideremos, por ejemplo, el número -1 en complemento a unos. Si lo desplazamos un bit a la izquierda obtenemos -3 . Otro desplazamiento de un bit a la izquierda produce -7 :

```
11111111 11111111 11111111 11111110 -1 en complemento a unos
11111111 11111111 11111111 11111100 -1 desplazado un bit a la izquierda = -3
11111111 11111111 11111111 11111000 -1 desplazado dos bits a la izquierda = -7
```

El desplazamiento a la izquierda de números negativos en formato de complemento a unos no multiplica por 2. Sin embargo, el desplazamiento a la derecha sí simula la división correctamente.

Consideremos ahora una representación de -1 en formato de complemento a dos. Si la desplazamos 6 bits a la derecha con extensión de signo, obtenemos -1 , lo cual es incorrecto porque la parte entera de $-1/64$ es 0:

```
11111111 11111111 11111111 11111111 -1 en complemento a dos
11111111 11111111 11111111 11111111 -1 desplazado 6 bits a la derecha = -1
```

En general, el desplazamiento a la derecha introduce errores porque trunca hacia abajo (hacia el entero más negativo), lo que es incorrecto en la aritmética de números enteros negativos. En cambio, el desplazamiento a la izquierda sí simula la multiplicación por 2.

Las operaciones de rotación son útiles para empacar y desempacar secuencias de bits de las palabras. Si queremos probar todos los bits de una palabra, girar la palabra de bit en bit en cualquier dirección coloca sucesivamente cada bit en el bit de signo, donde se le puede probar fácilmente, y también restaura la palabra a su valor original una vez que se han probado todos los bits. Las operaciones de rotación son más puras que las de desplazamiento porque no se pierde información: una operación de rotación arbitraria se puede anular con otra operación de rotación.

Ciertas operaciones diádicas ocurren tan a menudo con ciertos operandos que las ISA a veces cuentan con instrucciones monádicas para efectuarlas rápidamente. Poner cero en una palabra de memoria o un registro es muy común al inicializar un cálculo. Desde luego, mover un cero es un caso especial de las instrucciones generales de movimiento de datos. Por eficiencia, es común que se incluya una operación CLR (Clear, borrar), con una sola dirección, la posición que se quiere borrar (es decir, igualar a cero).

Al contar también es común sumar 1 a una palabra. Una forma monádica de la instrucción ADD es la operación INC, que suma 1. La operación NEG es otro ejemplo. Negar X en realidad es calcular $0 - X$, una resta diádica, pero por la frecuencia con que se usa a veces se proporciona una instrucción NEG aparte. Es importante señalar aquí la diferencia entre la operación aritmética NEG y la operación lógica NOT. La operación NEG produce el **inverso aditivo** de un número (el número que sumado al original da 0). La operación NOT simple-

mente invierte todos los bits individuales de la palabra. Las operaciones son muy similares y de hecho, en un sistema que usa la representación de complemento a uno, son idénticas. (En aritmética de complemento a dos la instrucción **NEG** se lleva a cabo invirtiendo primero todos los bits individuales y sumando luego 1.)

Las instrucciones diádicas y monádicas a menudo se agrupan por su uso, más que por el número de operandos que requieren. Un grupo abarca las operaciones aritméticas, incluida la negación. El otro incluye las operaciones lógicas y el desplazamiento, ya que estas categorías casi siempre se usan juntas para obtener datos.

5.5.4 Comparaciones y ramificaciones condicionales

Casi todos los programas requieren la capacidad para probar sus datos y alterar la secuencia de instrucciones a ejecutar con base en el resultado. Un ejemplo sencillo es la función raíz cuadrada, $\sqrt{x}$. Si x es negativo, el procedimiento produce un mensaje de error; si no, calcula la raíz cuadrada. Una función *sqrt* tiene que probar x y luego ramificar, dependiendo de si es negativo o no.

Un método común para hacerlo es proporcionar instrucciones de ramificación condicional que prueban alguna condición y saltan a una dirección de memoria dada si la condición se satisface. A veces un bit de la instrucción indica si la rama debe tomarse si la condición se cumple o si no se cumple, respectivamente. En muchos casos la dirección objetivo no es absoluta, sino relativa a la instrucción actual.

La condición que con más frecuencia se prueba es si un bit dado de la máquina es 0 o no. Si una instrucción prueba el bit de signo de un número y salta a *ETIQUETA* si es 1, los enunciados que comienzan en *ETIQUETA* se ejecutarán si el número es negativo, y los enunciados que siguen a la ramificación condicional se ejecutarán si es 0 o positivo.

Muchas máquinas tienen bits de código de condición que sirven para indicar condiciones específicas. Por ejemplo, podría haber un bit de desbordamiento que se pone en 1 cada vez que una operación aritmética produce un resultado incorrecto. Al probar este bit se verifica si hubo desbordamiento o no en la operación aritmética anterior, y si ocurrió podría tomarse una rama a una rutina de error.

Asimismo, algunos procesadores tienen un bit de acarreo que se enciende cuando hay un acarreo de salida del bit de la extrema izquierda; por ejemplo, cuando se suman dos números negativos. Un acarreo de salida del bit de la extrema izquierda es muy normal y no debe confundirse con un desbordamiento. Se requiere la prueba del bit de acarreo en la aritmética de precisión múltiple (es decir, cuando un entero se representa con dos o más palabras).

La prueba de cero es importante en ciclos y muchas otras cosas. Si todas las instrucciones de ramificación condicional probaran sólo un bit, para probar si una palabra es 0 necesitaríamos una prueba individual de cada bit, para constatar que ninguno es 1. A fin de evitar esta situación, muchas máquinas cuentan con una instrucción que prueba una palabra y toma la rama si es cero. Desde luego, esta solución simplemente pasa la responsabilidad a la microarquitectura. En la práctica, el hardware casi siempre tiene un registro cuyos bits se conectan todos a una compuerta OR, para dar un solo bit que indica si el registro contiene

algún bit 1. El bit Z de la figura 4-1 normalmente se calcularía haciendo OR con todos los bits de salida de la ALU e invirtiendo después el resultado.

También es importante comparar dos palabras o caracteres para ver si son iguales o, si no lo son, cuál es mayor. Un ejemplo de aplicación son los ordenamientos. Para realizar esta prueba se requieren tres direcciones: dos para los datos y una para la dirección a la que debe saltarse si la condición se cumple. Las computadoras cuyo formato de instrucciones permite tres direcciones por instrucción no tienen problemas, pero las otras deben hacer algo para superar esta deficiencia.

Una solución común es proporcionar una instrucción que realice una comparación y establezca uno o más bits de condición para registrar el resultado. Una instrucción subsecuente puede probar los bits de condición y ramificar si los dos valores comparados son iguales, o diferentes, o si el primero fue mayor, etc. Tanto el Pentium II como el UltraSPARC II adoptan este enfoque.

La comparación de dos números tiene algunos aspectos sutiles. Por ejemplo, la comparación no es tan sencilla como la resta. Si se compara un número positivo muy grande con un número negativo muy grande, la resta producirá un desbordamiento, porque el resultado de la resta no puede representarse. No obstante, la instrucción de comparación debe determinar si la prueba especificada se cumple o no, y devolver la respuesta correcta; no hay desbordamiento en las comparaciones.

Otra cuestión sutil relacionada con la comparación de números es decidir si se debe considerar que los números tienen signo o no. Los números binarios de tres bits pueden ordenarse de dos formas. De menor a mayor:

Sin signo	Con signo
000	100 (menor)
001	101
010	110
011	111
100	000
101	001
110	010
111	011 (mayor)

La columna de la izquierda muestra los enteros positivos del 0 al 7 en orden creciente. La columna de la derecha muestra los enteros con signo -4 a +3 en complemento a dos. La respuesta a la pregunta "¿Es 011 mayor que 100?" depende de si los números se consideran con signo o no. Casi todas las ISA cuentan con instrucciones que manejan ambos ordenamientos.

5.5.5 Instrucciones de llamada a procedimientos

Un procedimiento es un grupo de instrucciones que realiza alguna tarea y que puede invocarse (llamarse) desde varios puntos del programa. Es común usar el término **subrutina** en lugar

de procedimiento, sobre todo al referirse a programas en lenguaje ensamblador. En Java, el término que se usa es **método**. Una vez que el procedimiento ha terminado su trabajo, debe regresar al enunciado que sigue a la llamada. Por tanto, la dirección de retorno se debe transmitir al procedimiento o guardarse en algún lugar donde se le pueda encontrar cuando sea el momento de regresar.

La dirección de retorno se puede colocar en cualquiera de tres lugares: la memoria, un registro o la pila. Por mucho, la peor solución es colocarla en una sola localidad fija de memoria. Con esta técnica, si el procedimiento invoca a otro procedimiento, la segunda llamada hará que la dirección de retorno de la primera se pierda.

Una solución un poco mejor es hacer que la instrucción de llamada a procedimiento guarde la dirección de retorno en la primera palabra del procedimiento, y que la primera dirección ejecutable esté en la segunda palabra. Así, el procedimiento puede regresar tomando una rama indirecta a la primera palabra o, si el hardware coloca el código de operación de ramificación en la primera palabra junto con la dirección de retorno, tomando una rama directamente a ella. El procedimiento podría invocar otros procedimientos, porque cada procedimiento tiene espacio para una dirección de retorno. Si el procedimiento se invoca a sí mismo, este esquema falla, porque la primera dirección de retorno será destruida por la segunda llamada. La capacidad de un procedimiento para invocarse a sí mismo, llamada **recursión**, es en extremo importante tanto para los teóricos como para los programadores prácticos. Además, si el procedimiento *A* llama al procedimiento *B*, y *B* llama al procedimiento *C*, y *C* llama a *A* (recursión indirecta o encadenada), el esquema también falla.

Algo todavía mejor es hacer que la instrucción de llamada a procedimiento coloque la dirección de retorno en un registro, dejando la responsabilidad de guardarla en un lugar seguro al procedimiento. Si el procedimiento es recursivo, tendrá que poner la dirección de retorno en un lugar distinto cada vez que se invoque.

Lo mejor que puede hacer la instrucción de llamada a procedimiento con la dirección de retorno es almacenarla en una pila. Una vez que el procedimiento termina, saca la dirección de retorno de la pila y la coloca en el contador de programa. Si se cuenta con esta forma de llamada a procedimiento, la recursión no causa problemas especiales; la dirección de retorno se guarda automáticamente de una forma tal que se evita la destrucción de direcciones de retorno previas. Vimos esta forma de guardar la dirección de retorno en IJVM en la figura 4-12.

5.5.6 Control de ciclos

La necesidad de ejecutar un grupo de instrucciones un número fijo de veces surge con frecuencia y es por ello que algunas máquinas cuentan con instrucciones para facilitar esto. Todos los esquemas usan un contador que se incrementa o decrementa en alguna constante en cada iteración del ciclo. El contador también se prueba una vez en cada iteración. Si cierta condición se cumple, el ciclo termina.

Un método inicializa un contador afuera del ciclo y de inmediato comienza a ejecutar el código del ciclo. La última instrucción del ciclo actualiza el contador y, si la condición de

terminación todavía no se satisface, salta a la primera instrucción del ciclo. De lo contrario, el ciclo termina y se ejecuta la primera instrucción que sigue al ciclo. Esta forma de ciclo se describe como ciclo con prueba al final y se ilustra en C en la figura 5-29(a). (No podríamos usar Java aquí porque no tiene una instrucción goto.)

<pre> i = 1; L1: primer-enunciado; . . . último-enunciado; i = i + 1; if (i < n) goto L1; </pre> (a)	<pre> i = 1; L1: if (i > n) goto L2; primer-enunciado; . . . último-enunciado i = i + 1; goto L1; L2: </pre> (b)
---	--

Figura 5-29. (a) Ciclo con prueba al final. (b) Ciclo con prueba al principio.

Los ciclos con prueba al final tienen la propiedad de que el ciclo siempre se ejecuta al menos una vez, incluso si n es 0 o menor que 0. Considere, por ejemplo, un programa que mantiene los expedientes de personal de una compañía. En cierto punto del programa, se lee información acerca de un empleado en particular. Se lee n , el número de hijos que el empleado tiene, y se ejecuta un ciclo n veces, una por cada hijo, leyendo el nombre del hijo, su sexo y su fecha de nacimiento para que la compañía pueda enviarle un regalo de cumpleaños, lo cual es una de las prestaciones de la compañía. Si el empleado no tiene hijos, n será 0 pero el ciclo se ejecutará una vez y dará resultados erróneos.

La figura 5-29(b) muestra otra forma de realizar la prueba que funciona correctamente incluso cuando n es 0 o menor que 0. Observe que la prueba es diferente en los dos casos, de modo que si una sola instrucción ISA realiza tanto el incremento como la prueba, los diseñadores se verán obligados a escoger un método o el otro.

Considere el código que se debe producir para el enunciado

```
for (i = 0; i < n; i++) {enunciados}
```

Si el compilador no tiene información acerca de n , deberá usar la estrategia de la figura 5-29(b) para manejar correctamente el caso en que $n \leq 0$. En cambio, si puede determinar que $n > 0$, digamos examinando la asignación a n , podría usar el código de la figura 5-29(a), que es mejor. El estándar de FORTRAN solía exigir que todos los ciclos se ejecutaran al menos una vez, a fin de poder generar siempre el código más eficiente de la figura 5-29(a). En 1977 ese defecto se corrigió cuando hasta la comunidad FORTRAN comenzó a darse cuenta de que tener un enunciado de ciclo con semántica extraña que a veces daba la respuesta correcta no era una buena idea, aunque ahorrara una instrucción de ramificación en cada ciclo. C y Java siempre lo han hecho correctamente.

5.5.7 Entrada/Salida

Ningún otro grupo de instrucciones exhibe tanta variedad de una máquina a otra como las instrucciones de E/S. En las computadoras personales actuales se usan tres esquemas de E/S distintos. Éstos son

1. E/S programada con espera activa.
2. E/S controlada por interrupciones.
3. E/S de acceso directo a memoria (DMA).

A continuación examinaremos los tres esquemas por turno.

El método de E/S más sencillo posible es la **E/S programada**, que suele usarse en los microprocesadores de bajo costo, como los sistemas incorporados, o en los sistemas que deben responder rápidamente a cambios externos (sistemas de tiempo real). Estas CPU por lo regular tienen una sola instrucción de entrada y una sola instrucción de salida. Cada una de estas instrucciones selecciona uno de los dispositivos de E/S. Se transfiere un solo carácter entre un registro fijo del procesador y el dispositivo de E/S seleccionado. El procesador debe ejecutar una sucesión explícita de instrucciones para cada uno de los caracteres leídos o escritos.

Como ejemplo sencillo de este método, considere una terminal con cuatro registros de un byte, como se muestra en la figura 5-30. Se usan dos registros para entrada, estado y datos, y dos para salida, estado y datos. Cada uno tiene una dirección única. Si se usa E/S con correspondencia en la memoria, los cuatro registros forman parte del espacio de direcciones de memoria de la computadora y se pueden leer y escribir usando instrucciones ordinarias. Otra posibilidad es contar con instrucciones de E/S especiales, digamos IN y OUT, para leerlos y escribir en ellos. En ambos casos, la E/S se efectúa transfiriendo datos e información de situación entre la CPU y estos registros.

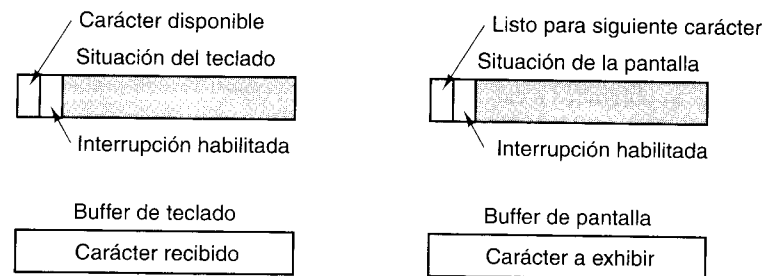


Figura 5-30. Registros de dispositivo para una terminal sencilla.

El registro de estado del teclado tiene dos bits que se usan y seis que no. El hardware pone en 1 el bit de la izquierda (7) cada vez que llega un carácter. Si antes el software encendió el bit 6, se genera una interrupción; si no, no (hablaremos de las interrupciones en breve). Al usar E/S programada para obtener entradas, la CPU normalmente está dando vueltas en un ciclo corto leyendo una y otra vez el registro de estado del teclado, esperando que se encienda

el bit 7. Cuando esto sucede, el software lee el registro buffer del teclado para obtener el carácter. La lectura del registro de datos del teclado hace que el bit **CARÁCTER DISPONIBLE** se ponga en 0.

La salida funciona de forma parecida. Para escribir un carácter en la pantalla, el software primero lee el registro de estado de la pantalla para ver si el bit **READY** vale 1. Si no, da vueltas en un ciclo hasta que el bit se enciende, lo que indica que el dispositivo está listo para aceptar un carácter. Tan pronto como la terminal está lista, el software escribe un carácter en el registro buffer de pantalla, lo que hace que se transmita a la pantalla, y también hace que el dispositivo apague el bit **READY** del registro de situación de la pantalla. Una vez que el carácter se exhibe y la terminal está lista para manejar el siguiente carácter, el controlador enciende automáticamente el bit **READY**.

Como ejemplo de E/S programada, considere el procedimiento Java de la figura 5-31. Este procedimiento se invoca con dos parámetros: un arreglo de caracteres que se enviará a la salida (buf[]) y el número de caracteres presentes en el arreglo (count), hasta 1K. El cuerpo del procedimiento es un ciclo que envía caracteres a la salida uno por uno. Para cada carácter, primero la CPU debe esperar hasta que el dispositivo está listo, y luego se envía el carácter a la salida. Los procedimientos *in* y *out* normalmente serían rutinas en lenguaje ensamblador para leer y escribir los registros de dispositivo especificados por el primer parámetro, de o en la variable especificada como segundo parámetro. La división entre 128 se deshace de los siete bits de orden bajo y deja el bit **READY** en el bit 0.

```
public static void output-buffer(int buf[], int count) {
    // Enviar un bloque de datos al dispositivo
    int status, i, ready;
    for (i = 0; i < count; i++) {
        do {
            status = in(display-status-reg); // obtener situación
            ready = (status << 7) & 0x01;    // aislar bit ready
        } while (ready == 1);
        out(display_buffer_reg, buf[i]);
    }
}
```

Figura 5-31. Ejemplo de E/S programada.

La principal desventaja de la E/S programada es que la CPU pasa casi todo su tiempo en un ciclo corto esperando que el dispositivo esté listo. Este enfoque se llama **espera activa**. Si la CPU no tiene nada más que hacer (por ejemplo, la CPU de una lavadora de ropa), la espera activa podría ser lo mejor (aunque incluso un controlador sencillo a menudo necesita monitorear varios sucesos concurrentes). Pero si hay otras cosas que hacer, como ejecutar otros programas, la espera activa es un desperdicio de recursos y se necesita un método de E/S distinto.

La forma de deshacerse de la espera activa es hacer que la CPU inicie el dispositivo de E/S y le diga que genere una interrupción cuando haya terminado. Mostraremos cómo se

hace esto con referencia a la figura 5-30. Si el software habilita el bit **HABILITAR INTERRUPTIONES** de un registro de dispositivo, el software puede solicitar que el hardware le envíe una señal cuando la E/S haya terminado. Estudiaremos las interrupciones con detalle en una sección posterior del capítulo cuando lleguemos al tema de control de flujo.

Vale la pena mencionar que en muchas computadoras la señal de interrupción se genera obteniendo el AND del bit **HABILITAR INTERRUPTIONES** y el bit **READY**. Si el software habilita las interrupciones primero (antes de iniciar E/S), ocurrirá una interrupción de inmediato, porque el bit **READY** será 1. Por ello, podría ser necesario iniciar primero el dispositivo e inmediatamente después habilitar las interrupciones. La escritura de un byte en el registro de situación no altera el bit **READY**, pues es sólo de lectura.

Aunque la E/S controlada por interrupciones es un gran adelanto en comparación con la E/S programada, le falta mucho para ser perfecta. El problema es que se requiere una interrupción por cada carácter transmitido. El procesamiento de interrupciones es costoso. Necesitamos una forma de deshacernos de la mayor parte de las interrupciones.

La solución está en regresar a la E/S programada, pero encargársela a alguien más. (La solución de muchos problemas consiste en hacer que otra persona efectúe el trabajo.) La figura 5-32 muestra cómo se logra esto. Hemos añadido un nuevo chip al sistema, un controlador de **acceso directo a la memoria (DMA, Direct Memory Access)**, con acceso directo al bus.

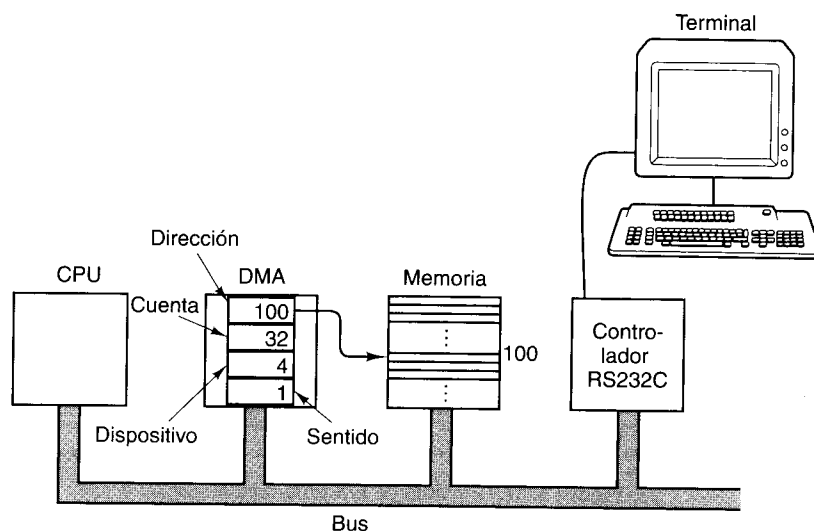


Figura 5-32. Sistema con controlador DMA.

El chip DMA contiene (al menos) cuatro registros, todos los cuales pueden ser cargados por software que se ejecuta en la CPU. El primero contiene la dirección de memoria que se leerá o en la que se escribirá. El segundo contiene el número de bytes (o palabras) que se transferirán. El tercero especifica el número de dispositivo o espacio de direcciones de E/S que se usará, con lo que se especifica qué dispositivo de E/S se desea. El cuarto indica si los datos se escribirán en el dispositivo de E/S o se leerán de él.

Para escribir un bloque de 32 bytes de la dirección de memoria 100 en una terminal (digamos, el dispositivo 4), la CPU escribe los números 100, 32 y 4 en los primeros tres registros de DMA, y luego el código de escribir (digamos, 1) en el cuarto, como se muestra en la figura 5-32. Una vez inicializado con estos valores, el controlador de DMA emite una solicitud de bus para leer el byte 100 de la memoria, del mismo modo como la CPU leería de la memoria. Una vez que obtiene el byte, el controlador de DMA emite una solicitud de E/S al dispositivo 4, para escribir el byte en él. Una vez completadas ambas operaciones, el controlador de DMA incrementa en 1 su registro de dirección y decrementa en 1 su registro de cuenta. Si el registro de cuenta sigue siendo mayor que 0, se lee otro byte de la memoria y se escribe en el dispositivo.

Cuando la cuenta por fin llega a 0, el controlador de DMA deja de transferir data y habilita la línea de interrupción en el chip de CPU. Con DMA, la CPU sólo tiene que inicializar unos cuantos registros. Una vez hecho eso, queda libre para hacer otras cosas hasta que se completa la transferencia, momento en el cual recibe una interrupción del controlador de DMA. Algunos controladores de DMA tienen dos, tres o más conjuntos de registros, lo que les permite controlar varias transferencias simultáneas.

Si bien el DMA alivia a la CPU de gran parte de la carga de E/S, el proceso no es totalmente gratuito. Si un dispositivo de alta velocidad, como un disco, se está controlando por DMA, se requerirán muchos ciclos de bus, tanto para las referencias a la memoria como para las referencias al dispositivo. Durante estos ciclos la CPU tendrá que esperar (el DMA siempre tiene precedencia sobre la CPU para usar el bus porque muchos dispositivos de E/S no pueden tolerar retrasos). El proceso por el que el controlador de DMA quita ciclos de bus a la CPU se denomina **robo de ciclos**. No obstante, la ganancia por no tener que manejar una interrupción por byte (o palabra) transferida rebasa por mucho la pérdida debida al robo de ciclos.

5.5.8 Las instrucciones del Pentium II

En esta sección y las dos que siguen examinaremos los conjuntos de instrucciones de nuestros tres ejemplos de máquinas, el Pentium II, el UltraSPARC II y el picoJava II. Cada una tiene un núcleo de instrucciones que los compiladores normalmente generan, más un grupo de instrucciones que pocas veces se usan, o que sólo el sistema operativo usa. En nuestra explicación nos concentraremos en las instrucciones comunes. Comenzaremos con el Pentium II.

El conjunto de instrucciones del Pentium II es una mezcla de instrucciones que tienen sentido en el modo de 32 bits y otras que se remontan a su anterior vida como un 8088. En la figura 5-33 mostramos algunas de las instrucciones para enteros que con más frecuencia usan los compiladores y programadores en la actualidad. La lista dista mucho de ser completa, pues no incluye instrucciones de punto flotante, instrucciones de control, y ni siquiera algunas de las instrucciones para enteros más exóticas (como usar un byte de 8 bits en AL para consultar una tabla). A pesar de todo, la lista nos da una buena idea de las capacidades del Pentium II.

Muchas de las instrucciones del Pentium II hacen referencia a uno o dos operandos, sea en registros o en la memoria. Por ejemplo, la instrucción **ADD** de dos operandos suma la fuente y el destino, y la instrucción **INC** de un operando incrementa su operando (le suma 1). Algunas de las instrucciones tienen variantes íntimamente relacionadas con ellas. Por ejemplo, las instrucciones de desplazamiento pueden desplazar a la derecha o a la izquierda y

pueden dar al bit de signo un tratamiento especial o no. Casi todas las instrucciones tienen varias codificaciones distintas, dependiendo de la naturaleza de los operandos.

En la figura 5-33, los campos **SRC** (*Source*) son fuentes de información y no se modifican. En contraste, los campos **DST** son destinos y la instrucción normalmente los modifica. Existen reglas acerca de qué está permitido y qué no como fuente o destino, las cuales varían de forma un tanto irregular de una instrucción a otra, pero no entraremos en detalles aquí. Muchas instrucciones tienen tres variantes, para operandos de 8, 16 y 32 bits, respectivamente. Éstas se distinguen por tener diferente código de operación y/o por un bit en la instrucción. En la lista de la figura 5-33 se hace hincapié en las instrucciones de 32 bits.

Por comodidad, hemos dividido las instrucciones en varios grupos. El primer grupo contiene instrucciones que trasladan datos dentro de la máquina, entre registros, la memoria y la pila. El segundo grupo efectúa operaciones aritméticas con y sin signo. Para la multiplicación y división, el producto o dividendo de 64 bits se guarda en **EAX** (parte de orden bajo) y **EDX** (parte de orden alto).

El tercer grupo efectúa operaciones aritméticas en decimal codificado en binario (BCD), tratando cada byte como dos **nibbles** de 4 bits. Cada nibble contiene un dígito decimal (0 a 9). Las combinaciones de bits 1010 a 1111 no se usan. Así, un entero de 16 bits puede contener un número decimal de 0 a 9999. Si bien esta forma de almacenamiento es poco eficiente, elimina la necesidad de convertir entradas decimales a binario y luego otra vez a decimal para enviarlos a la salida. Estas instrucciones sirven para realizar operaciones aritméticas con los números BCD, y se usan mucho en programas en COBOL.

Las instrucciones booleanas y de desplazamiento/rotación manipulan los bits de una palabra o byte de diversas maneras. Se proporcionan varias combinaciones.

Los dos grupos que siguen se ocupan de efectuar pruebas y comparaciones, para luego saltar con base en los resultados. Los resultados de las instrucciones de prueba y comparación se guardan en diversos bits del registro **EFLAGS**. **Jxx** representa un conjunto de instrucciones de saltos condicionales, dependiendo de los resultados de la comparación anterior (es decir, los bits de **EFLAGS**).

El Pentium II cuenta con varias instrucciones para cargar, almacenar, trasladar, comparar y examinar cadenas de caracteres o palabras. Estas instrucciones pueden ir precedidas por un byte especial llamado **REP**, que hace que se repitan hasta que se cumpla cierta condición, como que **ECX**, que se decrementa después de cada iteración, llegue a cero. Esto permite trasladar, comparar, etc., bloques arbitrarios de datos.

El último conjunto de instrucciones no puede clasificarse en ningún otro grupo. Incluyen conversiones, administración de pila, paro de la CPU, y E/S.

El Pentium II tiene varios **prefijos**, de los cuales ya mencionamos uno (**REP**). Cada uno de estos prefijos es un byte especial que puede anteponerse a casi todas las instrucciones, análogo a **WIDE** en **IJVM**. **REP** hace que la instrucción que sigue se repita hasta que **ECX** sea cero, como ya explicamos. **REPZ** y **REPNZ** ejecutan repetidamente la instrucción que sigue hasta que se enciende el código de condición **Z**, o hasta que se apaga, respectivamente. **LOCK** reserva el bus durante toda la instrucción para poder sincronizar multiprocesadores. Otros prefijos sirven para hacer que una instrucción opere en modo de 16 bits o en modo de 32 bits, lo que no sólo altera la longitud de los operandos, sino que también redefine totalmente los modos de direccionamiento. Por último, el Pentium II tiene un esquema de seg-

Traslados	
MOV DST, SRC	Pasar SRC a DST
PUSH SRC	Meter SRC en la pila
POP DST	Sacar una palabra de la pila a DST
XCHG DS1, DS2	Intercambiar DS1 y DS2
LEA DST, SRC	Cargar dir. efectiva de SRC en DST
CMOV DST, SRC	Traslado condicional
Aritmética	
ADD DST, SRC	Sumar SRC a DST
SUB DST, SRC	Restar DST de SRC
MUL SRC	Multiplicar EAX por SRC (sin signo)
IMUL SRC	Multiplicar EAX por SRC (con signo)
DIV SRC	Dividir EDX:EAX entre SRC (sin signo)
IDIV SRC	Dividir EDX:EAX entre SRC (con signo)
ADC DST, SRC	Sumar SRC a DST, luego sumar acarreo
SBB DST, SRC	Restar DST y acarreo a SRC
INC DST	Sumar 1 a DST
DEC DST	Restar 1 a DST
NEG DST	Negar DST (restarlo a 0)
Decimal codificado en binario	
DAA	Ajuste decimal
DAS	Ajuste decimal para restar
AAA	Ajuste ASCII para sumar
AAS	Ajuste ASCII para restar
AAM	Ajuste ASCII para multiplicar
AAD	Ajuste ASCII para dividir
Booleanas	
AND DST, SRC	AND booleano de SRC en DST
OR DST, SRC	OR booleano de SRC en DST
XOR DST, SRC	OR EXCLUSIVO booleano de SRC en DST
NOT DST	Sutituir DST por su complemento a uno
Desplazamiento/rotación	
SAL/SAR DST, #	Desplazar DST izq/der # bits
SHL/SHR DST, #	Desplazam. lógico de DST # bits a izq/der
ROL/ROR DST, #	Girar DST # bits a la izq/der
RCL/RCR DST, #	Girar DST # bits pasando por acarreo
Probar/comparar	
TST SRC1, SRC2	AND booleano de operandos, ajustar banderas
CMP SRC1, SRC2	Ajustar banderas con base en SRC1 - SRC2
Transferencia de control	
JMP ADDR	Saltar a la dirección ADDR
Jxx ADDR	Salto condicional con base en banderas
CALL ADDR	Llamar procedimiento que está en ADDR
RET	Regresar de procedimiento
IRET	Regresar de interrupción
LOOPxx	Ciclo hasta que se cumpla condición
INT ADDR	Iniciar interrupción por software
INTO	Interrumpir si se enciende bit de desbord.
Cadenas	
LODS	Cargar cadena
STOS	Almacenar cadena
MOVS	Trasladar cadena
CMPS	Comparar dos cadenas
SCAS	Examinar cadenas
Códigos de condición	
STC	Encender bit de acarreo en reg. EFLAGS
CLC	Apagar bit de acarreo en reg. EFLAGS
CMC	Complementar bit de acarreo en reg. EFLAGS
STD	Encender bit de sentido en reg. EFLAGS
CLD	Apagar bit de sentido en reg. EFLAGS
STI	Encender bit de interrupción en reg. EFLAGS
CLI	Apagar bit de interrupción en reg. EFLAGS
PUSHFD	Meter registro EFLAGS en la pila
POPFD	Sacar registro EFLAGS de la pila
LAHF	Cargar AH con el registro EFLAGS
SAHF	Guardar AH en el registro EFLAGS
Diversas	
SWAP DST	Cambiar endianditud de DST
CWQ	Extender EAX a EDX:EAX para dividir
CWDE	Extender núm. de 16 bits en AX a EAX
ENTER SIZE, LV	Crear marco de pila con SIZE bytes
LEAVE	Deshacer marco de pila construido con ENTER
NOP	Ninguna operación
HLT	Parar
IN AL, PORT	Aceptar un byte del puerto PORT en AL
OUT PORT, AL	Enviar un byte de AL al puerto PORT
WAIT	Esperar interrupción

SRC = fuente
DST = destino
= cuenta para despl./rot.
LV = núm. variables locales

Figura 5-33. Algunas de las instrucciones del Pentium II que manejan enteros.

mentación complejo con segmentos de código, datos, pila y extra, una reliquia del 8088. Se incluyen prefijos para obligar a las referencias a memoria a usar segmentos específicos, pero (por fortuna) no nos ocuparemos de éstos.

5.5.9 Las instrucciones del UltraSPARC II

Todas las instrucciones de modo de usuario del UltraSPARC II que manejan enteros y que un compilador podría generar se enumeran en la figura 5-34. No se presentan aquí las instrucciones de punto flotante, las de control (por ejemplo, gestión de caché y restablecimiento del sistema), las que hacen referencia a espacios de direcciones distintos del usuario, ni las obsoletas. El conjunto es sorprendentemente pequeño; el UltraSPARC II realmente es una computadora con conjunto de instrucciones reducido.

Las instrucciones **LOAD** y **STORE** son sencillas, con versiones para 1, 2, 4 y 8 bytes. Cuando se carga un número de menos de 64 bits en un registro (de 64 bits), el número puede extenderse con el signo o con cero. Hay instrucciones para ambas opciones.

El siguiente grupo es para aritmética. Las instrucciones que incluyen CC en su nombre ajustan los bits de código de condición NZVC; las demás no lo hacen. En las máquinas CISC, casi todas las instrucciones ajustan los códigos de condición, pero en una máquina RISC ello no es deseable porque restringe la libertad del compilador para cambiar las instrucciones de lugar cuando trata de llenar ranuras de retraso. Si el orden original de las instrucciones es A ... B ... C, y A establece los códigos de condición y B los prueba, el compilador no podrá insertar C entre A y B si C ajusta los códigos de condición. Por esta razón se proporcionan dos versiones de muchas instrucciones; el compilador normalmente usa la que no ajusta los códigos de condición, a menos que planea probarlos después. Hay instrucciones para multiplicación, división con signo y división sin signo.

La aritmética etiquetada es un formato especial con autoidentificación para números de 30 bits. Se puede usar para lenguajes como Smalltalk o Prolog, en los que las variables no tienen un tipo en el momento de la compilación y pueden cambiar de tipo durante la ejecución. Con números etiquetados, el compilador puede generar una instrucción **ADD**, y en el momento de la ejecución la máquina determina si necesita un **ADD** para enteros o uno para números de punto flotante.

El grupo de desplazamiento contiene un desplazamiento a la izquierda y dos a la derecha, cada uno con una versión de 32 bits y una versión extendida de 64 bits. En el caso de **SLL**, se desplazan los 64 bits, ya que esto sigue siendo compatible con el software viejo. Los desplazamientos se usan primordialmente para manipular bits. Casi todas las máquinas CISC tienen un número enorme de instrucciones de desplazamiento y rotación, casi todas ellas absolutamente inútiles. Pocos escritores de compiladores pasarán noches en vela lamentando su ausencia.

El grupo de instrucciones booleanas es análogo al grupo de instrucciones aritméticas: incluye **AND**, **OR**, **OR EXCLUSIVO**, **ANDN**, **ORN** y **XNORN**. Las últimas tres son de valor dudoso, pero pueden ejecutarse en un solo ciclo y casi no requieren hardware adicional, así que se incluyeron. Hasta los diseñadores de máquinas RISC ocasionalmente caen en la tentación.

El siguiente grupo de instrucciones contiene las transferencias de control. **BPcc** representa un conjunto de instrucciones que ramifican según las distintas condiciones, y especifi-

Cargas		Booleanas	
LDSB ADDR,DST	Cargar byte con signo (8 bits)	AND R1,S2,DST	AND booleano
LDUB ADDR,DST	Cargar byte sin signo (8 bits)	ANDCC "	AND booleano y fijar icc
LDSH ADDR,DST	Cargar media palabra con signo (16 bits)	ANDN "	NAND booleano
LDUH ADDR,DST	Cargar media palabra sin signo (16 bits)	ANDNCC "	NAND booleano y fijar icc
LDSW ADDR,DST	Cargar palabra con signo (32 bits)	OR R1,S2,DST	OR booleano
LDUW ADDR,DST	Cargar palabra sin signo (32 bits)	ORCC "	OR booleano y fijar icc
LDX ADDR,DST	Cargar extendido (64 bits)	ORN "	NOR booleano
		ORNCC "	NOR booleano y fijar icc
		XOR R1,S2,DST	XOR booleano
		XORCC "	XOR booleano y fijar icc
		XNOR "	XOR booleano
		XNORCC "	XOR booleano y fijar icc
Almacenamientos		Transferencia de control	
STB SRC,ADDR	Almacenar byte (8 bits)	BPcc ADDR	Ramif. con predicción
STH SRC,ADDR	Almacenar media palabra (16 bits)	BPr SRC,ADDR	Ramif. según registro
STW SRC,ADDR	Almacenar palabra (32 bits)	CALL ADDR	Llamar procedimiento
STX SRC,ADDR	Almacenar extendido (64 bits)	RETURN ADDR	Regresar de procedimiento
Aritmética		JMPL ADDR,DST	Saltar y vincular
ADD R1,S2,DST	Sumar	SAVE R1,S2,DST	Adelantar ventana de registros
ADDCC "	Sumar y fijar icc	RESTORE "	Restaurar ventana de registros
ADDC "	Sumar con acarreo	Tcc CC,TRAP#	Trampa si se cumple condición
ADDCCC "	Sumar con acarreo y fijar icc	PREFETCH FCN	Preobtener datos de memoria
SUB R1,S2,DST	Restar	LDSTUB ADDR,R	Carga/almacenamiento atómico
SUBCC "	Restar y fijar icc	MEMBAR MASK	Barrera de memoria
SUBC "	Restar con acarreo	Diversas	
SUBCCC "	Restar con acarreo y fijar icc	SETHI CON,DST	Establecer bits 10 a 31
MULX R1,S2,DST	Multiplicar	MOVcc CC,S2,DST	Trasladar si se cumple cond.
SDIVX R1,S2,DST	Dividir con signo	MOVr R1,S2,DST	Trasladar según registro
UDIVX R1,S2,DST	Dividir sin signo	NOP	Ninguna operación
TADCC R1,S2,DST	Suma etiquetada	POPC S1,DST	Cuenta de población
Desplazamientos/rotaciones		RDCCR V,DST	Leer registro de cód. de cond.
SLL R1,S2,DST	Despl. lógico a la izq. (64 bits)	WRCCR R1,S2,V	Escribir registro de cód. de cond.
SLLX R1,S2,DST	Despl. lógico a la izq. extendido (64)	RDPC V,DST	Leer contador de programa
SRL R1,S2,DST	Despl. lógico a la der. (32 bits)		
SRLX R1,S2,DST	Despl. lógico a la der. extendido (64)		
SRA R1,S2,DST	Despl. aritmético a la der. (32 bits)		
SRAX R1,S2,DST	Despl. aritmético a la der. ext. (64)		
SRC = registro fuente		TRAP# = Número de trampa	
DST = registro de destino		FCN = código de función	
R1 = registro fuente		MASK = tipo de operación	
S2 = fuente: registro o inmediato		CON = constante	
ADDR = dirección en memoria		V = designador de registro	
		CC = se ajusta cód. de cond.	
		R = registro de destino	
		cc = condición	
		r = LZ,LEZ,Z,NZ,GZ,GEZ	

Figura 5-34. Las instrucciones primarias de UltraSPARC II para enteros.

can en la instrucción si el compilador cree que se tomará la rama o no. BPr prueba un registro y ramifica según la condición detectada.

Se proporcionan dos mecanismos para invocar procedimientos. La instrucción **CALL** usa el formato 4 de la figura 5-14 con una distancia en *palabras* relativa al PC de 32 bits. Este valor basta para alcanzar cualquier instrucción que esté a menos de 2 gigabytes de la que invoca, en cualquier sentido. La instrucción **CALL** deposita la dirección de retorno en R15, que se convierte en R31 después de la llamada.

La otra forma de invocar un procedimiento es usar **JMPL**, que usa el formato 1a o 1b, y permite colocar la dirección de retorno en cualquier registro. Esta forma es útil cuando la dirección objetivo se calcula durante la ejecución.

SAVE y **RESTORE** manipulan la ventana de registros y el apuntador de la pila. Ambas causan una trampa cuando la ventana siguiente (anterior) no está disponible.

El último grupo contiene diversas instrucciones. **SETHI** es necesaria porque no hay forma de colocar un operando inmediato de 32 bits en un registro. La forma como se hace es usar **SETHI** para establecer los bits del 10 al 31 y luego hacer que la siguiente instrucción proporcione los demás bits empleando el formato inmediato.

La instrucción de cuenta de población es un misterio: cuenta el número de bits 1 en una palabra. Hay rumores de que es buena para simular explosiones de bombas, y que Los Alamos National Laboratory (que no escatima en gastos) ve con buenos ojos las máquinas que cuentan con ella. Las últimas tres instrucciones son para leer y escribir registros especiales.

Varias instrucciones CISC conocidas que están ausentes en esta lista se pueden simular usando **G0** o un operando constante (formato 1b). En la figura 5-35 se dan unas cuantas de ellas. El ensamblador de UltraSPARC II las reconoce y los compiladores las generan con frecuencia. Muchas de ellas aprovechan el hecho de que **G0** está conectado permanentemente a 0 y que escribir en él no tiene ningún efecto.

5.5.10 Las instrucciones del picoJava II

Ha llegado el momento de examinar el nivel ISA del picoJava II, que implementa el conjunto de instrucciones JVM completo, con sus 226 instrucciones, además de 115 instrucciones extra cuyo propósito es implementar C, C++ y el sistema operativo. Nos concentraremos primordialmente en las instrucciones JVM, ya que éstas son las únicas que produce el compilador de Java. La ISA JVM no tiene registros visibles para el usuario ni las demás características que la mayor parte de las CPU tienen en común. (El picoJava II tiene 64 registros en el chip para el tope de la pila, pero los programas de usuario no pueden verlos.) Casi todas las instrucciones de JVM meten palabras en la pila, operan con palabras que ya están en la pila, y obtienen palabras de la pila.

El hardware de picoJava II ejecuta directamente la mayor parte de las instrucciones JVM, pero unas cuantas están microprogramadas, y otras pocas saltan a través de una trampa a un manejador en software para su ejecución. Así, se requiere un sistema de tiempo de ejecución pequeño en el nivel ISA para que la máquina funcione, pero este sistema de tiempo de ejecución es mucho más pequeño que el intérprete JVM completo y no se invoca muy a menudo. Dicho sistema contiene código para interpretar unas cuantas instrucciones complejas, el cargador de clases, el verificador de códigos de byte, el gestor de líneas y el recolector de basura.

Instrucción	Cómo hacerlo
MOV SRC,DST	OR SRC con G0 y guardar el resultado en DST
CMP SRC1,SRC2	SUBCC SRC2 de SRC1 y guardar el resultado en G0
TST SRC	ORCC SRC1 con G0 y guardar el resultado en G0
NOT DST	XNOR DST con G0
NEG DST	SUB DST de G0 y guardar en DST
INC DST	ADD 1 a DST (operando inmediato)
DEC DST	SUB 1 de DST (operando inmediato)
CLR DST	OR G0 con G0 y guardar en DST
NOP	SETHI G0 a 0
RET	JMPL %17+8,%G0

Figura 5-35. Algunas instrucciones UltraSPARC II simuladas.

JVM tiene un conjunto de instrucciones relativamente pequeño y sencillo. El conjunto de instrucciones JVM se da en la figura 5-36 con excepción de algunas de las instrucciones anchas, rápidas y de forma corta. Aparte de esas excepciones, la lista está completa.

Las instrucciones JVM tienen tipo, así que hay diferentes instrucciones para efectuar la misma operación con diferentes tipos de datos. Por ejemplo, **ILOAD** mete un entero de 32 bits en la pila, mientras que **ALOAD** mete una referencia (apuntador) de 32 bits en la pila. Esta tipificación estricta no es necesaria para un funcionamiento correcto, ya que el resultado en ambos casos es que los 32 bits que están en la posición de memoria especificada se transfieren a la pila. La tipificación estricta se incluye en JVM para verificar durante la ejecución que un programa binario JVM no viola restricciones de seguridad al tratar de convertir subrepticamente un entero en un apuntador y así acceder a memoria que no se supone deba tocar.

Repasemos ahora las instrucciones JVM. La primera instrucción de la lista es

typeLOAD IND8

En realidad, ésta no es una instrucción, sino una plantilla para generar instrucciones. JVM es regular, así que en lugar de enumerar las instrucciones individualmente, en algunos casos proporcionamos una regla para generarlas. En este caso, **type** representa una de las cuatro letras **I**, **L**, **F** o **D**, que corresponden a los tipos *integer*, *long*, *float* (punto flotante de 32 bits) y *double* (punto flotante de 64 bits), respectivamente. Así, hay cuatro instrucciones implícitas aquí, **ILOAD**, **LLOAD**, **FLOAD** y **DLOAD**, cada una de las cuales usa el índice de 8 bits, **IND8**, para encontrar una variable local, y mete en la pila un valor con la longitud y el tipo correctos. Ya vimos una de éstas (**ILOAD**) en IJVM, pero las otras funcionan de la misma

Cargas		Comparación	
typeLOAD IND8	Apilar variable local	IF_ICMPrel OFFSET16	Ramificación condicional
typeALOAD	Apilar elemento de arreglo	IF_ACMPEQ OFFSET16	Ramif. si 2 apunt. iguales
BALOAD	Apilar byte de un arreglo	IF_ACMPEQ OFFSET16	Ramif. si apunt. distintos
SALOAD	Apilar short de un arreglo	IFrel OFFSET16	Probar valor y ramif.
CALOAD	Apilar char de un arreglo	IFNULL OFFSET16	Ramif. si apunt. nulo
AALOAD	Apilar apuntador de un arreglo	IFNONNULL OFFSET16	Ramif. si apunt. no nulo
Almacenamientos		LCMP	Comparar dos longs
typeSTORE IND8	Sacar valor y guardar en var local	FCMPL	Comparar 2 floats si <
typeASTORE	Sacar valor y guardar en arreglo	FCMPG	Comparar 2 floats si >
BASTORE	Sacar byte y guardar en arreglo	DCMPL	Comparar doubles si <
SASTORE	Sacar short y guardar en arreglo	DCMPG	Comparar doubles si >
CASTORE	Sacar char y guardar en arreglo	Transferencia de control	
AASTORE	Sacar pointer y guardar en arreglo	INVOKEVIRTUAL IND16	Invocación de método
Empilado		INVOKESTATIC IND16	Invocación de método
BIPUSH CON8	Apilar constante pequeña	INVOKEINTERFACE ...	Invocación de método
SIPUSH CON16	Apilar constante de 16 bits	INVOKESPECIAL IND16	Invocación de método
LDC IND8	Apilar constante de res. de ctes.	JSR OFFSET16	Invocar cláusula finally
typeCONST_#	Apilar constante inmediata	typeRETURN	Devolver valor
ACONST_NULL	Apilar apuntador nulo	ARETURN	Devolver apuntador
Aritmética		RETURN	Devolver void
typeADD	Sumar	RET IND8	Regresar de finally
typeSUB	Restar	GOTO OFFSET16	Ramif. incondicional
typeMUL	Multiplicar	Arreglos	
typeDIV	Dividir	ANEWARRAY IND16	Crear arreglo de apunt.
typeREM	Residuo	NEWARRAY ATYPE	Crear arreglo de atype
typeNEG	Negar	MULTINEWARRAY IN16,D	Crear arreglo multidim.
Booleanas/desplazamiento		ARRAYLENGTH	Obtener longitud de arreglo
iIAND	AND booleano	Varias	
iIOR	OR booleano	IINC IND8,CON8	Incrementar variable local
iIXOR	OR EXCLUSIVO booleano	WIDE	Prefijo ancho
iISHL	-Desplazamiento a la izq.	NOP	Ninguna operación
iISHR	Desplazamiento a la der.	GETFIELD IND16	Leer campo de objeto
iIUSHR	Desplazam. a la der. sin signo	PUTFIELD IND16	Escribir campo en objeto
Conversión		GETSTATIC IND16	Obt. campo static de clase
x2y	Convertir x en y	NEW IND16	Crear objeto nuevo
i2c	Convertir integer en char	INSTANCEOF OFFSET16	Determinar tipo de objeto
i2b	Convertir integer en byte	CHECKCAST IND16	Verif. tipo de objeto
Gestión de pila		ATHROW	Lanzar excepción
DUPxx	Seis instrucc. para duplicar	LOOKUPSWITCH ...	Ramif. multivías rala
POP	Desapilar integer y desechar	TABLESWITCH ...	Ramif. multivías densa
POP2	Desapilar dos integer y desechar	MONITORENTER	Ingresar en monitor
SWAP	Intercambiar dos int. del tope	MONITOREXIT	Salir de monitor

Figura 5-36. El conjunto de instrucciones JVM.

manera; la única diferencia radica en el número de palabras que se apilan y en el tipo del valor.

Además de estas cuatro instrucciones de carga, hay otras cuatro implícitas en **typeALOAD**, todas las cuales almacenan elementos de un arreglo en la pila. En los cuatro casos, la pila debe precargarse con un apuntador al arreglo y con el índice del elemento. Estas instrucciones desapilan el índice y el apuntador, realizan el cálculo necesario para localizar el elemento, y luego meten el elemento en la pila. El cálculo necesita conocer el tamaño de los elementos, que está implícito en el tipo. Estas instrucciones obtienen el índice del arreglo de la pila, por lo que la instrucción misma no tiene operando.

Las últimas cuatro instrucciones de este grupo también son para elementos de arreglo, pero de otros tipos: manejan byte (8 bits), short (16 bits), char (16 bits) y pointer (32 bits). Así, hay un total de 12 instrucciones **LOAD** en este grupo.

Las instrucciones **typeSTORE** funcionan igual que las instrucciones **typeLOAD**, pero en el otro sentido. Todas obtienen un elemento de la pila y lo guardan en una variable local. Una vez más, ya vimos una de éstas (**ISTORE**) en **IJVM**. En este caso también tenemos instrucciones para guardar elementos de un arreglo. En el tope de la pila hay un valor del tipo correcto. Debajo de él está el índice, y debajo de éste, el apuntador al arreglo. La operación desapila las tres cosas.

Las instrucciones de empujado meten valores en la pila. Ya vimos **BIPUSH**. **SIPUSH** hace lo mismo, sólo que con un número de 16 bits. **LDC** almacena en la pila un valor de la reserva de constantes. La siguiente instrucción representa todo un grupo de instrucciones inmediatas, de los cuatro tipos básicos (*integer*, *long*, *float* y *double*). Todas consisten únicamente en un código de operación y cada código almacena un valor específico en la pila. Por ejemplo, **ICONST_0** (código 0x03) almacena una palabra 0 de 32 bits en la pila. Desde luego, puede hacerse la misma cosa con **BIPUSH**, pero usando dos bytes. Al optimar los casos más comunes, se reduce el tamaño de los programas JVM. Los valores que se manejan son los enteros -1, 0, 1, 2, 3, 4, 5, los longs 0 y 1, los floats 0.0, 1.0 y 2.0, y los doubles 0.0 y 1.0. De forma similar, **CONST_NULL** almacena un apuntador nulo en la pila. La combinación de códigos de operación con las direcciones más comunes en una instrucción de un solo byte reduce considerablemente el tamaño del programa, ahorra memoria y acorta el tiempo que toma transportar programas binarios en Java por Internet.

Las operaciones aritméticas son totalmente regulares, con seis operaciones para cada uno de los cuatro tipos básicos. Las tres operaciones booleanas y las tres de desplazamiento sólo están disponibles para integers y longs. Tener una instrucción para efectuar un AND bit por bit con números de punto flotante habría sido contrario a las estrictas reglas de tipificación de la JVM. La entrada **x2y** de la tabla representa una matriz 4 × 4 de instrucciones de conversión. Valores de cada tipo se pueden convertir a cada uno de los otros tipos (pero no al mismo tipo), así que hay 12 instrucciones aquí. Una instrucción representativa es **i2f**, que convierte un integer en un float.

El grupo de gestión de pila tiene instrucciones para duplicar el valor o los dos valores del tope, colocarlos en diversos lugares de la pila (no siempre en el tope). Otras instrucciones sacan valores de la pila, e intercambian los dos valores del tope.

Las instrucciones del grupo de comparación sacan uno o dos valores de la pila y los examinan. Cuando se sacan dos valores de la pila, se restan y se prueba el resultado. Cuando se saca un solo valor, se prueba su valor. El sufijo **rel** representa los operadores relacionales **LT**, **LE**, **EQ**, **NE**, **GE** y **GT**. Las instrucciones que tienen una distancia toman la rama si la prueba se satisface. Las demás meten un resultado de vuelta en la pila.

El siguiente grupo permite invocar procedimientos y devolver valores. Vimos versiones muy simplificadas de **INVOKEVIRTUAL** e **IRETURN** en **IJVM**. Las versiones completas tienen muchas más opciones, y hay varias instrucciones para cubrir muchos casos. Una explicación completa rebasaría el alcance de este libro. Si quiere conocer toda la historia, vea Lindholm y Yellin (1997).

El siguiente grupo contiene cuatro instrucciones para crear arreglos unidimensionales y multidimensionales y preguntar por sus longitudes. Los arreglos se guardan en el montículo en **JVM** y se recolectan dinámicamente como basura cuando ya no se necesitan.

El último grupo es una colección miscelánea, cada una con un propósito específico relacionado con alguna característica del lenguaje Java. Su explicación rebasa el alcance de este libro.

Hablemos ahora un poco acerca del nivel ISA del picoJava II. Se trata de una máquina big endian (aunque hay unas cuantas instrucciones que se puede hacer que funcionen como little endian). Las palabras son de 32 bits, aunque hay instrucciones para manejar cantidades de 8, 16 y 64 bits. La pila crece hacia abajo en la memoria, de direcciones altas a direcciones bajas, al contrario que en **IJVM** (la especificación de **JVM** permite las dos cosas).

El picoJava II se diseñó para ejecutar programas en C y C++ además de programas en Java. Sin embargo, para hacerlo se requiere un compilador que convierta C o C++ a la ISA del picoJava II. Una vez compilado a **JVM** un programa en C o C++, todas las optimaciones de hardware que describimos en el capítulo 4 para Java (como el plegado) también aplican a C o C++.

Para poder ejecutar programas en C y C++ en el picoJava II, se añadieron 115 instrucciones al nivel ISA. Casi todas tienen dos o más bytes de longitud y comienzan con uno de dos códigos de operación **JVM** reservados (0xFE y 0xFF) para indicar que sigue una instrucción extendida. A continuación presentaremos un breve resumen de las características ajenas a la **JVM** del picoJava II en el nivel ISA.

El picoJava II tiene 25 registros de 32 bits. Cuatro de ellos son funcionalmente equivalentes a los registros **PC**, **LV**, **SP** y **CPP** de **IJVM**. El registro **OPLIM** establece un límite para **SP**. Si **SP** rebasa **OPLIM**, ocurre una trampa. Esta característica sirve para poder representar la pila como una lista enlazada de trozos de pila, en lugar de un bloque contiguo de la memoria. El registro **FRAME** marca el final de las variables locales y apunta a la palabra que contiene el contador de programa del procedimiento invocador.

Hay otros registros, como la palabra de situación del programa, un registro que se mantiene al tanto de hasta dónde se ha llenado la caché de pila de 64 registros, y cuatro registros que sirven para gestión de hilos. También hay cuatro registros para trampas y puntos de ruptura, y cuatro registros que los programas en C y C++ usan para invocar procedimientos y devolver valores. Puesto que el picoJava II no tiene memoria virtual, dos registros acotan la parte de la memoria a la que el programa vigente puede acceder, a fin de limitar los daños que

podría causar un programa en C o C++ desbocado. Otros registros contienen el número de versión e información sobre la configuración del hardware.

Las instrucciones extendidas pertenecen a cinco categorías amplias. Primero están las instrucciones para leer los registros anteriores y escribir en ellos. Luego vienen las instrucciones para manipular apuntadores. Éstas permiten leer o escribir palabras arbitrarias en la memoria. Casi todas sacan una dirección de máquina de la pila y luego almacenan en la pila el byte, short, palabra, etc., que está en esa dirección. Tales instrucciones violan descaradamente la seguridad de tipos de Java, pero se necesitan para C y C++.

En tercer lugar están las instrucciones útiles para programas en C y C++, como las que invocan procedimientos y regresan de ellos sin usar las instrucciones **JVM** (de trabajo pesado). En cuarto lugar están las instrucciones que manejan el hardware, como por ejemplo la caché. Luego vienen varias instrucciones de diversa índole, como para diagnóstico al encender la máquina. Claro que los programas que usan estas instrucciones extra no se pueden transportar a otras plataformas **JVM**.

5.5.11 Comparación de conjuntos de instrucciones

Los tres conjuntos de instrucciones de ejemplo son muy distintos. El Pentium II es una máquina CISC clásica de dos direcciones y 32 bits, con una larga historia, modos de direccionamiento peculiares e irregulares, y muchas instrucciones que hacen referencia a la memoria. El UltraSPARC II es una máquina RISC moderna de tres direcciones y 64 bits, con una arquitectura de carga/almacenamiento, pocos modos de direccionamiento y un conjunto de instrucciones compacto y eficiente. La arquitectura **JVM** es una máquina de pila casi sin modos de direccionamiento, un conjunto de instrucciones altamente regular, y codificación de instrucciones muy densa.

Cada máquina es como es por una buena razón. El diseño del Pentium II estuvo determinado por tres factores principales:

1. Compatibilidad con modelos anteriores.
2. Compatibilidad con modelos anteriores.
3. Compatibilidad con modelos anteriores.

Dados los modernos avances tecnológicos, nadie diseñaría ahora una máquina tan irregular con tan pocos registros, todos distintos. Esto dificulta la escritura de compiladores. La falta de registros también obliga a los compiladores a verter constantemente variables en la memoria y luego volverlas a cargar: actividades muy costosas aun con dos o tres niveles de caché. Es un tributo a la calidad de los ingenieros de Intel que el Pentium II sea tan rápido, aun con las restricciones de esta ISA. Sin embargo, como vimos en el capítulo 4, la implementación es en extremo compleja y requiere dos veces más transistores que el picoJava II y casi 50% más que el UltraSPARC II.

El UltraSPARC II representa lo más moderno en diseños de ISA: cuenta con una ISA cabal de 64 bits (con un bus de 128 bits); tiene muchos registros, y un conjunto de instruccio-

nes que hace hincapié en las operaciones de tres registros, además de un pequeño grupo de instrucciones **LOAD** y **STORE**. Todas las instrucciones tienen el mismo tamaño, aunque el número de formatos ha crecido demasiado. No obstante, el conjunto de instrucciones se presta a una implementación directa y eficiente. Casi todos los nuevos diseños tienden a parecerse al UltraSPARC II, pero con menos formatos de instrucciones.

La JVM es harina de otro costal. La ISA se diseñó originalmente de modo que las applets (programas pequeños) se pudieran transmitir por Internet e interpretarse en software de forma eficiente en el destino. También es un diseño para un solo lenguaje. Esto llevó al uso de una pila e instrucciones cortas pero de longitud variable con una densidad elevada (un promedio de sólo 1.8 bytes por instrucción). Construir un dispositivo de hardware que realmente ejecutara una instrucción JVM a la vez, haciendo dos o tres referencias a la memoria en cada instrucción, sería un desastre en términos de desempeño. Sin embargo, al colocar una pila de 64 palabras en el chip y plegar muchas instrucciones para convertir sucesiones enteras en instrucciones RISC modernas de tres direcciones, el picoJava II logra tomar una ISA muy ineficiente y trabajar bien con ella.

El criterio definitivo aquí es que el corazón de una computadora moderna es una máquina RISC de carga/almacenamiento, de tres registros, con filas de procesamiento. El UltraSPARC II simplemente expone la máquina al usuario. El Pentium II la oculta tomando una ISA más vieja y dividiendo las instrucciones CISC en microoperaciones RISC que se pueden ejecutar de forma eficiente. El picoJava II también oculta la máquina RISC que está en su centro, pero lo hace combinando varias instrucciones ISA en una sola instrucción RISC, exactamente lo contrario de lo que el Pentium II hace.

En conclusión, si Ricitos de Oro se encontrara con algunas instrucciones de computadora en medio del bosque, vería que algunas son demasiado grandes (las instrucciones del Pentium II tienen que partirse); otras son demasiado pequeñas (las instrucciones del picoJava II tienen que combinarse) y unas más son justo del tamaño correcto (las instrucciones del UltraSPARC II se ejecutan tal cual). Ésta es la Teoría de los Tres Osos de la Computación.

5.6 FLUJO DE CONTROL

El término “flujo de control” se refiere al orden en que las instrucciones se ejecutan dinámicamente, es decir, durante la ejecución del programa. En general, si no hay ramificaciones y llamadas a procedimientos, las instrucciones que se ejecutan sucesivamente se traen de posiciones de memoria consecutivas. Las llamadas a procedimientos hacen que el flujo de control se altere, deteniendo el procedimiento que se está ejecutando e iniciando el procedimiento invocado. Las corrutinas están relacionadas con los procedimientos y causan alteraciones similares en el flujo de control; son útiles para simular procesos paralelos. Las trampas e interrupciones también hacen que se altere el flujo de control cuando se presentan condiciones especiales. Todos estos temas se tratarán en las secciones que siguen.

5.6.1 Flujo de control secuencial y ramificaciones

La mayor parte de las instrucciones no altera el flujo de control. Después de ejecutarse una instrucción, se trae la que le sigue en la memoria, y se ejecuta. Después de cada instrucción, el contador de programa se incrementa en una cantidad igual a la longitud de la instrucción. Si observamos este contador durante un intervalo largo en comparación con el tiempo medio de una instrucción, vemos que es una función aproximadamente lineal del tiempo, pues se incrementa en la longitud media de una instrucción en el tiempo medio de una instrucción. Dicho de otro modo, el orden dinámico en que el procesador realmente ejecuta las instrucciones es igual al orden en que aparecen en el listado del programa, como se muestra en la figura 5-37(a).

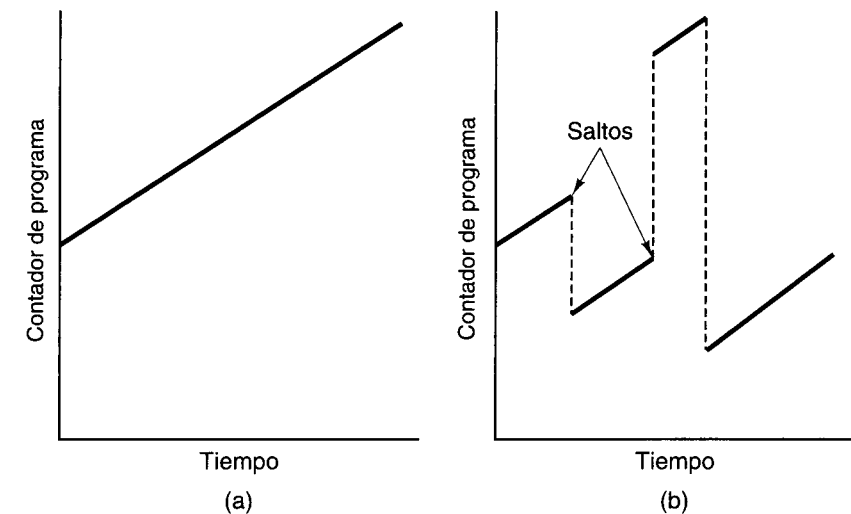


Figura 5-37. El contador de programa en función del tiempo (curva suavizada). (a) Sin ramificaciones. (b) Con ramificaciones.

Si un programa contiene ramificaciones, esta sencilla relación entre el orden en que las instrucciones aparecen en la memoria y el orden en que se ejecutan deja de ser real. Cuando hay ramificaciones, el contador de programa deja de ser una función monótonicamente creciente del tiempo, como se muestra en la figura 5-37(b). El resultado es que no es fácil visualizar la secuencia de ejecución de las instrucciones a partir del listado del programa. Cuando los programadores tienen dificultades para ver claramente el orden en que el procesador ejecutará las instrucciones, tienden a cometer errores. Esta observación llevó a Dijkstra (1986a) a escribir la carta, que tanta controversia causó, intitulada “El enunciado **GOTO** se considera perjudicial”, en la que sugería evitar los enunciados **goto**. Esa carta dio origen a la revolución de la programación estructurada, uno de cuyos preceptos es la sustitución de enunciados **goto** por formas más estructuradas de control de flujo, como los ciclos **while**. Desde luego, la compilación de estos programas produce programas de nivel 2 que podrían contener muchas ramificaciones, porque la implementación de **if**, **while** y otras estructuras de control de alto nivel requiere ramificaciones.

5.6.2 Procedimientos

La técnica más importante para estructurar programas es el procedimiento. Desde un punto de vista, una llamada a un procedimiento altera el flujo de control como lo hace una ramificación pero, a diferencia de ésta, cuando el procedimiento termina su tarea devuelve el control al enunciado o instrucción que sigue a la llamada.

Desde otro punto de vista, el cuerpo de un procedimiento puede verse como la definición de una nueva instrucción en un nivel más alto. Desde esta perspectiva, una llamada a un procedimiento puede verse como una sola instrucción, aunque el procedimiento sea muy complicado. Para entender un fragmento de código que contiene una llamada a un procedimiento, basta con saber *qué* hace, no *cómo* lo hace.

Un tipo de procedimiento especialmente interesante es el **procedimiento recursivo**, es decir, un procedimiento que se llama a sí mismo, sea directamente o por medio de una cadena de procedimientos distintos. El estudio de procedimientos recursivos ayuda mucho a entender cómo se implementan las llamadas a procedimientos, y qué son realmente las variables locales. A continuación veremos un ejemplo de procedimiento recursivo.

Las "Torres de Hanoi" son un antiguo problema que tiene una solución recursiva sencilla. En cierto monasterio de Hanoi hay tres varillas de oro. Ensayadas en la primera había una serie de 64 discos de oro concéntricos, cada uno con un agujero en el centro para la varilla. Cada disco tiene un diámetro un poco menor que el del disco que está inmediatamente abajo. Las varillas segunda y tercera estaban vacías inicialmente. Los monjes del lugar están ocupados transfiriendo todos los discos a la varilla 3, un disco a la vez, sólo que en ningún momento un disco mayor puede descansar sobre uno más pequeño. Se dice que cuando los monjes terminen, el mundo llegará a su fin. Si quiere adquirir algo de experiencia práctica, puede usar discos de plástico y menos discos, pero cuando resuelva el problema no sucederá nada. Para lograr el efecto de fin-del-mundo se necesitan 64 discos, y de oro. La figura 5-38 muestra la configuración inicial para $n = 5$ discos.

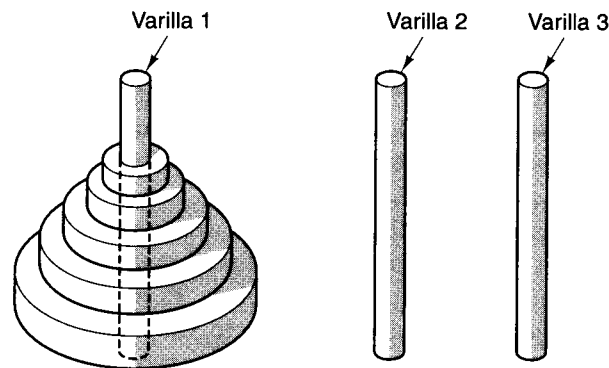


Figura 5-38. Configuración inicial del problema de las Torres de Hanoi con cinco discos.

La solución para transferir n discos de la varilla 1 a la 3 consiste en trasladar primero $n - 1$ discos de la varilla 1 a la 2, luego trasladar un disco de la varilla 1 a la 3, y por último trasladar $n - 1$ discos de la varilla 2 a la 3. Esta solución se ilustra en la figura 5-39.

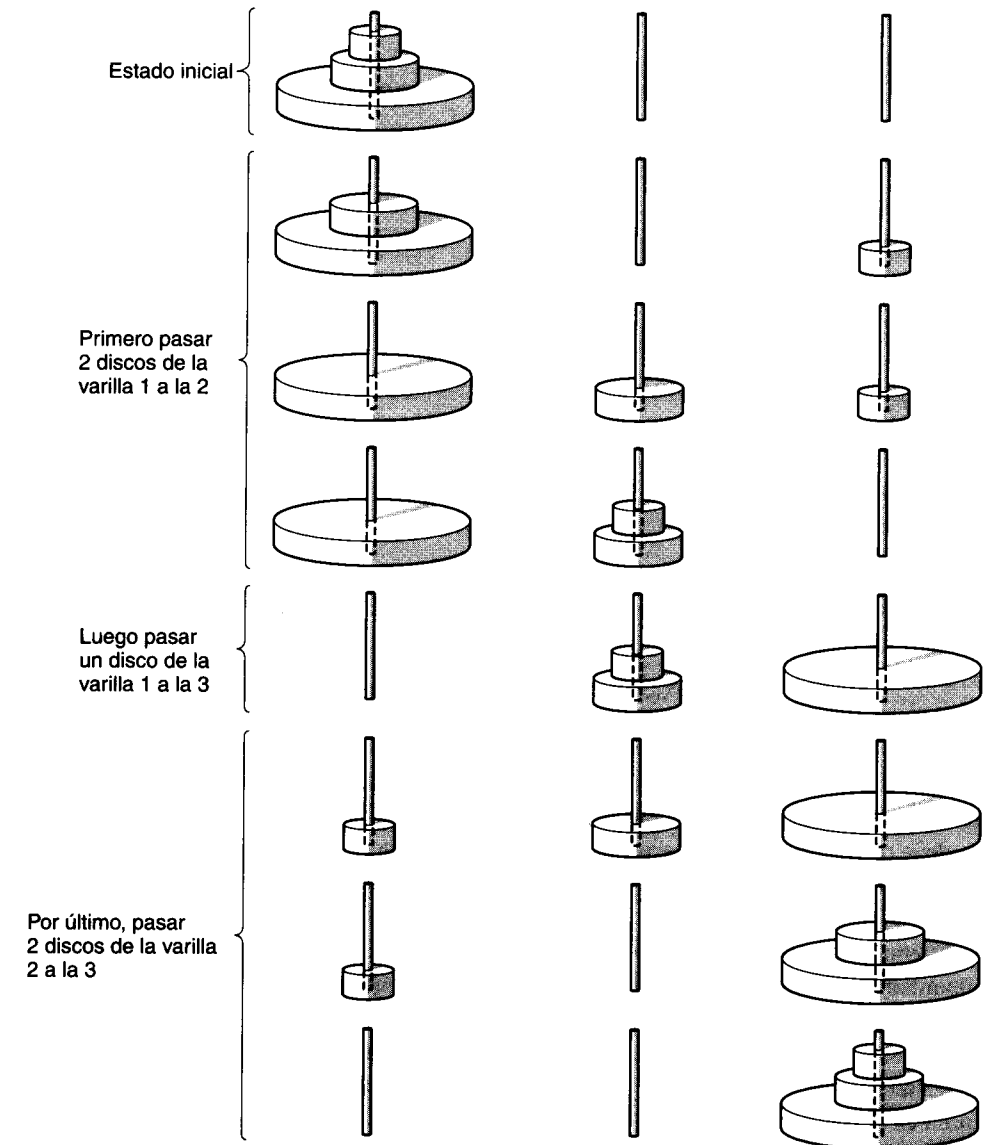


Figura 5-39. Pasos necesarios para resolver las Torres de Hanoi con tres discos.

Para resolver el problema necesitamos un procedimiento que traslade n discos de la varilla i a la varilla j . Cuando se invoque este procedimiento, con $\text{torres}(n, i, j)$

se imprimirá la solución. El procedimiento primero prueba si $n = 1$. Si es así, la solución es trivial. Basta con pasar el disco de i a j . Si $n \neq 1$, la solución consiste en las tres partes que ya mencionamos, cada una de las cuales es una llamada a un procedimiento recursivo.

La solución completa se muestra en la figura 5-40. La llamada

`torres(3, 1, 3)`

para resolver el problema de la figura 5-39 genera tres llamadas más, a saber,

`torres(2, 1, 2)`

`torres(1, 1, 3)`

`torres(2, 2, 3)`

La primera y la tercera generan tres llamadas cada una, para un total de siete.

```
public void torres (int n, int i, int j) {
    int k;

    if (n==1)
        System.out.println("Pase un disco de " + i + " a " + j);
    else {
        k = 6 - i - j;
        torres(n - 1, i, k);
        torres(1, i, j);
        torres(n - 1, k, j);
    }
}
```

Figura 5-40. Procedimiento para resolver las Torres de Hanoi.

Para tener procedimientos recursivos necesitamos una pila en la cual guardar los parámetros y variables locales de cada invocación, como la que teníamos en IJVM. Cada vez que se invoca un procedimiento, se asigna un nuevo marco de pila para el procedimiento en el tope de la pila. El marco de creación más reciente es el marco vigente. En nuestros ejemplos la pila crece hacia arriba, de direcciones de memoria baja a direcciones altas, igual que en IJVM.

Además de apuntador de pila, que apunta al tope de la pila, suele ser conveniente tener un apuntador de marco, *FP* (*Frame Pointer*), que apunta a una posición fija dentro del marco. Podría apuntar al apuntador de enlace, como en IJVM, o a la primera variable local. La figura 5-41 muestra el marco de pila para una máquina con palabras de 32 bits. La llamada original a *torres* mete n , i y j en la pila y luego ejecuta una instrucción **CALL** que mete la dirección de retorno en la pila, en la dirección 1012. Al entrar, el procedimiento invocado guarda el valor viejo de *FP* (1000) en la pila en 1016 y luego adelanta el apuntador de pila a modo de asignar espacio para las variables locales. Ya que sólo hay una variable local de 32 bits (k), *SP* se incrementa en 4, a 1020. En la figura 5-41(a) se muestra la situación después de hacerse todas estas cosas.

Lo primero que un procedimiento debe hacer cuando se le invoca es guardar el *FP* anterior (para poder restaurarlo cuando se salga del procedimiento), copiar *SP* en *FP*, y tal vez

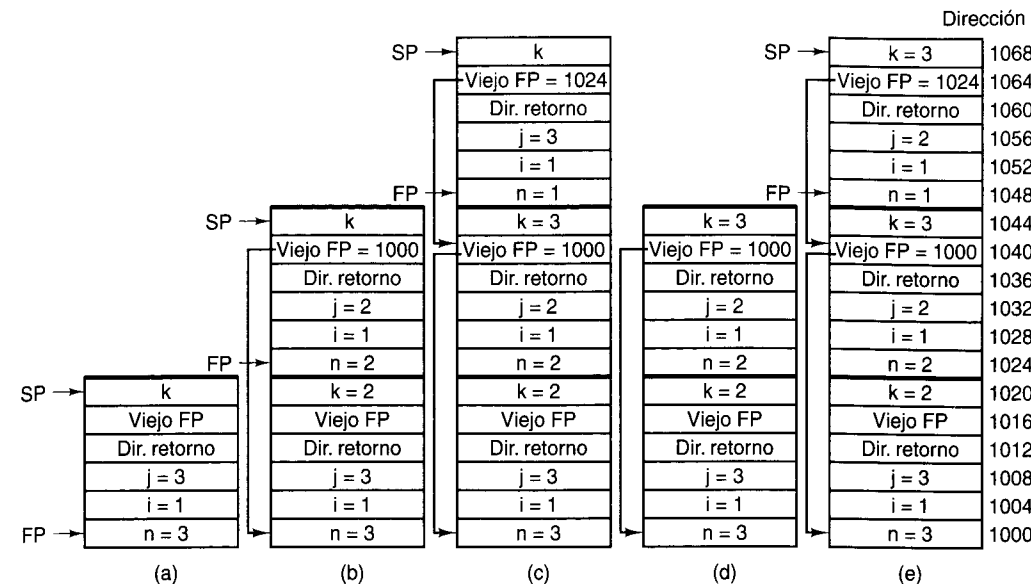


Figura 5-41. La pila en varios momentos durante la ejecución de la figura 5-40.

incrementarlo en una palabra, dependiendo de la parte del nuevo marco a la que *FP* apunta. En este ejemplo, *FP* apunta a la primera variable local, pero en IJVM, *LV* apuntaba al apuntador de enlace. Las diferentes máquinas manejan el apuntador de marco de forma un poco distinta; a veces lo colocan en la base del marco de pila, a veces hasta arriba y a veces a la mitad como en la figura 5-41. En este sentido, vale la pena comparar la figura 5-41 con la figura 4-12 para ver dos formas distintas de manejar el apuntador de enlace. Éstas no son las únicas formas. En todos los casos, la clave es la capacidad para efectuar después un retorno de procedimiento y restaurar la pila al estado que tenía justo antes de la llamada a procedimiento en curso.

El código que guarda el apuntador de marco viejo, ajusta el nuevo apuntador de marco y adelanta el apuntador de pila a fin de reservar espacio para las variables locales se llama **prólogo del procedimiento**. Cuando se sale del procedimiento, es preciso volver a limpiar la pila, es lo que se conoce como **epílogo del procedimiento**. Una de las características más importantes de cualquier computadora es cuán cortos y rápidos puede hacer al prólogo y al epílogo. Si son largos y lentos, las llamadas a procedimiento serán costosas. Los programadores que veneran la eficiencia aprenden a evitar la escritura de muchos procedimientos cortos y prefieren escribir programas grandes, monolíticos, no estructurados. Las instrucciones **ENTER** y **LEAVE** del Pentium II se diseñaron a modo de efectuar la mayor parte de las tareas del prólogo y epílogo de los procedimientos de manera eficiente. Desde luego, esas instrucciones tienen un modelo específico de la forma en que debe manejarse el apuntador de marco, y si el compilador tiene un modelo diferente, no pueden usarse.

Volvamos al problema de las Torres de Hanoi. Cada llamada de procedimiento añade un nuevo marco a la pila, y cada retorno de procedimiento elimina un marco de la pila. A fin de

ilustrar el uso de una pila al implementar procedimientos recursivos, seguiremos la pista a las llamadas, comenzando con

`torres(3, 1, 3)`

La figura 5-41(a) muestra la pila inmediatamente después de que se efectúa esta llamada. Lo primero que hace el procedimiento es probar si $n = 1$, y al descubrir que $n = 3$ sustituye k por su valor y efectúa la llamada

`torres(2, 1, 2)`

Una vez finalizada esa llamada, la pila está en el estado que se muestra en la figura 5-41(b) y el procedimiento vuelve a comenzar desde el principio (un procedimiento invocado siempre inicia en el principio). Esta vez la prueba de $n = 1$ falla también, por lo que se vuelve a sustituir k y se efectúa la llamada

`torres(1, 1, 3)`

La pila en ese momento tiene el aspecto que se muestra en la figura 5-41(c) y el contador de programa apunta al principio del procedimiento. Esta vez la prueba tiene éxito y se imprime una línea. Luego, el procedimiento regresa eliminando un marco de pila y restableciendo FP y SP a la figura 5-41(d). A continuación, el procedimiento continúa su ejecución en la dirección de retorno, que es la segunda llamada:

`torres(1, 1, 2)`

Esto añade un nuevo marco a la pila como se muestra en la figura 5-41(e). Se imprime otra línea; después del retorno se elimina un marco de la pila. Las llamadas a procedimientos continúan de esta manera hasta que termina de ejecutarse la llamada original y el marco de la figura 5-41(a) se elimina de la pila. Para entender mejor cómo funciona la recursión, se recomienda al lector simular la ejecución completa de

`torres(3, 1, 3)`

usando lápiz y papel.

5.6.3 Corrutinas

En la secuencia de invocación normal, hay una distinción clara entre el procedimiento que invoca y el procedimiento invocado. Considere un procedimiento A, que llama a un procedimiento B en la figura 5-42.

El procedimiento B calcula durante un rato y luego regresa a A. A primera vista podríamos pensar que esta situación es simétrica, porque ni A ni B son el programa principal; ambos son procedimientos. (El procedimiento A pudo haber sido llamado por el programa principal, pero eso no viene al caso.) Además, el control se transfiere primero de A a B —la llamada— y luego se transfiere de B a A —el retorno.

La asimetría surge del hecho de que cuando el control pasa de A a B, el procedimiento B comienza a ejecutarse al principio; cuando B regresa a A, la ejecución no inicia al principio de

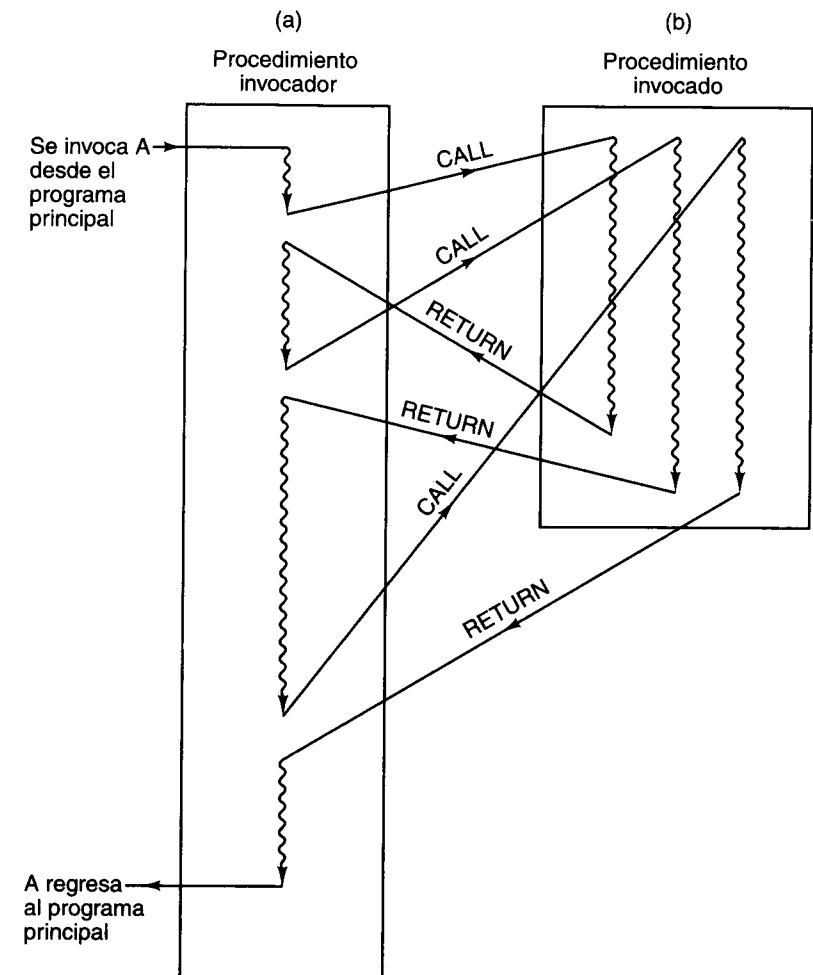


Figura 5-42. Cuando se invoca un procedimiento, la ejecución del procedimiento siempre inicia en el primer enunciado del procedimiento.

A, sino en el enunciado que sigue a la llamada. Si A trabaja durante un rato y luego vuelve a invocar a B, la ejecución inicia otra vez al principio de B, no en el enunciado que sigue al retorno anterior. Si, durante su ejecución, A llama a B muchas veces, B iniciará al principio en cada ocasión, mientras que A nunca vuelve a iniciar.

Esta diferencia se refleja en el método por el cual se transfiere el control entre A y B. Cuando A llama a B, usa la instrucción de llamada a procedimiento, la cual coloca la dirección de retorno (es decir, la dirección del enunciado que sigue a la llamada) en algún lugar útil; digamos, en el tope de la pila. Luego, la instrucción coloca la dirección de B en el contador de programa para completar la llamada. Cuando B regresa, no usa la instrucción de llamada, sino la de retorno, que simplemente saca la dirección de retorno de la pila y la coloca en el contador de programa.